Using AgentCore Runtime in an Agentic RL Workflow

Hands-on guidance, grounded in a working slime + AgentCore project

Kaige Yang — Senior Applied Scientist, AWS

September 10, 2026

Contents

1 Introduction **2**

2 Agentic RL and its reference architecture **4**
2.1 Where agentic RL is used . . . 6
2.2 The harness — the agent loop that owns the episode . . . 7
2.3 The proxy — every model call is the policy under training . . . 9
2.4 Token-in-token-out (TITO) — the proxy's core responsibility . . . 11
2.5 When one episode becomes many samples — subagents, summarization, and advantage assignment . . . 13
2.6 slime — the trainer, and its asynchronous rollout . . . 17
2.7 Scaling the loop — inference, harness, and sandbox in balance . . . 17

3 Tasks frontier labs run in a sandbox **25**
3.1 What the frontier reports describe . . . 26
3.2 Task families in detail . . . 27
3.2.1 Single-turn, generation-bound families . . . 28
3.2.2 Iterative, few-turn families . . . 29
3.2.3 Multi-turn, observation-heavy families . . . 29
3.2.4 Long-horizon, session-bound families . . . 30
3.3 Language and environment per family — and how one microVM serves them all . . . 31
3.4 The one family that stresses the image: SWE and the microVM's arm64 + 2 GB limits . . . 35
3.5 One image per Runtime, but a different env per row: how AgentCore reconciles it . . . 37

4 Foundations of AgentCore Runtime **38**
4.1 The microVM at a glance — what it is, what runs in it, what it supports, how you customize it . . . 38
4.2 Runtime, session, and microVM — three concepts, precisely . . . 39
4.2.1 A mixed-family batch — how many Runtimes, sessions, and microVMs, and who picks the image? . . . 41
4.3 microVM vs a Docker container vs E2B . . . 42
4.4 Two compute types — managed serverless (microVM) vs Instances (GPU EC2) . . . 43

4.5 How AgentCore manages the lifecycle — Runtime, session, microVM, and GPU instance, end to end . . . 46
4.6 Building a customized Runtime — bring your own container, end to end 50

5 Using AgentCore as the sandbox in an agentic RL workflow 52
5.1 Where AgentCore attaches to slime — hooks, not a fork . . . 53
5.2 The call path — one diagram . . . 53
5.3 Key code — three pieces . . . 53
5.4 Who controls the AgentCore lifecycle — deploy driver vs. the hook . . . 56
5.5 Ending the session at episode end — explicit teardown, and what "end" means per substrate . . . 58
5.6 Warm instance shared by session — one hot Instance, a fresh env per episode . . . 59
5.7 One harness, many task families . . . 61
5.8 Reading the episode shape into a config . . . 62
5.9 The episode, end to end . . . 62
5.10 Scaling, sharing, and network isolation . . . 63
5.11 Can AgentCore host every frontier sandbox? . . . 64

6 Demos — the storyline at a glance 65
6.1 Chapter 1 — the tasks: what the literature trains in a sandbox . . . 68
6.2 Chapter 2 — most tasks train on the default runtime (`demo_cpu_tasks`) 70
6.3 Chapter 3 — custom images at scale (`demo_cpu_terminalbench`) . . . 73
6.4 Chapter 4 — hiding the container launch: pre-open and heartbeat . . . 75
6.5 Chapter 5 — a warm CPU Instance pool shared by isolated sessions (`demo_cpu_instance_pool`) . . . 77
6.5.1 Isolation on a shared m5 — what is guaranteed, and what is not 78
6.6 Chapter 6 — scaling the rollout on decoupled tiers (`demo_cpu_scale_lanes`) 82
6.7 Chapter 7 — GPU sandboxes: one GPU, a fraction of a GPU, a whole GPU per episode . . . 84
6.7.1 `demo_gpu_single` — one warm custom GPU Runtime . . . 84
6.7.2 `demo_gpu_farm_fractional` — two episodes on one GPU . . . 85
6.7.3 `demo_gpu_farm_whole` — one whole GPU per episode, on one shared instance . . . 86
6.7.4 GPU isolation — what a shared GPU instance guarantees . . . 87
6.8 Chapter 8 (experimental) — one episode, many samples: `demo_cpu_branching` 88

7 Demos in depth — architecture, pipeline, lifecycle, and measured results 89
7.1 The shared skeleton — one pipeline, one set of metrics . . . 90
7.2 Sizing the config — response length per turn, and turns per episode . . 94
7.3 Experiment plan — training each demo to convergence . . . 95
7.4 `demo_cpu_tasks` in depth — `DATASET=mbpp` (MBPP, the default) . . . 98
7.5 `demo_cpu_tasks` in depth — `DATASET=math` (math word problems) . . . 99
7.6 `demo_gpu_single` in depth — KernelBench on a GPU Runtime instance 100
7.7 `demo_gpu_farm_fractional` and `demo_gpu_farm_whole` in depth — packing episodes onto GPUs . . . 101
7.8 `HEARTBEAT=1` and `demo_gpu_single` `SNAPSHOT_EVERY=1` in depth — surviving the pause and the recycle . . . 104
7.9 `demo_cpu_scale_lanes` in depth — 4 SGLang engines : 256 harness workers : 256 microVMs . . . 105
7.10 `demo_cpu_tasks` per dataset in depth — the eight other real-dataset values . . . 108
7.10.1 `DATASET=gsm8k` — grade-school math word problems . . . 108
7.10.2 `DATASET=math500` — competition-style math . . . 109
7.10.3 `DATASET=deepscaler` — RL-scale math (largest split) . . . 109
7.10.4 `DATASET=humaneval` — function-synthesis coding . . . 109
7.10.5 `DATASET=gretelsql` — text-to-SQL . . . 110
7.10.6 `DATASET=hotpotqa` — multi-hop RAG . . . 110
7.10.7 `DATASET=xlam` — function-calling / tool use . . . 110
7.10.8 `DATASET=mcp` — MCP-style tool schemas . . . 111
7.11 `demo_cpu_tasks` mixes in depth — one interleaved multi-family stream 111

8 Open problems in production agentic RL — infrastructure and algorithms 112
8.1 Infrastructure and systems . . . 113
8.2 Algorithms . . . 114
8.3 Takeaways . . . 116

9 Summary 116

10 References — papers & models 117

11 References — AgentCore docs (1/2) 118

12 References — AgentCore docs (2/2) 118

13 References — AgentCore SDK, CLI & samples **118**

14 References — production agentic RL (open problems) **118**

1 Introduction

This report is a **hands-on guide for AWS customers doing agentic RL**. It is grounded in a working slime + AgentCore project and in the **official AgentCore docs** [D1].

After reading it, you will know:

- **The reference architecture of agentic RL** — how the trainer, inference engine, proxy, harness, and sandbox fit together, and which seams AgentCore fills (§"Agentic RL and its reference architecture").
- **What tasks to train, and how the reward works** — the verifiable task families frontier labs run in a sandbox, each scored by a programmatic **RLVR** checker, not an LLM judge (§"Tasks frontier labs run in a sandbox").
- **What AgentCore can and cannot do** as an RL sandbox (and, optionally, harness), and **how to build and customize** your own Runtime — CPU (Code Interpreter) and GPU (Instances) (§"Foundations of AgentCore Runtime").
- **What AgentCore *is*, precisely** — the three concepts you must not conflate (**Runtime**, **session**, **microVM**), their **1 : N : 1** cardinality, the two compute types (serverless CPU microVM vs GPU **Instances**), and how the **lifecycle** works — implicit launch on first invoke, the **~15-minute idle timer**, and implicit termination on idle / max-lifetime / failed health check (§"Foundations of AgentCore Runtime").
- **How to run a real task on it** — datasets, Dockerfiles, deploy commands, and the single slime hook that wires AgentCore in without a fork (§"Using AgentCore as the sandbox in an agentic RL workflow"). The project ships **seven main demos plus one experimental** (`demo_cpu_branching`). Each is a self-contained folder run through the same six numbered step scripts (`1_prepare_data.sh` … `6_cleanup.sh`), and together they tell one storyline (§"Demos — the storyline at a glance"):

  1. the tasks the literature trains in a sandbox;
  2. most of them train on the **default** AgentCore runtime, the managed Code Interpreter, with **no image** (`demo_cpu_tasks`);
  3. custom images at scale, one per task (`demo_cpu_terminalbench`);
  4. hiding the container launch with **pre-open and heartbeat** (its `warm-start/` probe);
  5. a warm CPU Instance pool shared by sessions, **with isolation** (`demo_cpu_instance_pool`);

6. scaling the rollout on decoupled tiers (`demo_cpu_scale_lanes`);
7. three GPU demos, with **GPU isolation**;
8. the experimental branching demo.

The report is plain about evidence: two demos carry convergence-shaped curves, and the rest are loop-verified 10-step smokes on real AgentCore hardware — a *capability* demonstration, not a convergence study.

- **How to train agentic RL on a GPU Runtime** — an AgentCore Runtime **Instance backed by GPU-based EC2** (g6 / L4), where untrusted CUDA builds and runs against a checker on a disposable GPU: one GPU per session (`demo_gpu_single`), two episodes on a fraction of one GPU (`demo_gpu_farm_fractional`), or one whole GPU per episode on a shared instance (`demo_gpu_farm_whole`).
- **How to scale the loop** — measure the per-step bottleneck, then scale the three tiers **jointly**: inference, the sandbox, **and the harness itself**, because rollout throughput is `min(inference, sandbox, harness)`; and **balance load correctly per side** — a cache-aware SGLang router for inference, sticky round-robin by session id for the sandbox pool (§"Scaling the loop").
- **How to hide the sandbox cold start** — pre-open the next one or two batches' sessions, and keep idle own-Runtime sessions alive across a weight sync with a **heartbeat** (`HEARTBEAT=1`, an opt-in knob of every demo; always on in `demo_cpu_terminalbench`). On the Code Interpreter a heartbeat has no effect; the session timeout is the lever.
- **How isolation is guaranteed when sessions share one instance** — a per-episode Landlock jail (or uid where capabilities allow), private directory, resource limits and kill-and-scrub on a shared `m5` or GPU instance, and exactly what still shares a kernel (Chapters 5 and 7).
- **How to snapshot and resume at the application level** on a GPU-EC2 Runtime Instance — a **Dockerfile + a custom script, triggered by the weight-sync event**, checkpoints application state to a store you own, so training resumes after a recycle from the saved snapshot (`demo_gpu_single` with `SNAPSHOT_EVERY=1`).
- **Where the open problems are** — the infrastructure and algorithmic questions production agentic RL still faces (§"Open problems in production agentic RL").

Scope. AgentCore = the execution **sandbox**; **slime** = the RL trainer; the **harness stays in-process** (warm, next to the trainer) — though the report shows the harness is itself a scaling dimension, replicated as warm workers rather than made

ad-hoc. The through-line is the sandbox seam — build image → deploy Runtime → seed a task → invoke → read the reward — and then scaling the loop around it and surviving its stalls.

Roadmap. The report proceeds in order: the **reference architecture** of agentic RL and how slime drives it (§2); the **verifiable task families** and their rewards (§3); the **foundations of AgentCore Runtime** — concepts, compute types, and lifecycle (§4); **wiring AgentCore into slime** as the sandbox, with the call path and key code (§5); the **demo storyline** in eight chapters (§6) and then **in depth** with measured results (§7); the **open problems** that remain (§8); and a closing **summary** of what AgentCore can and cannot do (§9).

2 Agentic RL and its reference architecture

What is agentic RL? In *agentic* reinforcement learning the model under training is not asked for a single answer — it acts as an **agent** over many turns against an **executable environment**: it writes code, runs a command, reads the result, and tries again. Only when the trajectory ends does a **programmatic checker** turn the outcome into a scalar reward (tests pass, result-set matches, flag verified) — **no LLM judge** [1, 2]. This is reinforcement learning with *verifiable* rewards (RLVR): the environment, not a model, decides who was right.

Training that loop end-to-end uses a **disaggregated** design (following Agent Lightning [1]): a training side (trainer + inference engine) and an agent side (harness + sandbox), joined by an LLM-proxy / rollout-collection gateway. The reference architecture below traces the full cycle — a **generation path** across the top and a **training-data path** back along the bottom.

The core components.

- **Trainer** — the learner (here **slime** driving **Megatron**, on GPU). It samples prompts, collects finished trajectories with their rewards, computes the policy-gradient update, and periodically pushes fresh weights to the inference engine.
- **Inference engine** — the generation server (here **SGLang**, on GPU) that produces the model's turns at rollout speed. It holds the *current* policy weights; the trainer refreshes them via **weight sync** so rollouts always reflect the latest policy.

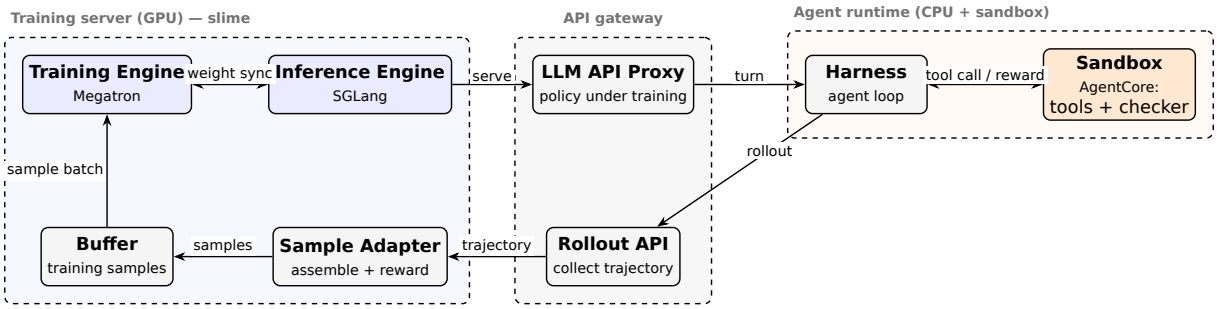


Figure 1: Reference architecture for (harnessed) agentic RL, following the disaggregated design of Agent Lightning [1] with slime [9] as the training server. Three domains: a **training server** (GPU), an **API gateway**, and an **agent runtime**. **Generation lane (top, →):** the training engine syncs weights to the inference engine; the LLM API proxy fronts inference so every turn is served by the policy under training; the harness drives the agent loop; the sandbox (AgentCore) runs each tool call and the checker. **Training-data lane (bottom, ←):** the rollout is collected via the Rollout API, the Sample Adapter assembles it from the recorded token IDs (no re-tokenization — the TITO property) and attaches the reward to form training samples, which accumulate in the Buffer; the trainer draws a sample batch and updates — closing the loop.

- **Proxy** — a thin adapter (CPU) fronting the inference engine so that every model call the harness makes is answered by the **policy under training** (not some external API). It is the seam that lets an off-the-shelf agent loop drive an in-training model.
- **Harness** — the **agent loop** (CPU): it holds the conversation, calls the proxy for each turn, extracts the model's proposed action (a code block, a tool call), submits it to the sandbox, and feeds the result back as the next observation until the episode ends.
- **Sandbox** — the **isolated execution environment** (CPU, or GPU for CUDA work) where the model's untrusted output actually runs (write → run → read) and where the **checker scores the reward**. It must be ad-hoc and isolated per episode.

The training-data path. The bottom of Figure 1 shows how a finished episode becomes a gradient step. The **Rollout API** collects the trajectory (the sequence of LLM request–response pairs the harness made); the **Sample Adapter** assembles those calls from their recorded token IDs (no re-tokenization — §"Token-in-token-out") and attaches the episode's reward and advantage, turning one rollout into

one or more **training samples**; those land in the **Buffer**; and the trainer draws a **sample batch** from the buffer for each update. In *harnessed* agentic RL the training engine never sees the environment directly — only this stream of samples reconstructed from the proxy's request/response log [1].

2.1 Where agentic RL is used

The same loop — act in an environment, score with a checker — covers a wide range of tasks. The common thread is a **verifiable reward**: an environment (not a human or an LLM judge) decides success. Representative use cases:

- **Software engineering (SWE)** — edit a real repository to resolve an issue; reward = the project's test suite turns green (e.g. SWE-bench [6]).
- **Code generation & competitive programming** — write a function that passes hidden unit tests (e.g. MBPP [8]).
- **GPU-kernel / systems** — write a CUDA kernel and optimize it; reward = numerical `allclose` plus a speedup target (e.g. KernelBench [7]).
- **Math & logical reasoning** — produce an answer checked for exact/numeric equality against ground truth.
- **Tool use / function calling** — orchestrate multi-step API or tool calls; reward = correct final result *and* the right tools invoked.
- **SQL & data analysis** — generate a query whose result-set matches the expected rows.
- **RAG & search agents** — retrieve then answer; reward = answer grounded in the retrieved facts.
- **Web & computer use** — drive a browser or GUI to complete a task.
- **Security / CTF** — exploit a challenge and capture the flag; reward = the checker validates the flag.
- **Multi-agent & long-horizon workflows** — several agents or many turns cooperate toward a verifiable end state.

Every one of these needs an **isolated place to execute untrusted output and run the checker** — the sandbox seam this report is about.

2.2 The harness — the agent loop that owns the episode

What it is. The harness is the **agent loop**: the orchestration code that runs *one* episode end to end. In this project it is `run_episode` (in `harness.py`), and it is **task-agnostic** — the same loop serves every task family; only the task's prompt, seed files, and checker change. We keep it **in-process** (a warm Python coroutine next to the trainer), because episodes are independent and the loop needs *warmth*, not isolation — that isolation belongs to the sandbox.

How it works with the other components. The harness sits between the proxy and the sandbox and is the only component that sees a whole episode:

- **↔ Proxy:** for each turn it sends the running conversation to the proxy and receives the model's response (the policy under training).
- **↔ Sandbox:** it extracts the model's proposed action (a code block, a tool call) and executes it in the sandbox, then reads back stdout / exit-code as the next observation.
- **↔ Task / checker:** it seeds the workspace from the task and, at the end, runs the checker (in the sandbox) to get the reward.
- **→ Rollout / Sample Adapter:** the trajectory it produces (the proxy-logged calls + the final reward) becomes the training sample.

Step-by-step example — one MBPP episode from the harness's view.

```
async def run_episode(task, policy, sandbox):          # one episode
    # 1. SEED: write the problem + the checker into the fresh sandbox
    for name, content in task.setup_files():           # e.g. PROBLEM.md
        await sandbox.write_file(name, content)
    await sandbox.write_file("run_checks.py", task.runner_source())

    messages = task.initial_messages()                 # system + PROBLEM.md
    solved = False
    for _ in range(task.max_turns):                    # 2. TURN LOOP
        text = await policy.complete(messages)         #    (a) ask proxy -> policy
↪   turn
        code = task.extract_submission(text)           #    (b) parse the action
↪   (fenced code)
        await sandbox.write_file("solution.py", code)  #    (c) place the artifact
        exit_code, out, err = await sandbox.exec(       #    (d) run the checker IN
↪   the sandbox
            "python3 run_checks.py", timeout=60)
        if exit_code == 0:                              #    (e) checker passed?
            solved = True
```

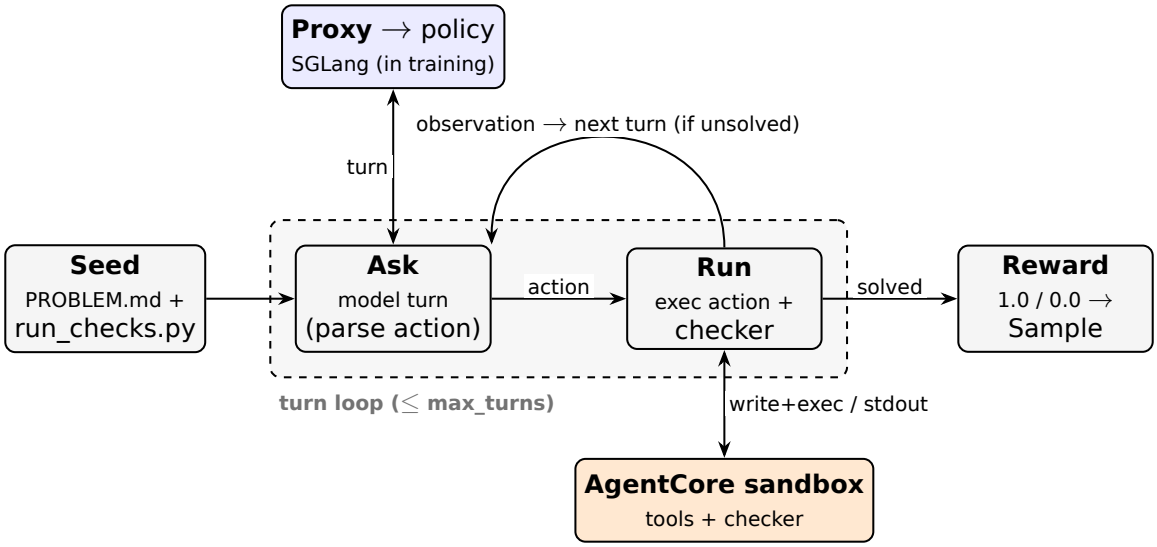


Figure 2: The harness's episode loop (`run_episode`). It **seeds** the sandbox once, then repeats a **turn loop**: ask the proxy for a model turn, parse the proposed action, run it (and the checker) in the AgentCore sandbox, and feed the observation back for the next turn. The loop exits when the checker passes or `max_turns` is reached; the final **reward** and the loss-masked trajectory become one training `Sample`. The proxy (policy under training) and the sandbox are the only two services the loop calls each turn.

```
            break
        messages.append(task.observation(out, err))    #    (f) feed the failure
↪  back, loop

    reward = 1.0 if solved else 0.0                    # 3. REWARD (from the checker)
    return build_token_stream(messages, reward)        # 4. loss-masked trajectory ->
     ↪  Sample
```

Walking it: the harness **seeds** the sandbox (step 1), then repeats **ask → parse → run → observe** (step 2a–f), each "ask" going through the proxy to the in-training policy and each "run" going to the AgentCore sandbox. When the checker exits `0` the episode is **solved**; otherwise the failure output is appended and the model tries again, up to `max_turns`. The final **reward** (step 3) comes from the checker — not a judge — and the harness hands the loss-masked trajectory to the training-data path (step 4). The harness owns the loop; the proxy and sandbox are the two services it calls each turn.

2.3 The proxy — every model call is the policy under training

What it is. The proxy is a thin CPU service that exposes a **standard LLM API** (in this project an Anthropic-Messages–compatible `AnthropicAdapter`, but an OpenAI-compatible endpoint works the same) and routes every request to the **in-training inference engine** (SGLang) instead of to any external provider. It is the single seam that lets an *off-the-shelf agent harness* — one written to call a normal LLM API — drive a model that is still being trained.

What it does. Three jobs, all on the critical path of every turn:

1. **Substitute the endpoint.** The harness thinks it is calling a hosted LLM; the proxy answers with the **policy under training**, holding the latest synced weights. No harness change is needed to point it at the trainer.
2. **Record the exact tokens.** For each call the proxy captures the **prompt token IDs and the sampled response token IDs** (and log-probs). The training engine only ever observes this stream of request/response pairs — so recording the *exact* sampled tokens is what avoids the retokenization/decode-drift problem in harnessed agentic RL [1].
3. **Bucket by episode.** Each request is tagged with the episode's `session_id`, so all calls of one rollout are grouped into the right trajectory before the Sample Adapter turns them into training samples.

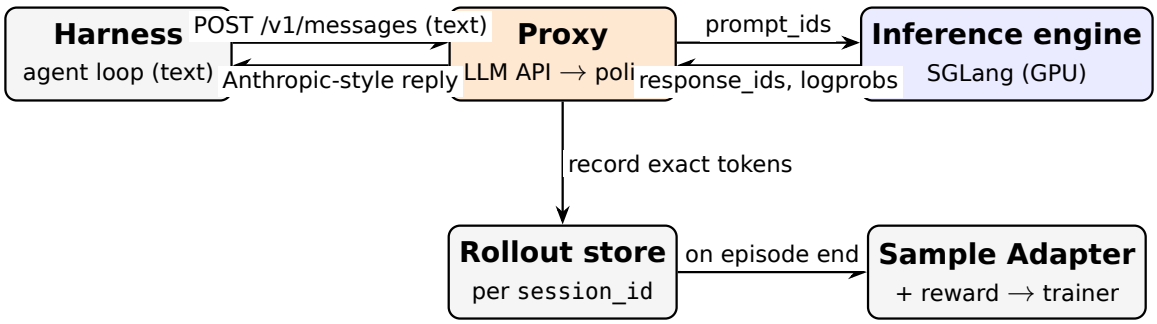


Figure 3: The proxy on the seam between the text-based harness and the token-based inference engine. It **substitutes** the endpoint (the harness's ordinary LLM API call is answered by the policy under training), and it **records** the exact prompt and response token IDs (with log-probs) into a per-`session_id` rollout store. On episode end the Sample Adapter reads that store, attaches the reward, and emits training samples — so the trainer learns from the exact tokens that were sampled — the *token-in-token-out* contract discussed next.

End-to-end example — one turn through the proxy. Consider a single MBPP coding turn:

```
1. Harness builds the chat and calls the proxy (drop-in LLM API):
     POST /v1/messages   { model: "policy",
                           messages: [system, PROBLEM.md, prior observations],
                           metadata: { session_id: "ep_9f3c…" } }

2. Proxy → Inference engine (SGLang, current policy weights):
     generate(prompt_ids) → response_ids, text, logprobs

3. Proxy records the on-policy call for training:
     rollout_store[session_id].append(
        { prompt_ids, response_ids, logprobs } )   # exact sampled tokens

4. Proxy returns an Anthropic-style response to the harness:
     { content: "```python\n def solve(): …```", stop_reason: "end_turn" }

5. Harness extracts solution.py, runs it in the AgentCore sandbox,
   appends the checker's stdout/exit-code as the next observation, loops.

6. Episode ends → reward = 1.0/0.0. Sample Adapter reads
   rollout_store["ep_9f3c…"], attaches the reward/advantage, and emits
   loss-masked training samples (mask 0 on prompt/observation tokens,
   1 on the policy's own response tokens).
```

The key point: **without the proxy, the trainer could not observe — or take credit for — the tokens the harness actually sampled.** The proxy is simultaneously the *substitution* point (external API → policy) and the *observation* point (turns → training samples), which is exactly why the disaggregated design in Figure 1 puts it on the seam between the agent runtime and the training server.

2.4 Token-in-token-out (TITO) — the proxy's core responsibility

What it is, and where it comes from. *Token-in-token-out* (TITO) is the contract that the rollout path carries **token IDs**, not just text: the inference engine returns the **exact token IDs it sampled** (plus their log-probs), and the trainer consumes **those same token IDs** for the policy-gradient update — never a sequence re-derived by tokenizing decoded text. The need for it is intrinsic to *harnessed* agentic RL [1]. In the traditional setup the training engine owns the environment loop and keeps one continuously extended token history, so a rollout is a single linear token sequence by construction. In the harnessed setup the **harness** owns context construction, control flow, and tool execution; it builds each prompt independently and talks to the model over an ordinary, **text** LLM API. The trainer never touches the environment — it only observes a sequence of request/response pairs $(p_1, a_1), (p_2, a_2), \ldots$ crossing the LLM endpoint. TITO is the guarantee that what crosses that seam is captured *token-exact*: the a_i the trainer learns from are the very tokens the policy sampled.

The problem — token-prefix continuity breaks (retokenization drift). Harnesses chain calls in **text**: the previous call is a clean text-level prefix of the next prompt. RL, however, needs continuity at the **token** level, and the two do not coincide — a tokenizer is not a clean round-trip, so re-encoding the decoded history can yield a *different* token sequence than the one sampled. The paper isolates three mechanisms:

- **Chat-template non-compositionality.** Rendering a whole message history is not the same as concatenating the renderings of its parts ($\text{Template}(A \| B) \neq \text{Template}(A) \| \text{Template}(B)$): templates insert or drop delimiters at message boundaries. E.g. Qwen's chat template can strip an earlier <think> marker.
- **Decode–retokenize drift.** Decoding is not injective, so re-tokenizing the decoded text need not recover the original IDs: the word *having*, sampled as `h+aving`, can re-tokenize to `hav+ing` inside a later prompt — identical text, different token boundaries.

- **Inference-time output transformation.** Tool-call and structured-output handlers parse, normalize, repair, and reserialize a response before returning it, changing whitespace, delimiters, or JSON — so even the *text* folded into the next prompt can differ from what the sampled IDs represent.

If training reconstructs tokens from this text rather than using the recorded IDs, three things silently misalign:

- **The action.** The token the loss is computed on is no longer the token the policy emitted.
- **The log-prob / importance ratio.** On-policy RL weights each token by the ratio of new to old policy probability; if the "old" tokens are reconstructed rather than recorded, the ratio is computed against tokens that were never sampled, biasing the gradient.
- **The loss mask.** The mask that marks *policy* tokens (train) vs. *prompt/observation* tokens (ignore) is defined per token; a shifted boundary leaks environment tokens into the loss, or drops the model's own.

The same drift also breaks *sample merging*: two consecutive calls can be stitched into one training sequence only when their token IDs actually satisfy prefix continuity, so drift turns one rollout into a *dynamic* number of samples. For RLVR — where the reward is a precise 1.0/0.0 from a checker — this token-level slop quietly corrupts the very signal the method depends on.

The solution space. The paper frames four strategies, trading correctness against prefix-computation reuse:

- **Independent per-call training.** Treat each (p_i, a_i) as its own sample; compute loss without assuming any relation to neighbors. Token-exact by construction, but recomputes the long shared prefixes across calls — wasteful.
- **Buffered token replacement** (AReaL, verl Uni-Agent). The proxy buffers each call's text and tokens; when a new request's text matches, it splices the *recorded* response IDs back in so token-prefix continuity always holds. This recovers reuse, but when the buffered IDs differ from those in the actual next request it evaluates the response under a *stitched* prompt the policy never saw — an off-policy discrepancy.
- **Prefix-shared / tree-structured training.** Store common prefixes once and use a branch-aware attention mask so each token attends only to its ancestors. Recovers the reuse of independent training, but demands heavy backend support (tree packing, custom masks/kernels, distributed-gradient handling).

- **Best-effort sequence merging** (Agent Lightning v1.0's choice). Merge two calls only when their observed token IDs satisfy prefix continuity; otherwise close the sequence and start a new one. This preserves the prompt actually consumed during rollout, works with standard dense causal kernels, and lets drift merely lower the merge ratio rather than corrupt the gradient.

Why the proxy owns it. Every one of these strategies rests on the same primitive: *capturing the sampled token IDs at the moment of generation* instead of guessing them from text later. The proxy is the only component that straddles both sides of the seam — it speaks the **text** LLM API to the harness and the **token** API to the inference engine (SGLang returns `response_ids` + log-probs) — so it is the natural, and only correct, place to do it. Concretely the proxy records `{prompt_ids, response_ids, logprobs}` per turn (Figure above), keyed by `session_id` (Agent Lightning's per-call `model_request` event); the **Sample Adapter** then assembles the training `Sample` directly from those recorded IDs and the loss mask and attaches the reward — so the trainer learns from the exact tokens sampled and the whole drift class is sidestepped before any merging decision. The harness stays blissfully text-only, yet the trainer still gets token-exact, on-policy data. In short: **the proxy is where an off-the-shelf, text-based agent loop is reconciled with the token-exact requirement of RL — that reconciliation is TITO.**

2.5 When one episode becomes many samples — subagents, summarization, and advantage assignment

This is a different problem from TITO — its sibling. TITO asks *"are the tokens I train on exactly the ones the policy sampled?"* — a question about a single call chain crossing the text↔token seam. The problem in this subsection asks *"one episode produced how many training samples, and how do I attach reward, advantage, and loss across them?"* — a question about the **shape of a whole rollout**. Both descend from the same root — in *harnessed* agentic RL the harness (not the trainer) owns context construction and control flow, so a rollout is a set of separately-constructed call chains rather than one linear history [1] — but they are handled in different places. Get TITO right and each individual sample is token-exact; get *this* wrong and you train on token-exact samples with **biased rewards and mis-normalized gradients**.

The problem. In traditional agentic RL one rollout is one Markov trajectory and maps to exactly one training sample. In harnessed agentic RL a single episode can

fan out into a **dynamic number of training samples** N_ρ, known only *after* the harness has run and samples have been constructed. This is not a corner case: in Agent Lightning's coding-agent runs only **36% of rollouts on average stay a single training sample, and the mean rollout yields 2.4 samples** [1]. Three harness behaviours drive the fan-out — and all three are exactly the ones you called out:

- **Context summarization / compaction.** To stay under the context window the harness periodically replaces the running message history with a summary. That *rewrites the token prefix* the next call is built on, so the new call no longer has the previous call as a token-level prefix — the chain must be cut and a new sample started.
- **Response rewriting / expansion before acting.** The harness may parse, repair, reserialize, or expand a model response (tool-call normalization, JSON repair, template surgery) before it acts on it or folds it into the next prompt — so what re-enters the history differs from the sampled tokens, again breaking the merge condition.
- **Subagents.** The harness spawns a sub-agent that runs its **own** chain of model calls with its **own** prompt history. These branches do not share one linear history with the parent at all; each is its own sample (or its own little tree).

The cause — the harness is the only component that sees a whole episode. The proxy sees a stream of independent per-call (`prompt_ids, response_ids`) pairs; the sandbox sees a stream of independent tool ops and one terminal reward. **Neither sees the episode as a whole** — only the harness, sitting between them, knows that these calls, that summary, and those subagent branches all belong to *one* rollout ρ with *one* outcome. So the harness is the only place that can stamp every fragment with a shared `rollout_id` (and `prompt_group_id`), and the training side must honour that grouping — otherwise it silently treats an incidental fan-out as if it were more independent data. The distinction from TITO matters here: retokenization drift (§"Token-in-token-out") is *incidental* and merely lowers the sample-merge ratio, whereas summarization and subagent spawning are *structural* — they are deliberate harness operations that split a rollout by design.

Why the naive handling corrupts training. Once one rollout ρ owns N_ρ samples that all inherit its outcome reward R_ρ, two group-level computations become ambiguous:

- **Advantage baseline (GRPO).** GRPO's advantage subtracts a group baseline

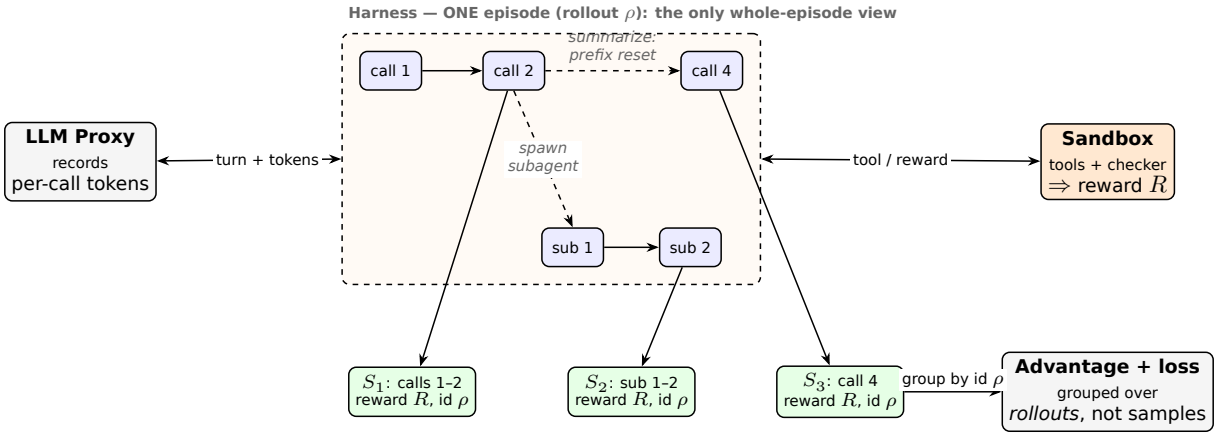


Figure 4: How it works with the other components. The harness sits between the proxy and the sandbox and is the **only** component that sees a whole episode. One rollout ρ fans out into several training samples: a spawned *subagent* runs its own call chain (S_2), and a *summarize* step resets the token prefix so calls 1–2 close as S_1 while the post-summary call 4 opens S_3. Every sample inherits the episode's terminal reward R and carries the same `rollout_id` ρ; advantage baselines and loss normalization must then be computed **per rollout**, not per sample, so an incidental fan-out cannot reweight the update.

$\bar{r}$ over the samples generated from one prompt. If the baseline is taken at the **sample** level, a rollout that happened to split into more samples silently gets more weight in the baseline. Concretely: in one group Rollout 1 succeeds ($R = 1$) and Rollout 2 fails ($R = 0$). If Rollout 1 fanned out into 3 samples and Rollout 2 stayed 1, the *rollout-level* baseline is the intended $\bar{r} = (1+0)/2 = \frac{1}{2}$, but the *sample-level* baseline is $\bar{r} = (1+1+1+0)/4 = \frac{3}{4}$ — a biased baseline that no one chose, driven purely by how the harness happened to fragment. (Frameworks disagree today: verl Uni-Agent and Polar compute advantage at the rollout level; slime and AReaL at the sample level [1].)

- **Loss normalization.** With a dynamic per-rollout sample count, sample-level *seq-mean-token-mean* normalization (GRPO's default) gives disproportionately more gradient weight to rollouts that produced more samples. Since the sample count is an incidental artifact of summarization/subagents, that reweighting is unprincipled.

The solutions — reconcile at the rollout level. The fix is a small, coherent set of choices that make the incidental fan-out invisible to the gradient:

1. **Sample construction: best-effort merging, honest splits.** Merge consecutive calls into one sample only when their token IDs satisfy prefix continuity (the TITO merge condition); when summarization, response rewriting, or a subagent breaks that prefix, **close the current sample and open a new one** rather than force a stitch. Represent the episode as the resulting *set/tree* of samples, all tagged with the shared `rollout_id`.
2. **Reward: outcome-inherited.** The reward is terminal and per-episode (the sandbox checker's $1.0/0.0$); **every sample in the rollout inherits R_ρ.** A subagent's samples carry the parent episode's outcome.
3. **Advantage at the rollout level.** Compute the GRPO group baseline over **rollouts**, not samples, so — as in the worked example — a rollout that fragmented into more samples cannot move the baseline. Agent Lightning argues this is the principled default; retokenization and harness internals "should not be allowed to change the baseline of the entire group" [1].
4. **Loss normalization at the rollout level.** Use a **rollout-level token-mean** loss: pool all response tokens of a rollout, then average uniformly over rollouts (slime's choice). This is insensitive to how many samples a rollout produced. Plain token-mean is theoretically clean too but empirically unstable when a batch has many long negative samples; sample-level seq-mean is the one to avoid here.
5. **Preserve provenance to the backend.** When samples are flattened into a physical tensor batch, every sequence must retain its `rollout_id` and `prompt_group_id` so steps 3–4 can still be computed after packing onto fixed GPU workers.
6. **Open problem: intra-rollout credit assignment.** Outcome-inheritance is a placeholder — it does not say *which* branch, subagent, or turn earned the reward. Finer credit assignment across the samples within a rollout (turn-level grouping such as GiGPO [26], process rewards) is active research; see §"Credit assignment over long horizons".

Where this project sits. The training demos in this report deliberately use **short, single-chain episodes**: a binary end-of-episode reward with `EPISODE_MAX_TURNS=3`, no context summarization and no subagents, so every rollout collapses to exactly one sample and the fan-out above never bites — which is why `build_token_stream` can safely return a *bare* `Sample` (§"Key code — three pieces"). That is a modelling convenience, not a limitation of the stack. The moment the harness is a real, full-featured agent — one that compacts context and spawns subagents — this subsection's machinery becomes mandatory;

§“demo_cpu_branching” below is a standalone demo that switches the in-process harness for the **Claude Code CLI**, deliberately provokes summarization and subagent branching, and is built to show one rollout producing several samples that are then reconciled with rollout-level advantage and rollout-level token-mean loss.

2.6 slime — the trainer, and its asynchronous rollout

The trainer in this report is **slime** (THUDM) — an RL post-training framework built on **Megatron** (training) + **SGLang** (rollout), designed so that agentic workloads plug in as *data-generation* functions without forking the training kernel [9]. Its documented architecture maps cleanly onto Figure 1: a **Data Buffer** bridges a **Megatron** training group and one or more **SGLang** rollout servers behind a **router**; a *custom rollout generation* hook (where our harness + AgentCore sandbox live) produces samples into the buffer, and Megatron periodically pushes updated weights to the SGLang servers.

Asynchronous mode. By default training and rollout alternate, so the GPUs on each side idle while the other runs. slime avoids this with an **async training driver** (`train_async.py` instead of `train.py`): while Megatron trains on the current batch, the rollout side already generates the *next* rollout’s data, overlapping the two [9]. For agentic RL — where rollouts have a **long tail** (one slow episode, e.g. a lengthy compile, can stall a whole step) — slime also offers a **fully-async rollout** path (`--rollout-function-path slime.rollout.fully_async_rollout.generate_rollout_fully_async`) that keeps a fixed pool of in-flight generations across rollout boundaries, so a training step never waits on the slowest sample. This is exactly the regime this report targets: **AgentCore sandboxes have variable, sometimes long, per-episode latency**, and async rollout keeps the trainer fed while they run.

Sources: slime repository https://github.com/THUDM/slime and documentation https://thudm.github.io/slime/ [9].

2.7 Scaling the loop — inference, harness, and sandbox in balance

The rollout loop has **three** components, and they scale as one system because they are interconnected. The **inference engine** (SGLang) produces every model turn;

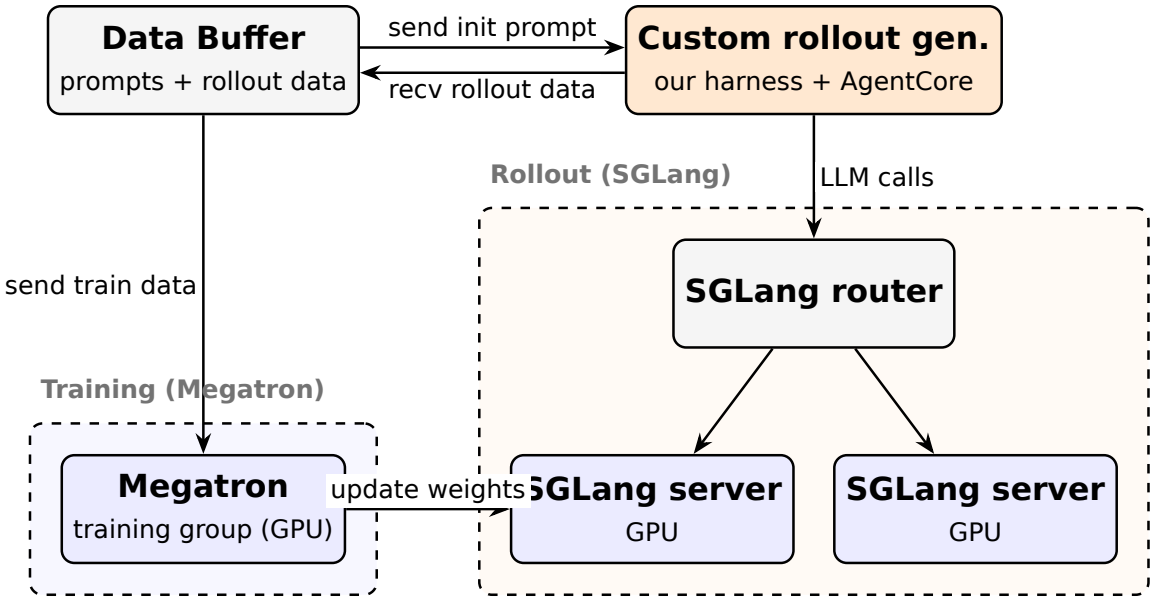


Figure 5: slime's architecture (redrawn after the slime documentation [9]). A **Data Buffer** bridges the two sides: it hands prompts to a *custom rollout generation* hook — where our **harness + AgentCore sandbox** live — and receives finished rollout data back. Generation runs through an **SGLang router** to one or more GPU **SGLang servers**. On the training side the buffer feeds **Megatron**, which periodically pushes **updated weights** to the SGLang servers. Agentic workloads plug into the hook without forking the training kernel.

the **sandbox** (AgentCore) runs every tool call and the checker; and the **harness** — the agent-loop driver that opens each episode, calls the policy turn by turn, runs the checker, and packs the result into a `Sample` — sits between them and paces both. Rollout throughput is bounded by the slowest of the three: **buffer-fill rate = `min(inference, sandbox, harness)`.** Scale two and leave the third and the bottleneck merely moves onto the third. So the job has three parts, in order: **diagnose** which tier is slow, **scale** it, and **balance load correctly on each tier** — the three balance by *different* rules, and applying one tier's rule to another is a mistake. This subsection treats all three together; the diagram below is the map.

Why scale the sandbox at all? A policy only *generalizes* if it is trained across many **diverse tasks** — SWE repos, SQL, math, tool-use, RAG, CTF, GPU kernels — and across **many episodes of each** so the reward signal is not dominated by one family. That means running a large number of **concurrent episodes** per training step, and every concurrent episode needs its own isolated sandbox. So scaling *up the number of concurrent sandboxes* (the harness concurrency knob, `CONC`) is the direct lever on task diversity and throughput.

But sandboxes do not run in isolation. Every one of those concurrent episodes still gets its model turns from the **same** inference engine. If we raise sandbox concurrency while holding the number (and size) of inference replicas **fixed**, generation requests start to **queue** at the inference engine: per-turn latency climbs, and the extra sandboxes sit idle waiting for their next turn. Past that point the inference engine — not the sandbox — is the bottleneck, and adding sandboxes buys nothing.

So first diagnose which side is slow. Per episode the wall-clock is split between (a) **generation** — the inference engine producing the turn — and (b) **sandbox execution** — running the tool call and the checker. The harness measures both: `[sglang-gen]` → `perf/sglang_gen_ms_{p50,p95}` for generation, and `[sbx-op]` → `perf/sandbox_ms_{p50,p95}` for the sandbox round-trip. Compare them before scaling:

- **Generation dominates** (`sglang_gen_ms` ≫ `sandbox_ms`) ⇒ **inference-bound**. Scale the inference engine (more replicas / more GPUs / larger tensor-parallel), *not* the sandbox count.
- **Sandbox dominates** (`sandbox_ms` ≫ `sglang_gen_ms`) ⇒ **sandbox-bound**. Scale sandbox concurrency / instances; one inference replica can feed many episodes.

Examples.

- **Sandbox-bound — KernelBench (GPU).** The model emits a short CUDA kernel in a few hundred ms, but `nvcc` compilation plus `allclose` on the GPU takes **seconds**. Sandbox time dwarfs generation → add GPU sandbox Instances; a single inference replica keeps up.
- **Inference-bound — math / long-reasoning.** The checker is a one-line stdlib assert (sandbox ≈ tens of ms), but the model generates a long chain of thought (generation ≈ seconds). Inference time dwarfs the sandbox → adding sandboxes does nothing; add inference replicas.
- **Balanced — SWE with a moderate test suite.** Generation and a multi-second test run are comparable → scale **both** roughly in proportion to keep either side from queuing.

The practical loop: pick a concurrency, read `perf/sglang_gen_ms_*` and `perf/sandbox_ms_*`, and grow whichever side dominates until the two are balanced — then raise both together.

Balancing load as you scale — let the router do inference, round-robin the sandbox pool. Once you add replicas on *either* side, the next question is how episodes get spread across them. The two sides answer it differently, and it matters that you *not* apply the same rule to both:

> **Do you need a load balancer?** For the **sandbox: no** within one Runtime — the session id *is* the balancer (the capacity provider places each session for you); across a *pool* of Runtime ARNs you add only trivial **sticky round-robin by session id**, not a real LB. For **inference: also no** — slime's one SGLang router already does cache-aware balancing; you just target its URL. In short: you never stand up a load balancer of your own; you mint distinct session ids, and (only across a Runtime pool) route them stickily.

- **Inference — do not hand-roll round-robin.** slime already runs **one SGLang router** in front of its inference replicas, and the router does **cache-aware (prefix-affinity) load balancing**: it steers each request to the replica most likely to already hold that prompt's prefix in its KV cache, falling back to shortest-queue only when caches are cold. That beats naive round-robin, which would scatter a shared system-prompt/few-shot prefix across every replica and *destroy* the prefix cache — inflating time-to-first-token. So every proxy and policy simply targets the **router URL**; scaling inference is "add replicas," with zero client change and no load-balancing code of your own.

- **Sandbox — the session id *is* the load-balancing key.** One `runtimeSessionId` maps to exactly one isolated microVM / instance. Within a **single** Runtime, AgentCore's capacity provider already **places each new session on an available instance**, so simply minting a distinct per-episode session id spreads the load for you. You only need *explicit* round-robin when you run a **pool of sandbox Runtimes** (several ARNs). Three things trigger a pool — otherwise a single ARN is enough: **(1) concurrency** — your in-flight episodes ($S = \text{CONC} \times \text{num_engines}$, or $N \times S$ once you replicate harness workers) would exceed one Runtime's per-region session cap, so you split the load across K ARNs; **(2) AZ/account spread** — placing Runtimes in different availability zones (resilience) or accounts (isolation, separate quotas); **(3) capability routing** — different families need *different sandbox images* (a GPU family, a DB-baked image, a per-repo SWE image, §3), so you register one ARN per capability and pick by `family` rather than pure round-robin. Below these thresholds — episode count fits one Runtime's quota and one generic image serves every family — you stay on a **single ARN** and let the capacity provider place sessions for you. When you do pool, pin each episode to one ARN and make it **sticky** so every op of that episode returns to the same warm sandbox that holds its workspace:

```python
import hashlib, os
from .generate import SGLangPolicy

# 1) Inference: target the ROUTER, never an individual replica. The router
#    load-balances (cache-aware, shortest-queue fallback) across all replicas,
#    so scaling inference is "add replicas" — no client-side balancing at all.
policy = SGLangPolicy(
    router_url=f"http://{args.sglang_router_ip}:{args.sglang_router_port}",
    model_path=args.hf_checkpoint,
)

# 2) Sandbox: within ONE Runtime the session id already spreads sessions across
#    instances. Across a POOL of sandbox Runtimes, round-robin BY session id so
#    the spread is even AND sticky (all ops of an episode hit one warm microVM).
SANDBOX_ARNS = os.environ["AGENTCORE_SANDBOX_RUNTIME_ARNS"].split(",")

def pick_sandbox_arn(session_id: str) -> str:
    h = int(hashlib.sha1(session_id.encode()).hexdigest(), 16)
    return SANDBOX_ARNS[h % len(SANDBOX_ARNS)]   # deterministic, sticky per episode
```

Hashing the session id (rather than a shared counter) gives an even, **sticky** spread with no cross-thread lock — which matters because episodes run concurrently on `asyncio.to_thread`. A plain counter (`itertools.cycle` behind a lock) is the sim-

pler *literal* round-robin, but only use it if you do not need stickiness — and here you do: every op of an episode (seed → exec → read) must land on the microVM that holds that episode's files.

(Implementation note: the shipped `make_sandbox()` (`sandbox.py`) actually uses the ***`itertools.cycle`-behind-a-lock*** *counter, not the session-id hash sketched above. Stickiness still holds — the ARN is chosen* ***once*** *when the episode's sandbox object is created and fixed for all its ops — so each episode keeps one warm microVM; only the* selection* is a rotating counter rather than a hash. The hash form above is the lock-free alternative you would switch to if the shared counter's lock became a contention point at high `N × S`.)*

The third tier — the harness, the driver behind both dials. The two dials above, inference and sandbox, are both *remote*; both are **driven** by a third component that is easy to forget because it is neither on a GPU nor remote: the **harness** — the agent loop that opens each episode's sandbox, calls the policy turn by turn, runs the checker, and packs the result into a `Sample`. It has its own throughput ceiling, and if you scale inference and sandbox to fill a deep async buffer while leaving the harness alone, the bottleneck simply **moves onto the driver**: the buffer-fill rate is `min(inference, sandbox, harness)`, and adding replicas to two of the three buys nothing once the third saturates.

Today the harness is a single process. slime runs **one** `RolloutManager` Ray actor whose single event loop drives *every* `generate` coroutine (`slime/ray/rollout.py`); the number of concurrent episodes is bounded by one `asyncio.Semaphore` sized `S = CONC × num_engines` — with the demos' `CONC=4` and two SGLang engines, `S = 8` in flight (`slime/rollout/sglang_rollout.py`). There is **no slime knob for "number of rollout worker processes"**: `CONC` raises concurrency *within* the one process, not the number of processes. A second, finer semaphore (`INFER_CONC`, default `0` = off; `policy.py`) can independently cap concurrent SGLang POSTs, but it too lives inside that single process.

Why one process ceilings. Concurrency here is `asyncio`, so it overlaps *I/O waits* — but not CPU work, which the Python **GIL** serializes: every turn JSON-encodes an invoke payload, decodes the response, and (at episode end) builds the loss-masked token stream, and those run one-at-a-time. The synchronous boto3 `invoke_agent_runtime` is offloaded with `asyncio.to_thread` onto the **default, unconfigured** executor (≈ `min(32, cpu+4)` threads; `agentcore_runtime_sandbox.py`), and each episode builds its own boto3 client at botocore's default **10-connection** pool (`max_pool_connections` is never tuned).

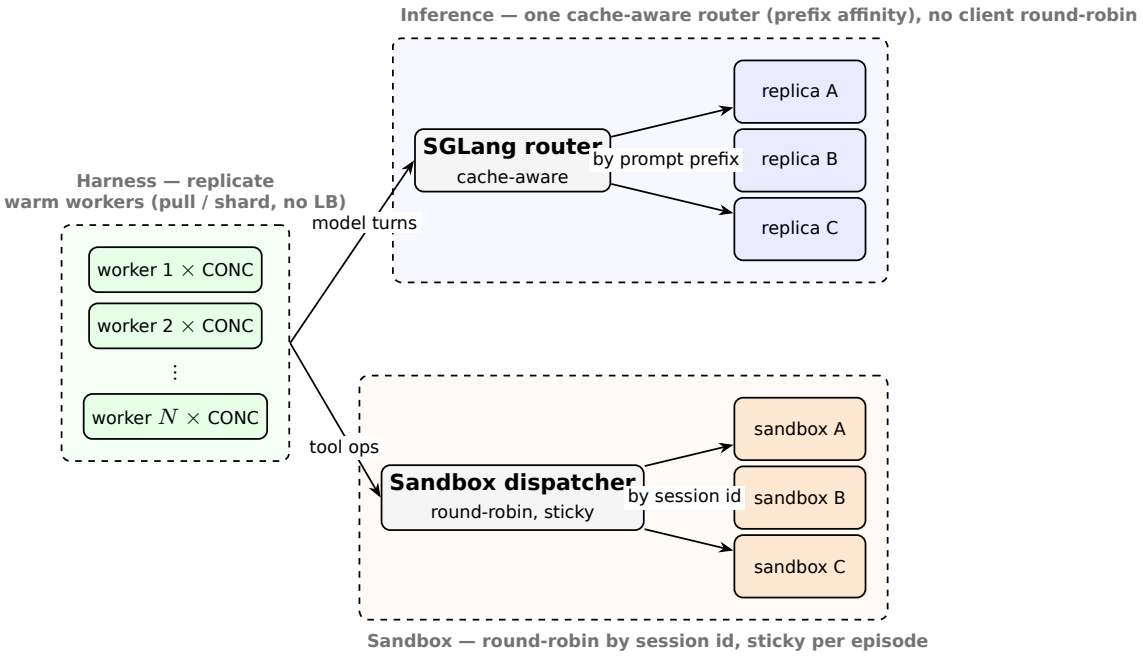


Figure 6: The three interconnected tiers scale together, and **each balances load by a different rule** — buffer-fill rate is `min(inference, harness, sandbox)`. **Harness (left):** the driver is a persistent, warm process; you scale it by **replicating worker processes** (N workers $\times$ `CONC` episodes each $= N \times S$ in flight), and they need *no* load balancer — a free worker *pulls* the next episode (work-stealing) or prompts are sharded across workers. **Inference (top):** every worker's model turns target *one* SGLang router, which does **cache-aware** balancing — it steers a request to the replica most likely to already hold its *prompt prefix* in the KV cache (shortest-queue fallback), so a shared system/few-shot prefix is not scattered; the client never round-robins, and scaling is "add replicas." **Sandbox (bottom):** one `runtimeSessionId` = one isolated microVM/instance. Within a single Runtime the capacity provider spreads sessions for you; across a *pool* of sandbox Runtimes you **round-robin by session id**, kept **sticky** so every op of an episode (seed $\rightarrow$ exec $\rightarrow$ read) returns to the same warm microVM that holds its files.

Every in-flight episode also holds open file descriptors / sockets and buffered trajectory state in RAM. So past some S, raising `CONC` just deepens internal queues: per-episode latency climbs and the buffer stops filling faster. This project **flags but has not measured** that ceiling — a `run.sh` comment notes raising `CONC` "also raises client-side load (asyncio thread pool, boto3 clients)," and the chosen `CONC=4` is documented as "a guess, not a measured optimum" (`docs/sandbox-concurrency-ablation.md`).

The asymmetry that decides how you scale it. The sandbox is **ad-hoc**: one microVM is created and torn down *per episode*. The harness is the opposite — a **persistent, warm** process that holds the event loop, the SGLang/proxy connection, the boto3 clients, and the trajectory buffers *across* episodes (it needs warmth, not isolation, precisely because episodes are independent — §"The harness"). So "scale the harness" does **not** mean "make it ad-hoc like the sandbox." It means **replicate warm worker processes**: run `N` rollout processes, each driving S concurrent episodes, for `N` × S in flight. The sandbox multiplies by *launching more disposable microVMs*; the harness multiplies by *running more long-lived drivers*.

Component	Lifetime / isolation	Scaling unit	Ceiling of one unit	Where it runs
Inference (SGLang)	persistent / shared	replica behind the router	GPU KV cache + decode throughput	trainer GPUs
Harness	**persistent, warm / none needed**	**worker process × `CONC`**	**GIL, `to_thread` pool, FDs, RAM**	**trainer box, or a CPU fleet**
Sandbox (AgentCore)	ad-hoc, per episode / strong (microVM)	session id; pooled Runtime ARNs	per-Runtime session cap	AgentCore microVM / Instance

Where to put the harness — three options.

- **Co-located on the trainer box (today's default).** The one `RolloutManager` runs next to Megatron and SGLang. Cheapest and lowest-latency to the buffer, and fine at S ≈ 8; but its threads, FDs, and RAM compete with the trainer for the box's CPU budget, so it contends as you push S up.

- **Scaled out to a CPU rollout fleet (disaggregated).** Because slime drives generation from a single actor, going past one process is an *architecture* change, not a flag flip: run several rollout drivers (shard prompts across CPU nodes, each executing the same `generate` hook) that dial the **shared** SGLang router and the **pooled** sandbox ARNs. This decouples harness CPU/FDs/RAM from the GPU box and lets you scale drivers independently — the disaggregated-placement lever of §"Open problems" applied to the harness rather than to the trainer/inference split.
- **Hosted on an AgentCore Runtime.** AgentCore *can* run the agent loop as a persistent CPU Runtime (this project keeps that path as untested scaffolding, `agentcore_runtime_generate.py`). A pool of long-lived harness Runtimes is an elastic option, but it adds a network hop between the harness and slime's buffer and spends isolation the harness does not need — which is why the report defaults to in-process.

The joint recipe. First diagnose the harness the way §"Scaling the loop" diagnoses the other two: raise `CONC` and watch whether **samples-per-second / buffer depth** (both instrumented, §"The shared skeleton") keep climbing — if they flatten while the harness host's CPU is pegged (or you hit "too many open files"), the driver, not the GPUs, is the limit. Then grow the three in balance: inference replicas behind the cache-aware router, sandbox Runtimes round-robined sticky-per-episode, **and** harness workers (`N` processes × `S`). A Little's-law sizing keeps them matched: the in-flight episodes needed to saturate inference ≈ `inference throughput × mean episode latency`; provision `N × S` to hit that number, then raise all three together until GPU **goodput** — not any single dial — stops improving.

3 Tasks frontier labs run in a sandbox

Frontier models are post-trained with **reinforcement learning against executable, verifiable environments** (RLVR). The common denominator is a **sandbox seam**: the model's untrusted output runs in an isolated environment, and a **programmatic verifier** — not a human, and increasingly not an LLM judge — turns the outcome into a scalar reward.

3.1 What the frontier reports describe

The trend is toward **verifiable rewards at scale**: Kimi K2, for example, runs a rollout infrastructure of **more than 10,000 concurrent execution sandboxes** on Kubernetes for its coding and software-engineering RL [5]. What the major open reports describe:

Model	RL / verifier setup	Task domains	Reward
DeepSeek-R1 [2]	GRPO; rule-based verifier + code compiler (no neural reward model, to avoid reward hacking)	math, competitive code	answer match / tests pass (binary)
Qwen3 [3]	Reasoning RL from **3,995 query–verifier pairs**; General RL over **20+ tasks**; multi-turn tool use with real-environment feedback	math, code, tool-use, RAG, instruction	rule-based verifiable + reference-based model judge + reward model
Llama-Nemotron [4]	GRPO (science) + RLOO (instruction following); vLLM + Megatron; prompts difficulty-filtered by pass-rate	science reasoning, math, instruction	**LLM-judge** (Llama-3.3-70B) answer-match + `<think>` format reward
AceReason-Nemotron [10]	math-then-code RL; a **local sandbox verifier** runs code against *all* test cases (64-worker pool)	math (~49k problems), code (~8.5k problems)	binary: pass **all** tests / `sympy` symbolic match on `\boxed{}`

Model	RL / verifier setup	Task domains	Reward
Kimi K2 [5]	agentic RL; **verifiable-rewards "gym" + a tool simulator (world model) + real execution sandboxes** (>10k concurrent)	agentic tool use, coding, SWE, math, instruction	executable unit tests + verifiable checks + self-critique rubric

NVIDIA also **publishes the post-training data mix** openly: the Nemotron post-training datasets [12] quantify the domain balance used to train an open reasoning model — roughly **Math 239k, Code 175k, STEM 355k, Chat 628k** samples (plus large multilingual splits) — a concrete picture of how much of each family a frontier post-training run consumes.

3.2 Task families in detail

Each family below lists its **dataset, reward function, key metric, and environment requirement** — the properties that determine what a sandbox must provide — plus an **episode shape**: the typical *turns per episode*, the *response length per turn* (the policy's own tokens), and the resulting *total episode length* (all turns' responses plus the observations fed back). The episode shape is what you size the training config against: it sets **`--rollout-max-response-len`** (per-turn generation budget), the **max turns** cap (`EPISODE_MAX_TURNS`, default **3** in this project's demos), the **sequence-length / context window** the trainer and inference engine must hold, and the **rollout-buffer** memory per sample. The numbers below are typical order-of-magnitude ranges (reasoning models sit at the high end of every response figure), not hard limits — measure your own before committing a config.

3.2.1 Single-turn, generation-bound families

The checker is milliseconds; nearly all the wall-clock is one long generation. These are the *inference-bound* families — scale the SGLang side.

- **Math & logical reasoning** — *Dataset:* GSM8K, MATH-500 [2], AIME / HMMT; **DeepScaleR + NuminaMath** for RL (~49k problems) [10]. *Reward:* **verifiable answer match** — e.g. a `sympy`-based check of symbolic equivalence on the `\boxed{}` answer, binary, with no format or length bonus [10]. *Metric:* accuracy, pass@k (Qwen3-235B's AIME'24 rose **70.1 → 85.1** over 170 RL steps [3]). *Env:* a lightweight **CPU checker** (milliseconds). *Episode shape:* **almost always single-turn** — there is no environment feedback loop, just one answer. But that one response is a **long reasoning trace: ~1k–8k tokens, and up to 16k–32k** for hard competition problems on a reasoning model. Episode length ≈ that one response; strongly **inference-bound** (the checker is a one-line assert).
- **Code generation & competitive programming** — *Dataset:* MBPP [8], HumanEval, **LiveCodeBench** [11]; competitive sets from **Codeforces, AtCoder, LeetCode** [10]. *Reward:* **hidden unit tests pass** — AceReason grants reward *only if the code passes every test case* [10]. *Metric:* pass@1 / pass@k, Codeforces ELO, LiveCodeBench (Kimi K2 v6: **53.7%** [5]). *Env:* a language interpreter plus a **test-runner sandbox with per-case time limits**, run at scale (AceReason: a 64-worker process pool [10]; Kimi K2: >10k concurrent [5]). *Episode shape:* **few turns — often 1, up to ~5** with retry on failed tests. The submitted program is short (**~100–800 tokens**), but a *reasoning* model prepends a long chain-of-thought, pushing the per-turn response to **several thousand tokens**; total episode is typically **~2k–16k tokens**, generation-heavy rather than context-heavy.
- **Instruction following** — *Dataset:* synthetic prompts carrying **one to ten verifiable instructions** (Llama-Nemotron's RLOO stage [4]; part of Qwen3's 20+-task General RL [3]). *Reward:* **deterministic verification via a code interpreter** for the checkable constraints, plus a **format reward**, with an LLM judge for the open-ended remainder [3, 4]. *Metric:* IFEval-style constraint satisfaction. *Env:* a **code interpreter** to evaluate the constraints. *Episode shape:* **typically single-turn** — one response scored against the constraints. Response length tracks the asked-for output, **~200 tokens–2k**; episode ≈ that one response.

3.2.2 Iterative, few-turn families

A short retry loop against a checker — write, run, read the failure, fix. One family (GPU kernels) is where the *checker* dominates cost.

- **GPU kernels / systems** — *Dataset:* KernelBench [7]. *Reward:* numerical **`allclose`** vs. a reference **plus a speedup target**. *Metric:* correctness rate, speedup. *Env:* an **NVIDIA GPU + CUDA toolchain** (`nvcc`) — the one family that requires a GPU, not just CPU. *Episode shape:* **a few turns ($_{1\text{–}5}$), iterating on compile errors and speedup. Each response is one CUDA kernel, 200–1,000 tokens**; total episode a **few thousand tokens**. Cost lives in the *sandbox* (`nvcc` + `allclose` take seconds), not in generation — the canonical sandbox-bound family.
- **SQL / data analysis** — *Dataset:* text-to-SQL suites (Spider-style). *Reward:* **result-set equality** against gold rows (the query is executed against a real engine). *Metric:* execution accuracy. *Env:* a local SQL engine (e.g. SQLite), self-contained. *Episode shape:* **1–3 turns** (write query; retry on a SQL error or empty result-set). Response is a single query, **~30–300 tokens**; episode is **small (a few hundred to ~2k tokens)** — cheap on both sides.

3.2.3 Multi-turn, observation-heavy families

The response each turn is short, but the sandbox feeds a large observation (tool result, retrieved passages, command output) back every turn — cost moves into *context growth and the sandbox*.

- **Tool use / function calling** — *Dataset:* live and **simulated** tool environments; Kimi K2 builds a repository of **3,000+ real MCP tools plus 20,000+ synthetic tools** and generates multi-turn trajectories against a **tool simulator that acts as a world model** [5]; Qwen3 trains with real-environment execution feedback [3]. *Reward:* **correct final result *and* the right tools invoked** (verifiable), sometimes with a model judge. *Metric:* task success on agentic tool-use benchmarks (Kimi K2: **τ^2-Bench 66.1, ACEBench 76.5** [5]). *Env:* tool stubs / an API sandbox / a **stateful simulator** that returns realistic feedback across turns. *Episode shape:* **multi-turn, one turn per tool call — typically ~3–10+ turns**. The policy's response each turn is short (a tool call plus brief reasoning, **~50–300 tokens**), but the *observations* (tool results)

can be large; total episode is **~2k–10k tokens**, dominated by fed-back tool output rather than by generation.

- **RAG & search agents** — *Dataset:* open-domain QA over a corpus or live web search (part of Qwen3's General RL, which explicitly targets RAG [3]). *Reward:* **answer grounded in retrieved facts** — Kimi K2 adds a **sentence-level faithfulness judge** [5]. *Metric:* exact-match / F1. *Env:* a retrieval index or search API plus **network egress**. *Episode shape:* **multi-turn — ~2–8 turns** (issue query → read passages → refine or answer). The policy's responses (queries, final answer) are short, but each retrieved-document observation is large, so the **episode context is retrieval-heavy — commonly 8k–32k+ tokens** even though the model generates comparatively little.
- **Security / CTF** — *Dataset:* CTF challenge sets. *Reward:* a **checker validates the captured flag**. *Metric:* solve rate. *Env:* an isolated shell, sometimes with network to a remote challenge service. *Episode shape:* **multi-turn exploration — ~5–30+ turns** of shell commands. Per-turn responses are short (a command plus reasoning, **~50–400 tokens**); observations (command output) vary from a line to a hex dump, so the **episode ranges from a few thousand to tens of thousands of tokens**.

3.2.4 Long-horizon, session-bound families

Tens to hundreds of turns; the accumulated transcript is the largest of all, and the session must stay alive for the whole workflow — the families that most need a warm, long-lived sandbox.

- **Software engineering (SWE)** — *Dataset:* SWE-bench / SWE-bench Verified [6]; Kimi K2 additionally mines **executable unit tests from real GitHub pull requests and issues** [5]. *Reward:* the repo's **test suite turns green** (`FAIL_TO_PASS` + `PASS_TO_PASS`), binary. *Metric:* % issues resolved (Kimi K2 reports **65.8%** single-attempt on SWE-bench Verified [5]). *Env:* a per-repository toolchain — usually a **prebuilt Docker image** carrying the exact dependencies, run in isolation per task. *Episode shape:* **long-horizon and multi-turn** — real SWE agents take **~15–50+ turns** (locate → edit → run tests → read failure → re-edit); each turn's response is a command or a **diff/patch of ~100–1,000 tokens**, but the observations (file contents, test logs) are large, so the **episode context routinely reaches tens of thousands of tokens (16k–128k+)** — the most demanding family for context length.

- **Web / computer use** — *Dataset:* WebArena-style browser tasks. *Reward:* **goal state reached** (a DOM / state checker). *Metric:* task success. *Env:* a real **browser / rendering engine** and network. *Episode shape:* **long-horizon — ~10–50+ turns**, one UI action (click/type/scroll) per turn. Each response is tiny (**~20–150 tokens**), but every observation (a DOM dump or accessibility tree, sometimes a screenshot) is very large, making this — with SWE — the **heaviest family for episode context (tens of thousands of tokens, growing per turn)**.
- **Multi-agent / long-horizon** — *Dataset:* cooperative or long-workflow tasks. *Reward:* a **verifiable end state**. *Metric:* task success. *Env:* many turns / long wall-clock, with state persisted across a long session. *Episode shape:* **the longest horizon — tens to hundreds of turns**, per-turn responses short-to-moderate, but the accumulated transcript makes the **episode the largest of all (often 32k–128k+ tokens)** and demands a session that stays alive for the whole workflow.

3.3 Language and environment per family — and how one microVM serves them all

The families above differ sharply in *what must be installed to run and check them*. The table names, per family, the **language the model's artifact is written in**, the **environment dependencies the checker needs**, and whether it fits the **CPU microVM** or requires the **GPU Instances** substrate. This is the concrete "what runs inside" that the sandbox image (or a per-session install) must provide.

Family (§3)	Dataset(s)	Artifact language	Environment / dependencies the checker needs	Substrate
Math & logical reasoning	GSM8K, MATH-500, DeepScaleR	Python (answer string)	Python stdlib + `sympy` (symbolic/numeric match)	CPU

Family (§3)	Dataset(s)	Artifact language	Environment / dependencies the checker needs	Substrate
Code gen & competitive	MBPP, HumanEval, Live-CodeBench	Python (competitive: often C++)	Python interpreter + test runner (`unittest`/asserts) + per-case timeout	CPU
Instruction following	synthetic (IFEval-style)	text / Python	code interpreter (stdlib) to verify each constraint	CPU
GPU kernels / systems	KernelBench	**CUDA C++** + Python driver	**NVIDIA GPU + CUDA toolkit (`nvcc`) + PyTorch** (`allclose`, speedup)	**GPU**
SQL / data analysis	Spider-style, Gretel text-to-SQL	SQL (+ Python harness)	a SQL engine — `sqlite3` (in the Python stdlib)	CPU
Tool use / function calling	xLAM, ToolACE	Python / JSON tool calls	Python + tool stubs / API sandbox / a **stateful simulator**; MCP via Gateway	CPU

Family (§3)	Dataset(s)	Artifact language	Environment / dependencies the checker needs	Substrate
RAG & search agents	HotpotQA, open-domain QA	Python (queries + answer)	a retrieval index / search API + **network egress**	CPU
Security / CTF	CTF challenge sets	shell + C / Python / binaries	an isolated shell + common tooling, sometimes **network** to a challenge service	CPU
Software engineering (SWE)	SWE-bench / Verified	Python (repo language varies)	**per-repo toolchain** — pinned deps + system packages, `pytest`; a large image	CPU (amd64)
Web / computer use	WebArena-style	Python + browser actions	a **headless browser / rendering engine** + network	CPU
Multi-agent / long-horizon	cooperative / long-workflow	varies	the sub-tasks' deps + a **long-lived session** holding state	CPU

How can one microVM support such different environments? The key is that **the microVM does not need one universal image** — it reconciles the variation three ways, in increasing order of how structural the difference is (this is the §"One image per Runtime, but a different env per row" logic applied to *languages/deps* rather than to per-repo images):

1. **Task-agnostic sandbox → most differences are just *files*, not images.** The sandbox exposes only `write_file` / `exec` / `read_file`, and every session is an **isolated microVM** booting *your* OCI image. All task specificity — the prompt, seed files, and the hidden checker — is *written into that episode's session at seed time*, not baked into the image. So a **single generic Python image** already serves the CPU families that need nothing beyond the interpreter and stdlib: math (`sympy`), code (`unittest`), SQL (`sqlite3` is stdlib), instruction following, tool-use stubs, and light CTF. A math episode, a SQL episode, and a tool-use episode are just *different files* in three identical, isolated microVMs.
2. **Per-session runtime install → heavier or unusual deps go into the ephemeral workspace.** This has two variants with *different* implementation status. **(a) Deferred to `CMD`** — the sandbox image installs a large dep when the container boots, before the server binds, keeping the *built* image small; this is **implemented** and is how the GPU sandbox installs the ~2.7 GB CUDA torch wheel to stay under the 2 GB image cap (`deploy/Dockerfile.sandbox.gpu`). **(b) Installed at *seed time*** — the harness would `git clone` / `pip install` / `apt` a family's deps into that session's workspace on turn 0. This is a **described design, not yet implemented**: the shipped demos seed only small files (`PROBLEM.md`, `run_checks.py`, `solution.py`), and the seed-time-install path is the SWE workaround the report explicitly flags as *"a design answer, not a measured one"* (§"One image per Runtime, but a different env per row"). Either way, because each `runtimeSessionId` is a **separate microVM**, one episode's installed environment **never leaks into another's** — the isolation is what would make per-session customization safe.
3. **Per-capability image → a *pool* of Runtimes when the difference is structural.** Some differences cannot be installed at runtime: a **GPU toolchain** (KernelBench needs `nvcc` + a GPU — the **Instances/amd64** substrate, not the CPU microVM), a *pinned OS image* per task (Terminal-Bench), root `apt` the base image can't grant, or an image over the 2 GB microVM cap. For those you register **one Runtime ARN per image**, and routing is now **implemented**: every task carries an `image` tag, and `make_sandbox(image=⋯)` → `_runtime_arn_for_image` looks it up in `AGENT-`

CORE_SANDBOX_RUNTIME_ARN_BY_IMAGE (falling back to round-robin over AGENTCORE_SANDBOX_RUNTIME_ARNS). AgentCore itself never sees the task — the routing is entirely client-side (§"Chapter 2 — most tasks train on the default runtime (demo_cpu_tasks)" walks the chain).

In short: **most task diversity needs no image at all.** The managed AgentCore **Code Interpreter** — the default SANDBOX=agentcore — already ships Python 3.12 with numpy, pandas, scipy, scikit-learn, sympy, torch (CPU), sqlite3, gcc and node, and a live probe (2026-09-25) passed **all 13 task families** at reward 1.0 on it (#1). Only genuine image/substrate differences — a pinned per-task OS (Terminal-Bench), an image over 2 GB, git/internet-dependent installs, or CUDA — need your own Runtime (#3); seed-time installs (#2b) remain a design for SWE workloads, not a path the current demos exercise.

3.4 The one family that stresses the image: SWE and the microVM's arm64 + 2 GB limits

Most families above run inside a small, generic image — a Python interpreter and a stdlib checker fit comfortably. **Software engineering (SWE)** is the exception, and it is worth calling out before the platform section because it collides with two properties of AgentCore's *managed serverless microVM* — and the two collide with each other:

- **arm64-only (documented).** The managed microVM runs **only arm64** images: the official Runtime service contract states the *"Platform: ARM64 container — Required for compatibility with the AgentCore Runtime environment"* [D17] (the GPU capacity-provider substrate, by contrast, is amd64 — §"Building a customized Runtime"). A per-repository SWE image must therefore be **cross-built to arm64**.
- **A tight built-image ceiling.** The managed microVM image must stay small — this project measured a **~2 GB (2048 MB)** ceiling on the built image, which is exactly why its GPU sandbox installs the CUDA torch wheel at CMD rather than baking it in (§"Building a customized Runtime"). A realistic SWE-bench repo image — the exact compiler, system packages, and pinned dependencies a project needs — routinely exceeds a couple of GB.

The two interact badly for SWE specifically, because **SWE-bench's own tooling treats arm64 as second-class**: its README states plainly that *"Support for arm64*

machines is experimental", and on ARM hosts it does not ship prebuilt images — you must pass `--task-repo` so *"the images are built locally with Docker Buildx"* [6b]. So the very images you would reuse are (a) only *experimentally* available on the architecture the microVM forces, and (b) sized for x86 developer boxes, not a 2 GB serverless cap. The heaviest-image family lands on the substrate least suited to it.

Workarounds — in rough order of how well they fit this project's architecture:

1. **Fetch the repo and deps at *seed* time, not in the image.** The sandbox is task-agnostic (`write_file` / `exec` / `read_file`), so the harness can `git clone` the target repo and `pip install`/build its dependencies into the *session workspace* on the first turn, instead of shipping them in the OCI image. This moves the repo bulk off the image entirely and into the per-session ephemeral filesystem (good for 8 h) — sidestepping the image cap. Cost: per-episode setup latency and **network egress**, so the Runtime needs public egress or VPC interface endpoints (§"Network isolation"; [D11]).
2. **Defer heavy installs to `CMD`.** Keep the *built* image lean and `pip`/`apt` the large layers when the session boots — the exact trick the GPU sandbox already uses for torch (§"Building a customized Runtime"). Cost: added cold-start (the GPU demo pays ~90 s for its torch install, §`demo_gpu_single`).
3. **Slim the repo image.** Multi-stage builds, `--no-install-recommends`, drop build toolchains and package caches after compiling, and start from a slim arm64 base — standard Docker hygiene, often enough to bring a repo image under the cap.
4. **Cross-build to arm64 with Buildx, and *validate the checker there*.** `docker buildx build --platform linux/arm64`; for SWE-bench use its `--task-repo` local-build path. Because arm64 is experimental upstream, re-run the `FAIL_TO_PASS`/`PASS_TO_PASS` checker on arm64 before trusting the reward — an arch-specific test flake would silently corrupt the RLVR signal.
5. **Move SWE to the amd64 GPU/Instances substrate.** Instances run **amd64**, allow far larger images, and add a **persistent EBS volume** (so a one-time repo/deps install survives stop/resume) and **14-day** sessions (§"Two compute types"). This sidesteps both the arch and the image cap at once; the cost is spending an EC2 instance (and its GPU) that a CPU-only SWE checker does not otherwise need.

This is the concrete reason the §"Can AgentCore host every frontier sandbox?" table

marks the per-repo Docker path with a ◑ rather than a clean ✓: for SWE, the sandbox *image* — not the checker or the GPU — is the binding constraint.

3.5 One image per Runtime, but a different env per row: how AgentCore reconciles it

SWE-bench raises a sharper version of the same problem: **each task instance ships its own Docker environment** (a specific interpreter version, system packages, and pinned dependencies checked out at one commit) — hundreds of distinct images across the benchmark. AgentCore's binding constraint is that a **Runtime ARN is bound to exactly one container image**, chosen once at `CreateAgentRuntime` (`containerConfiguration.containerUri`, an ECR image — §"Building a customized Runtime"). An `invoke_agent_runtime` call passes only a **`runtimeSessionId` + payload — never an image [D8]**, so you **cannot swap the Docker environment per invocation**. Three ways reconcile "one image per ARN" with "a different env per row" — and the practical answer is usually a hybrid:

- **A — Move the per-row difference *out of the image, into per-session setup* (this project's native model).** The sandbox is task-agnostic (`write_file` / `exec` / `read_file`) and all task specificity already lives in files the harness writes into *that episode's session* at seed time. Extend that: on turn 0 the harness runs `git clone` + `git checkout <base_commit>`, installs the pinned deps, and drops in the hidden `run_checks.py` — so "a different image per row" becomes "a different **setup script** per row, run inside one generic image." Each `runtimeSessionId` is an isolated microVM, so row *i*'s installed env never leaks into row *j*. *Cost:* per-episode setup latency + **network egress** (public or VPC endpoints, [D11]). *Limit:* works when the delta is *installable at runtime*; it struggles when a row needs a **different interpreter version**, **root apt** the base image can't grant, or a heavy compiled toolchain — those belong in the image.
- **B — A *pool* of Runtimes, one image per ARN, routed by row.** Register **K Runtime ARNs, each with a different baked image**, and have the harness pick the ARN by the row's repo/family instead of round-robin (the *capability-routed* dispatch described in §"Demos"). But hundreds of per-instance images cannot each get a Runtime (operationally heavy, per-region limits), so group at **repo-family** granularity (one `django` image, one `sympy` image, …), **not** per-instance.

- **C — The practical hybrid (family base image + per-row delta).** Bake the *expensive, shared* layer per repo-family ARN (interpreter version, system libs, repo at a common base) → B; install the *cheap per-commit delta* (that instance's pip deps) at seed → A. This keeps each image small — under the microVM's ~2 GB arm64 cap above — while one ARN still covers many instances. If images are simply too large or too x86-bound, fall back to the amd64 **Instances** substrate (workaround #5).

Caveat: this is a **design answer, not a measured one** — SWE is the one family this project has **not** yet wired to a real dataset or run on hardware (the real-dataset work covers GSM8K/MATH-500/HumanEval; SWE-bench/SQL/RAG remain TODO), and, per above, SWE-bench's arm64 support is itself experimental.

4 Foundations of AgentCore Runtime

Amazon Bedrock AgentCore [D1] is a family of managed services for running and securing agents and tools — any framework, any model, fully serverless. This report uses one of them, **AgentCore Runtime**, as the execution **sandbox** for RL. This section defines its three core concepts, its two compute types, how AgentCore manages their lifecycle, and how you build and deploy a customized Runtime — cross-checking the official documentation [D1–D13, D17] against AWS's own SDK and sample code [D14–D16].

4.1 The microVM at a glance — what it is, what runs in it, what it supports, how you customize it

Before the precise definitions, a compact answer to the four questions a practitioner asks first. Each is expanded in the subsection named after it.

- **What is the microVM?** The default AgentCore Runtime substrate is a **Firecracker-class, hardware-virtualized microVM** — a lightweight VM with its **own guest kernel, CPU, memory, and filesystem**, one per session, that still boots **your OCI container image** inside it and is **sanitized on termination**. It gives VM-strength isolation for untrusted, model-generated code while keeping container packaging. It is **CPU-only** (no GPU [D5]), runs an **ARM64** image serving `POST /invocations` + `GET /ping` on `0.0.0.0:8080`,

and lasts up to **8 hours** with a 15-minute idle timeout. When a task needs a GPU, Runtime switches to the **Instances** (EC2) substrate instead (§"Two compute types"). *Details:* §"Runtime, session, and microVM" and §"microVM vs a Docker container vs E2B".

- **What environment / dependencies does it have?** Only what **your image** provides — there is no fixed "AgentCore OS" you inherit. A minimal sandbox is a slim Python base plus the `bedrock-agentcore` SDK; the built image must stay **ARM64 and under ~2 GB**. You add dependencies three ways, in increasing weight: bake them into the image; **install at `CMD`** when the container boots (how the GPU image adds the large CUDA torch wheel without blowing the 2 GB cap); or **install at seed time** into the per-session ephemeral workspace (`git clone`/`pip`/`apt` on turn 0, needing egress). Each session's installs are isolated and wiped on termination. *Details:* §"Language and environment per family" and §"Building a customized Runtime".
- **What tasks can it support?** Effectively every **CPU** sandbox family frontier labs train against: code + unit tests, math/SQL checkers, instruction following, tool-use simulators, RAG, and light CTF all fit one **generic** image (task specificity is just *files* seeded per session). **SWE-bench**-style per-repo images fit with workarounds (arm64 cross-build, seed-time clone, or moving to the amd64 Instances substrate). The one family the microVM **cannot** serve is **GPU kernels** (`nvcc`/`allclose`) — that needs the GPU Instances substrate. *Details:* §"Language and environment per family" and §"Can AgentCore host every frontier sandbox?".
- **How do you customize it as a Runtime?** Author a container that satisfies the contract (ARM64; `0.0.0.0:8080`; `/invocations` + `/ping`), cross-build it with `docker buildx --platform linux/arm64`, push to ECR, and register it with `CreateAgentRuntime`'s `containerConfiguration.containerUri`. AWS also offers a lower-effort **`agentcore` CLI** (CodeZip or Container build) and a **`bedrock-agentcore` Python SDK** whose `@app.entrypoint` implements the HTTP contract for you. *Details:* §"Building a customized Runtime".

4.2 Runtime, session, and microVM — three concepts, precisely

These three words are used loosely elsewhere; the integration depends on keeping them distinct.

- **Runtime (agent runtime).** A *versioned, containerized host* you create with

`CreateAgentRuntime` [D9]. It packages your code — a container image in ECR, or a zipped Python bundle — behind an ARN. Creating a Runtime auto-creates version **V1** and a **DEFAULT endpoint**; you can publish further versions and custom endpoints and route invocations to them. The Runtime is the *deployable unit*; it holds no per-user state of its own.

- **Session.** One logical task or conversation, addressed by a **`runtimeSessionId`** you pass on `InvokeAgentRuntime` [D8] (client-supplied or Runtime-generated; **minimum 33 characters**, max 256, carried in the `X-Amzn-Bedrock-AgentCore-Runtime-Session-Id` header). The **first** invoke for a new id **cold-starts** dedicated compute; every **subsequent** invoke with the same id **re-routes to that same warm compute**, so files, memory, and processes persist *across turns within an episode*. Different ids are fully isolated.
- **microVM.** The *unit of isolation* backing a session on the default compute: a lightweight virtual machine with its **own CPU, memory, and filesystem**, sanitized on termination. One session maps to one microVM; a microVM is never shared across sessions. Its container contract [D17]: an **ARM64** image listening on **`0.0.0.0:8080`** that serves **`POST /invocations`** (the work) and **`GET /ping`** (health, returning `{"status": "Healthy"}` or `"HealthyBusy"`) — the same contract AWS's SDK scaffolds for you [D15].

The cardinality. These three nest **one-to-many-to-one**: **one Runtime serves many concurrent sessions, and each session maps to exactly one microVM (1 : N : 1)**. The Runtime is a stateless *template* (one image, one ARN); every distinct `runtimeSessionId` you invoke against it mints its own isolated microVM, and AgentCore's capacity provider places each one for you — so the session id, not a second Runtime, is how you get a second sandbox. You create *additional* Runtimes only when the **image**, the per-region **session cap**, or the **AZ/account** must differ (§"Scaling the loop"); otherwise a single Runtime backs all your concurrent sessions. Put another way: **the number of Runtimes tracks the number of distinct sandbox *images* (capabilities), not the number of tasks** — because the sandbox is task-agnostic (`write_file` / `exec` / `read_file`), one generic image serves every task family that runs on the same substrate, so even a mixed-family training run stays on **one Runtime, many sessions** (§"`demo_cpu_tasks` mixes").

In RL terms: **the Runtime is the deployable sandbox image, the session is one episode, and the microVM is the isolated box that episode executes in.** So N concurrent episodes = N session ids on one Runtime — not N Runtimes.

4.2.1 A mixed-family batch — how many Runtimes, sessions, and microVMs, and who picks the image?

The sharpest way to feel the distinction is a **heterogeneous rollout step**: `demo_cpu_tasks` with a comma-list `DATASET=` (e.g. `math,sql,ctf,tool_use`) draws a batch whose rows span several families — say a math problem, a text-to-SQL query, a CTF challenge, and a tool-use task — with `n-samples-per-prompt` episodes each. A natural worry is that four task types need four sandbox images, so four Runtimes. They do not. Count each concept:

- **Runtimes: one** (a small pool of *K* only for throughput, not for variety). Every one of these families is a **pure-CPU / stdlib checker** — `sympy`, `unittest`, `sqlite3`, an isolated shell, tool stubs — so a **single generic CPU image** runs and checks all of them. The Runtime count tracks distinct *images* (capabilities), and here there is exactly one capability. You would add a second Runtime only for a genuinely different image — the **GPU-kernel** family (`nvcc`, a different substrate) — not for a different CPU family.
- **Sessions: one per in-flight episode, recycled.** If the batch is `RB` prompts × `NS` samples, that is `RB×NS` *episodes*, but they do **not** all run at once: only `S = CONC × num_engines` are concurrent, and those few sessions **stream through all `RB×NS` episodes** over the step. A math episode and a SQL episode are just two different session ids against the same Runtime.
- **microVMs: one per session, 1:1.** Each `runtimeSessionId` mints its own isolated microVM (booting that same generic image), sanitized when the session ends. So at any instant there are `S` live microVMs, whatever mix of families they happen to be running.

So how do slime and AgentCore know which image a given data row needs? They don't — and by design, they don't have to. No per-row image selection happens anywhere, because the image is *task-agnostic*. The routing that *does* happen is a division of labor:

- **slime / the harness picks the *task*, not the image.** Each dataset row carries `metadata.task.family`; the generate hook (`_task_from_sample`) rebuilds the matching **Task subclass** — its prompt, seed files, and hidden checker — and the harness **writes those task-specific files into that episode's session** at seed time (`write_file`). The family discriminator chooses *what gets written into the box*, never *which box*.

- **AgentCore never sees a task at all.** An `invoke_agent_runtime` call carries only a **`runtimeSessionId` + payload — never an image** [D8]; the image was fixed once, at `CreateAgentRuntime`. AgentCore's only job is to route the session id to its microVM. It is oblivious to whether the payload is a math check or a SQL query.

In other words, "a different environment per row" is realized as **a different set of *files* per session inside one shared image**, not a different image per row. The one time a row truly needs a different *image* — a CPU family vs. the GPU-kernel family — that is a **substrate** choice made per run by an env var, not per row by a router; genuine per-row *capability routing* (pick the ARN by family) is a described design, not something the shipped demos do (§"One image per Runtime, but a different env per row"). This is exactly why a mixed-family run stays on **one Runtime, S sessions, S microVMs**.

4.3 microVM vs a Docker container vs E2B

Because "sandbox" and "container" are often used interchangeably, it is worth being precise about what the microVM actually is — especially since we are running **untrusted, model-generated code**, where the strength of the isolation boundary is the whole point.

- **A Docker container** isolates via Linux namespaces and cgroups but **shares the host kernel**. That is fine for packaging and for trusted workloads, but a shared kernel is a comparatively soft boundary for hostile code: a kernel exploit escapes to the host and to co-tenants. Containers alone are not a safe multi-tenant boundary for arbitrary generated code.
- **An AgentCore microVM** is a lightweight, **hardware-virtualized** virtual machine (a Firecracker-class microVM) with its **own guest kernel**, its own CPU/memory/filesystem, and a VM-strength boundary — while still booting **your OCI container image** inside it. So it is not "container *or* VM": you keep container packaging *and* gain VM isolation, provisioned per session and sanitized on termination. On top of that it is **AWS-native** — reached only by in-account SigV4/IAM, placeable in your VPC, and integrated with the other AgentCore services (Gateway, Memory, Identity, Observability) — and it can carry a **GPU** (via Instances) and **long sessions** (8 h microVM / 14 d Instances).

- **E2B** is an independent sandbox platform that, like AgentCore, runs code in **Firecracker microVMs** (its templates are also defined from Dockerfiles), so the *isolation model is comparable*. The differences are operational, not architectural: E2B is its own cloud (or self-hosted) service with its own SDK and identity, oriented to short-lived code-interpreter sandboxes, whereas AgentCore is a managed AWS service that runs **in your own account** under IAM/VPC, adds **GPU on Instances** and **multi-hour/multi-day sessions**, and slots into the rest of the Bedrock AgentCore stack.

	Docker container	AgentCore microVM	E2B sandbox
Isolation boundary	shared host kernel (namespaces/cgroups)	**own kernel** — Firecracker-class microVM	own kernel — Firecracker microVM
Runs your image?	yes (OCI)	yes — your OCI image *inside* a microVM	Dockerfile-defined templates
Managed by	you / your orchestrator	**AWS, in your account** (IAM/SigV4/VPC)	E2B service (cloud or self-host)
GPU	host-dependent	**via Instances** (EC2)	not the core offering
Session length	you manage	8 h microVM / 14 d Instances	short-lived (code-interpreter)

For an RL sandbox running untrusted policy output, the microVM's per-session hardware isolation is the property that matters: one episode's generated code cannot see another episode's files, the trainer's weights, or the host — and the box is destroyed and its memory wiped when the session ends.

4.4 Two compute types — managed serverless (microVM) vs Instances (GPU EC2)

AgentCore Runtime offers two substrates, chosen at create time; the loop code is identical across them.

- **Serverless microVM (default).** No servers to manage: AgentCore provisions a per-session microVM on demand. **CPU only** — a microVM has *no GPU* [D5]. Sessions last up to **8 hours** with a **15-minute idle timeout** (reset on each invoke). This fits the large majority of RL sandboxes: code execution + unit tests, math/SQL checkers, tool simulators.
- **Instances (GPU).** When a task needs a GPU, Runtime runs on the **Instances** compute type — real **EC2 instances in your own account**, provisioned through a **capacity provider** (`capacityProviderConfiguration` → `ec2Configuration`) [D5, D9]. Supported GPU families include **g4dn, g5, g6, g6e, gr6, g6f, gr6f, g7e** (plus `inf2`); AgentCore provisions the NVIDIA drivers, so a standard CUDA image works unmodified. Sessions can run up to **14 days**, a **persistent EBS volume** survives a session stop and re-attaches on resume, and — unlike the microVM — one instance/session can host **many agents** (1:N). Crucially, one `runtimeSessionId` still maps to exactly **one instance = one GPU**, so there is **no cross-session GPU time-slicing**: you scale GPUs by adding sessions/instances, not by fractioning one.

> **What about a multi-GPU instance — two sessions, one GPU each?** No, not natively: **one AgentCore session = one whole instance** (session→instance is 1:1; the capacity provider hands you the *instance*, not a GPU). So you **cannot** point two distinct `runtimeSessionIds` at the *same* 2-GPU box and expect one GPU each — two session ids try to acquire two *separate* instances, and you never get to pin a session to a particular GPU on a shared box. To put several episodes on the GPUs of *one* multi-GPU instance you must instead use **one shared session** plus a **self-managed in-instance farm** (a Ray farm keyed by per-episode `sub_session`), which is exactly what the two farm demos do — set `B4_GPU_FRACTION=1.0` with `B4_FARM_GPUS=N` to lease **one whole GPU per sub-session** (N GPUs ⇒ N concurrent whole-GPU episodes, as in `demo_gpu_farm_whole`), or `B4_GPU_FRACTION=0.5` on a single GPU to pack two fractional episodes onto it (as in `demo_gpu_farm_fractional`) — no MIG and no cross-session time-slicing required.

> **Can you pick a CPU-only EC2 instance for an Instance?** Not as a documented path. The `ec2Configuration` families AWS lists [D5, D9] are all **accelerated** (g4dn/g5/g6/g6e/gr6/g6f/gr6f/g7e + `inf2`) — there is **no general-purpose (m/c/t) CPU family**. The split is deliberate: **CPU-only work belongs on the serverless microVM**, and the Instances substrate exists to add

> a **GPU**. So if you want a larger/x86/longer-lived *CPU* box (e.g. to escape the microVM's arm64 + 2 GB image limits, §3), today's only route is to launch a GPU-family Instance and leave the GPU idle — you pay for an accelerator you don't use. (The supported list is per the docs as of this work; re-check `ec2Configuration` if a plain-CPU Instance later appears.)

Property	Serverless microVM	Instances (GPU)
Compute	managed, per session	EC2 in your account (capacity provider)
GPU	none [D5]	g4dn/g5/g6/g6e/g7e… + inf2
vCPU / RAM	**allocated per session; not a customer-set knob** — metered by vCPU-hour + GB-hour [D2, D13]	the chosen EC2 type's spec (e.g. `g6.xlarge` = 4 vCPU / 16 GB RAM / 1× L4)
Disk	**ephemeral only — no persistent volume**; in-VM state lost at session end [D3]	ephemeral **plus a sized, persistent EBS volume** (survives stop/resume)
Container image	**arm64**, built image **≤ ~2 GB** (project-measured, §"Building a customized Runtime")	**amd64**, far larger images allowed
Max session	8 h	14 d
Isolation unit	1 microVM / session	1 instance / session (1 GPU)
Filesystem	ephemeral	persistent EBS across stop/resume
Agents / session	1	many (1:N)

On the microVM's resource envelope. Two limits bite in practice and are worth stating plainly. First, **the filesystem is ephemeral**: "any data stored in memory or

written to disk persists only for the compute lifecycle" [D3], so there is *no* persistent disk on the serverless substrate — a repo you `git clone` or a wheel you install lives only for that session (up to 8 h) and is wiped on termination; durable state needs **session storage / Memory**, or the Instances substrate's EBS volume. Second, the **built container image must stay small** — this project measured a **~2 GB** ceiling (§"Building a customized Runtime"), which is why the GPU image installs its multi-GB CUDA wheel at `CMD` rather than baking it in. AWS does **not** publish a fixed per-session vCPU/RAM *number* for the microVM — capacity is provisioned per session and billed by **vCPU-hour and GB-hour** (the `agent.runtime.vcpu.hours.used` / `agent.runtime.memory.gb_hours.used` usage metrics [D13]), so treat CPU/RAM as *metered, not sized*, and confirm the current hard caps on the **Service Quotas** page for your account before sizing a heavy checker. When a task genuinely needs a large, fixed RAM/disk footprint (a big SWE repo image, a data-heavy checker), that is a reason to move it to **Instances**, where the resources are simply the EC2 instance type's and the EBS volume you attach (§3, §"Two compute types").

4.5 How AgentCore manages the lifecycle — Runtime, session, microVM, and GPU instance, end to end

An RL workload touches **four** AgentCore objects, with **nested lifetimes**. Keeping them straight is what makes cost and capacity predictable; conflating them is what makes a run's instance count surprising. The whole section below is one answer to a single question — *from the moment you deploy to the moment the run ends, what is created, how long does it live, and what ends it?*

Object	Created by	Lives for	Ended by	How many in a run
Runtime (the deployable + endpoint)	`CreateAgentRuntime`, at **deploy time**	the **whole run** (and beyond)	`DeleteAgentRuntime`	**1** by default; a small pool only to raise invoke throughput

Object	Created by	Lives for	Ended by	How many in a run
Session (`runtimeSessionId`)	first invoke with a **fresh id**	**one episode**	`StopRuntimeSession` / idle (15 min) / max-life	≈ **CONC** live at once, **recycled** through every row
microVM (serverless / CPU substrate)	provisioned on session launch	its **session**	session end → **memory sanitized**	1 per live CPU session
GPU instance (EC2, on Instances)	**cold-launched** on session launch	its session (the instance may stay **warm** in the capacity provider)	session end / capacity-provider scale-in	1 instance = **one GPU**, ≈ CONC live

The Runtime is a deploy-time artifact — it spans the whole run. `CreateAgentRuntime` builds version V1 and the DEFAULT endpoint; updates publish new versions; you route traffic by endpoint. Deleting the Runtime removes the deployable — **sessions are not part of the Runtime's own state.** slime is simply handed the ARN and never creates or deletes it (§"Who controls the AgentCore lifecycle"). A run normally has **one** Runtime; pooling **K** of them raises invoke throughput but changes neither how many tasks nor how many images you can serve (§"A mixed-family batch").

The session is the RL unit — one per episode, Active → Idle → Terminated [D3, D4]. The generate hook mints a fresh `runtimeSessionId` per episode — *"one session id == one isolated sandbox for the whole episode"* — so the session id is the isolation unit.

- **Launch / cold start.** There is no "create session" API. The first `invoke_agent_runtime` under a fresh id (the hook's `__aenter__` sends a `ping`) provisions the compute: a microVM on the serverless substrate, or a **cold-launched EC2 instance** on Instances (*minutes* for a GPU box). Same-id invokes thereafter hit warm compute.

- **Active / Idle.** Each op (`write_file` / `exec` / `read_file`) is one more invoke under the *same* id and resets the **15-minute idle timer**; between invokes the session sits Idle but warm.
- **Termination.** Explicit via **`StopRuntimeSession`** [D12] (this project now calls it at episode end — see below), or automatic on **idle expiry (15 min)**, **max lifetime** (microVM **8 h** / Instances **14 d**), or a failed health check → the microVM is **terminated and its memory sanitized**.

The microVM and the GPU instance are the compute a session rides on. One live session == one microVM (CPU) or **one EC2 instance = one GPU** (Instances), a strict 1:1 with **no cross-session GPU time-slicing** (§"Two compute types"). The microVM is **ephemeral** — its filesystem and memory vanish at session end. A GPU instance's **EBS volume persists across a stop**, and the capacity provider may keep the *instance* warm for the next session even after the *session* ends. So the session's lifetime and the instance's lifetime are related but not identical: the session is what your episode owns; the instance is what the fleet manages.

Terminate at episode end — this project now stops sessions explicitly (per backend). A session can be ended explicitly or abandoned to the idle timer. The three sandbox backends differ at episode end (`__aexit__`):

Backend (`SANDBOX=`)	At episode end	Session ends by
`agentcore` (Code Interpreter microVM)	calls the client `stop()`	**explicit**, immediately
`agentcore-runtime` (sandbox-as-Runtime)	calls **`StopRuntimeSession`** (best-effort; `AGENTCORE_STOP_SESSION=0` to opt out)	**explicit**, immediately — idle reclaim is only the fallback if the stop call fails
`agentcore-ray-farm` (B4)	sends `op=close` for the sub_session	frees the **fractional-GPU actor** immediately; the shared g6 session persists

Because each episode mints a **fresh** `runtimeSessionId`, there is no warm session to preserve across episodes: stopping at `__aexit__` returns the microVM (and its concurrent-session quota slot) **immediately** instead of holding it idle for up to 15

minutes — which matters when a batch runs many episodes concurrently against a bounded session quota. Set `AGENTCORE_STOP_SESSION=0` for a GPU fleet that deliberately keeps instances warm for the next session, letting idle reclaim apply instead.

A batch of prompts — row count ≠ instance count. A 100-row dataset (KernelBench level_1) does **not** launch 100 instances. Rows stream through episodes across training steps; at any instant only `--sglang-server-concurrency` (default **4**) episodes are in flight, so the live-instance count = concurrent sessions ≈ CONC, and those few instances **recycle through all rows** over time. What multiplies instances is *concurrency*, not data:

- **Demo B (`agentcore-runtime`, `demo_gpu_single`):** one session per episode × 1:1 ⇒ **N concurrent episodes = N concurrent g6 instances** (bounded by CONC and the capacity provider's max-instance cap).
- **Demo B4 (`agentcore-ray-farm`, `demo_gpu_farm_fractional` / `demo_gpu_farm_whole`):** all concurrent episodes share **one** `runtimeSessionId` → **one** g6 instance; each episode's sub_session leases a fractional-GPU Ray actor, so N concurrent episodes still = one instance.

This is the concrete cost lever: to raise sandbox concurrency on a GPU task **without** paying for N instances, pack episodes onto one instance (B4) rather than fanning out sessions (B).

A mixed-task batch — same session budget, different task per row. When a batch spans task families (`demo_cpu_tasks` with a comma-list `DATASET=`), the lifecycle above is **unchanged**: one Runtime image serves every family, and the harness writes each row's task files into its session and picks the *task*, not the image (§"A mixed-family batch"). A mixed-family batch therefore consumes exactly the same Runtime / session / microVM / instance budget as a single-family batch of the same size — the family mix changes what runs *inside* each session, not how many sessions exist. Capability routing (a different image per family) changes that by one Runtime per extra image — `demo_cpu_terminalbench` does exactly this at task scale (one Runtime per task); on the Code Interpreter, `DATASET=mixed`'s pandas-needing `datasci` family needs no extra image at all.

Latency — measure, don't assume. The docs promise "fast cold starts" but publish no number [D2]; a GPU first-invoke can take **minutes** to launch the EC2. This project therefore instruments every sandbox round-trip (`[sbx-op]` → `perf/sandbox_ms_{p50,p95}`) and every generation (`[sglang-gen]` →

`perf/sglang_gen_ms_*`) — the same signals the scaling analysis in §"Scaling the loop" depends on.

4.6 Building a customized Runtime — bring your own container, end to end

AWS offers three ways to put code on a Runtime — the `agentcore` CLI's zero-Docker **CodeZip** build [D16], the **`bedrock-agentcore`** Python SDK's `@app.entrypoint` [D15], and **bring your own container**. This project uses the **bring-your-own-container** path, because it is the only one that gives you the whole image: base OS, system packages, CUDA toolchain, a cloned repo — exactly what a task-specific sandbox needs. `CreateAgentRuntime` takes a **union** — `containerConfiguration` (an ECR image) **or** `codeConfiguration` (a zip); BYO container is the former [D9]. This section walks the full path from a Dockerfile to a training run that uses it.

Step 1 — write a container that satisfies the contract. AgentCore only requires that your image serve HTTP on `0.0.0.0:8080` with two routes — **`/invocations`** (one request = one op) and **`/ping`** (health) [D17] — and, for the managed microVM fleet, that it be built for **arm64**. The `bedrock-agentcore` SDK's `BedrockAgentCoreApp` + `@app.entrypoint` implement both routes, so the image stays tiny. This project's actual CPU sandbox image (`deploy/Dockerfile.sandbox.cpu`, build context `code/`):

```
FROM public.ecr.aws/docker/library/python:3.12-slim
WORKDIR /app
RUN pip install --no-cache-dir bedrock-agentcore
# sandbox_server_agent.py is self-contained (stdlib + bedrock_agentcore, no relative
# imports), so copy it flat — no package, nothing heavy.
COPY agentcore_slime_demo/sandbox_server_agent.py /app/sandbox_server_agent.py
ENV SANDBOX_WORKDIR=/workspace
EXPOSE 8080
CMD ["python", "-m", "sandbox_server_agent"]      # BedrockAgentCoreApp serves :8080
```

A **GPU** sandbox (`Dockerfile.sandbox.gpu`) changes two things: AgentCore injects `libcuda.so.1` (point `LD_LIBRARY_PATH` at it), and the CUDA torch wheel is large — so install it **at `CMD`, not baked into the image**, to stay under AgentCore's **~2 GB** image cap (§"Two compute types"):

```
ENV LD_LIBRARY_PATH=/usr/lib64          # AgentCore injects libcuda.so.1 here
CMD ["sh","-c","pip install torch==2.5.1 --index-url
↪  https://download.pytorch.org/whl/cu124 \
               && exec python -m sandbox_server_agent"]
```

Step 2 — create the ECR repo and log Docker in.

```
export AWS_REGION=us-west-2
ACCT=$(aws sts get-caller-identity --query Account --output text)
REG=$ACCT.dkr.ecr.$AWS_REGION.amazonaws.com
aws ecr create-repository --repository-name acs-sandbox-cpu --region $AWS_REGION  #
↪  idempotent
aws ecr get-login-password --region $AWS_REGION | docker login --username AWS
↪  --password-stdin $REG
```

Step 3 — cross-build for arm64 and push straight to ECR. The managed microVM fleet is **arm64** (`CreateAgentRuntime` rejects amd64 with *"Supported platforms: [arm64]"*), while a GPU capacity provider is **amd64** — so a build host of the other arch must cross-build. Use a `docker-container` buildx builder (with QEMU) and emit a **plain** single-arch image — `--provenance=false --sbom=false` keeps it out of the attestation-manifest-list shape that `containerUri` rejects:

```
docker buildx create --name xbuild --driver docker-container --bootstrap   # once
docker buildx build --builder xbuild --platform linux/arm64 \
  --provenance=false --sbom=false \
  -f deploy/Dockerfile.sandbox.cpu -t $REG/acs-sandbox-cpu:latest --push code/
```

Step 4 — register the image as a Runtime. An **execution role** (trusts `bedrock-agentcore.amazonaws.com`; grants ECR pull + CloudWatch logs; for a harness Runtime that calls the sandbox, also `InvokeAgentRuntime`) is passed to `create_agent_runtime`, whose `agentRuntimeArtifact` carries the `containerUri`:

```
control = boto3.client("bedrock-agentcore-control", region_name=REGION)
resp = control.create_agent_runtime(
    agentRuntimeName="acs-sandbox-cpu",
    roleArn=exec_role_arn,
    agentRuntimeArtifact={"containerConfiguration": {"containerUri":
↪  f"{REG}/acs-sandbox-cpu:latest"}},
    protocolConfiguration={"serverProtocol": "HTTP"},
    networkConfiguration={"networkMode": "PUBLIC"},   # or VPC + networkModeConfig
    environmentVariables={"SANDBOX_WORKDIR": "/workspace"},
)
# poll DescribeAgentRuntime until status == READY, then keep the ARN:
sandbox_arn = resp["agentRuntimeArn"]
```

For a **GPU** sandbox this step is preceded by **`CreateCapacityProvider`** (`ec2Configuration`: instance type e.g. `g6.xlarge`, VPC subnets/SGs, EBS, `idleInstanceTimeout=600`, `maxLifetime=86400`), attached via `capacityProviderConfiguration` [D5, D9]; the execution setup then also needs the operator policy `BedrockAgentCoreRuntimeInstancesOperatorRolePolicy`.

This project bundles all four steps into one idempotent driver — it creates the ECR repos, cross-builds each image for its required arch, pushes, mints the roles, and calls `CreateAgentRuntime` (plus the capacity provider for GPU), printing the resulting ARN:

```
export AWS_REGION=us-west-2
python3 deploy/deploy_two_runtime.py all
#   ECR create → buildx --platform (arm64 managed / amd64 GPU) → push
#   → exec/operator roles → CreateAgentRuntime (containerConfiguration)
#   → (GPU) CreateCapacityProvider + attach → prints agentRuntimeArn
```

Step 5 — use it in a training run. Nothing in slime references the image or the ARN at build time; the sandbox client reads the ARN from the environment at rollout time (and raises if unset). So "using" a custom Runtime in training is one export — the ARN that Step 4 printed — before launching `run.sh`:

```
export AGENTCORE_SANDBOX_RUNTIME_ARN=arn:aws:bedrock-agentcore:us-west-
↪  2:<acct>:runtime/acs-sandbox-cpu-⋯
↪
export AGENTCORE_REGION=us-west-2
export SANDBOX=agentcore-runtime          # select the sandbox-as-Runtime backend
bash run.sh                               # slime's generate hook now invokes YOUR
↪  Runtime per op
```

From here the lifecycle in §"How AgentCore manages the lifecycle" takes over: the first `invoke_agent_runtime` under each episode's fresh session id cold-starts a microVM (or GPU instance) from **your** image, every tool call is one more invoke, and `__aexit__` stops the session at episode end.

5 Using AgentCore as the sandbox in an agentic RL workflow

With the foundations in place, this section shows how AgentCore Runtime plugs into a real RL trainer — **slime** — as the execution **sandbox**, with the harness kept **in-**

process. The integration is deliberately small: **no fork of slime**, two hook flags, and a sandbox client that turns each tool call into one `InvokeAgentRuntime`.

5.1 Where AgentCore attaches to slime — hooks, not a fork

slime [9] runs Megatron (training) and SGLang (rollout) asynchronously (§"slime — the trainer"). It exposes the rollout path as a **custom generation function**, so an agentic workload plugs in without touching the training kernel:

- `--custom-generate-function-path agentcore_slime_demo.generate.generate` — the per-prompt rollout: it drives the in-process harness, and each tool call goes to an AgentCore sandbox.
- `--custom-rm-path` — an optional custom reward hook (here the reward comes straight from the in-sandbox checker, so the generate hook attaches it directly).

The key design choice: **AgentCore is the sandbox, not the harness.** The harness (agent loop) is independent per episode and benefits from *warmth*, so it stays in-process next to the trainer; the sandbox needs *isolation* per episode, which is exactly AgentCore's per-session microVM. AgentCore *can* also host the harness (this project keeps that as untested scaffolding, `agentcore_runtime_generate.py`), but the recommended path is in-process.

5.2 The call path — one diagram

5.3 Key code — three pieces

1. The generate hook (in-process rollout). slime calls this once per prompt. It builds the policy pointed at the SGLang **router** (so inference stays cache-aware-balanced, §"Scaling the loop"), opens **one sandbox per episode**, runs the episode, and returns a single loss-masked `Sample`:

```
# agentcore_slime_demo/generate.py  (wired via --custom-generate-function-path)
async def generate(args, sample, sampling_params, evaluation=False):
    policy  = SGLangPol-
↪   icy(router_url=f"http://{args.sglang_router_ip}:{args.sglang_router_port}",
                           model_path=args.hf_checkpoint,
                           chat_template_kwargs={"enable_thinking": False})
```

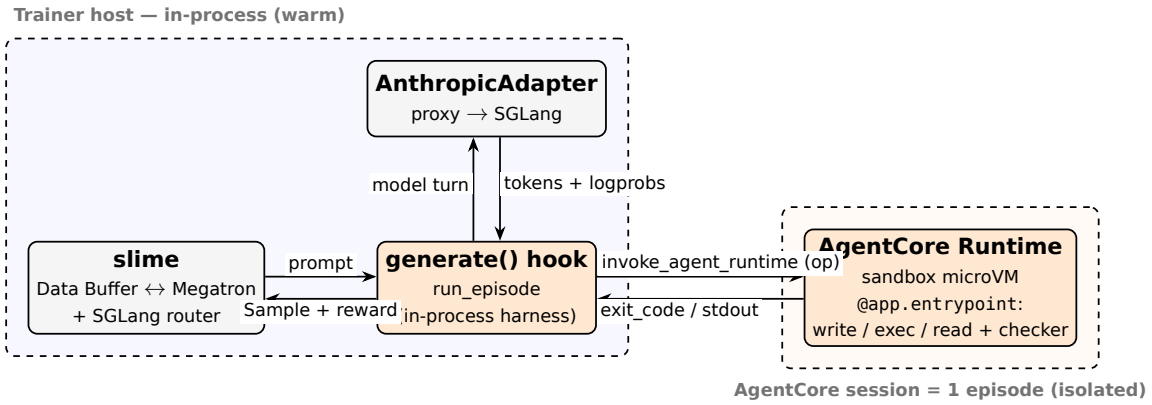


Figure 7: How AgentCore drops into slime. slime hands a **prompt** to the `generate()` hook, which runs the agent loop *in-process*. Each **model turn** goes through the **proxy** (which returns the exact sampled **tokens + logprobs** — the TITO property of §"Token-in-token-out"); each **tool call** becomes one SigV4 `invoke_agent_runtime` to the **AgentCore sandbox**, where `@app.entrypoint` runs the op and the **checker** (the reward source). The finished, loss-masked **Sample + reward** returns to slime's buffer. Everything but the sandbox is co-located on the trainer host; the sandbox is an isolated per-episode microVM.

```
    task    = _task_from_sample(sample)                    # rebuild Task from row
↪   metadata
    sandbox = make_sandbox(os.environ.get("SANDBOX", "agentcore"), region=...)
    async with sandbox as sb:                               # one session == one
    ↪   episode
        traj = await run_episode(policy, sb, task, sampling_params)
    tokens, resp_len, loss_mask, rollout_logprobs = build_token_stream(traj,
↪   tokenizer)
    return Sample(tokens=tokens, response_length=resp_len, reward=traj.reward,
                  loss_mask=loss_mask, rollout_log_probs=rollout_logprobs,
                  metadata={"task_id": task.task_id, "solved": traj.solved,
                            "num_turns": traj.num_turns, "sandbox_id": sb.sandbox_id})
```

Two RL-critical details: the loss mask is **1 on the policy's own tokens, 0 on prompt/observation/tool tokens**, and `build_token_stream` reuses the **recorded `token_ids` per turn** — no re-tokenization, the TITO property of §"Token-in-token-out". It returns a *bare* `Sample` (not a list): slime's fully-async path chokes on nested sample lists.

2. The sandbox client (one op = one invoke). Each `write_file` / `exec` / `read_file` becomes a single SigV4 `invoke_agent_runtime`, keyed by the episode's

session id. boto3 is synchronous, so it is offloaded with `asyncio.to_thread`, letting N episodes run concurrently:

```
# agentcore_slime_demo/agentcore_runtime_sandbox.py
def _invoke_sync(self, payload: dict) -> dict:
    resp = self._client.invoke_agent_runtime(               # boto3
↪  "bedrock-agentcore"
        agentRuntimeArn=self._arn,
        runtimeSessionId=self._session_id,                  # >=33 chars; one per
↪  episode
        payload=json.dumps(payload).encode())
    return json.loads(resp["response"].read().decode())

async def exec(self, cmd, *, timeout=120, **kw):
    res = await asyncio.to_thread(self._invoke_sync,
              {"op": "exec", "command": cmd, "timeout": int(timeout), "workdir":
↪  self._workdir})
    return int(res.get("exit_code", 1)), res.get("stdout", ""), res.get("stderr", "")
```

The real client also **retries** transient 5xx/throttle blips with backoff (one bad round-trip must not crash a multi-hour run) and carries a much larger budget for GPU "insufficient capacity" while a g6 launches.

3. The Runtime server (what runs inside the sandbox). On the Runtime, the `bedrock-agentcore` SDK's `@app.entrypoint` receives each payload and dispatches on `op`. This is the whole sandbox server — and *the checker runs here, so the microVM is the reward source*:

```
# agentcore_slime_demo/sandbox_server_agent.py  (deployed as the Runtime image)
from bedrock_agentcore.runtime import BedrockAgentCoreApp

def handle(request: dict) -> dict:              # pure dispatch (unit-testable
↪  without AgentCore)
    op = (request or {}).get("op", "ping")
    if op == "ping":       _ensure_workdir(request.get("workdir")); return {"ok":
↪  True}
    if op == "write_file": return _do_write(request)     # -> {"ok": True}
    if op == "read_file":  return _do_read(request)      # -> {"text": ...}
    if op == "exec":       return _do_exec(request)      # -> {exit_code, stdout,
↪  stderr}  (runs the checker)

app = BedrockAgentCoreApp()

@app.entrypoint
def entrypoint(request):
    return handle(request)
```

```
app.run()      # serves 0.0.0.0:8080 — POST /invocations + GET /ping
```

5.4 Who controls the AgentCore lifecycle — deploy driver vs. the hook

A precise point that the code above implies but is worth stating outright: **slime's training kernel controls neither the Runtime nor the session.** Lifecycle is split across two planes, and slime's core (`RolloutManager`, Data Buffer, Megatron, the SGLang router) is entirely **AgentCore-unaware** — it only calls the function registered at `--custom-generate-function-path` and gets a `Sample` back.

- **Runtime lifecycle (create/delete the ARN) — outside slime, at deploy time.** No slime component calls `CreateAgentRuntime`/`DeleteAgentRuntime`. The Runtime (and, for GPU, the capacity provider) is provisioned **once before training** by the deploy driver (`deploy/deploy_two_runtime.py`; for GPU, `deploy/redeploy_gpu_sandbox.py`), and slime is simply handed the ARN via `AGENTCORE_SANDBOX_RUNTIME_ARN`. The sandbox client only *reads* that env var and raises if it is unset — it never creates the Runtime. The Runtime is a **deploy-time artifact**, not a training-loop object.
- **Session lifecycle (launch/terminate per episode) — the custom generate hook and its sandbox client.** All session control lives *below* slime's seam, inside `agentcore_slime_demo.generate.generate`, which opens one sandbox per prompt (`async with make_sandbox(...) as sb:`). The client (`AgentCoreRuntimeSandbox`) mints **one `runtimeSessionId` per episode**, and:
  - **Launch is implicit** — there is no "create session" API. `__aenter__` sends the **first `invoke_agent_runtime`** (a `ping`) under the fresh session id, and that first invoke is what **cold-starts the microVM** (and, on GPU, warms the g6).
  - **Each op** (`write_file` / `exec` / `read_file`) is one more `invoke_agent_runtime` under the *same* session id.
  - **Termination is explicit** — `__aexit__` calls **`StopRuntimeSession`** (best-effort; `AGENTCORE_STOP_SESSION=0` opts out) so the microVM and its concurrent-session quota slot are released **immediately** at episode end. AgentCore's **~15-minute idle timeout** remains the fallback if the stop

call fails. slime's kernel still never terminates a session itself — the control lives in the hook's sandbox client.

Action	API	Controlled by	When
Create / delete Runtime (+ capacity provider)	`CreateAgentRuntime` / `Delete`⋯ / `CreateCapacityProvider`	**deploy driver** — *not slime*	once, before/after the run
Drive rollout	—	slime `RolloutManager` / Data Buffer — **AgentCore-unaware**	per prompt
Launch session (cold-start microVM)	first `invoke_agent_runtime` (`ping`)	the hook's sandbox client `__aenter__`	per episode
Run ops	`invoke_agent_runtime` (write/exec/read)	sandbox client, same session id	each turn
Terminate session	**`StopRuntimeSession`** (best-effort; idle timeout is the fallback)	the hook's sandbox client `__aexit__`	per episode, at end
Defer termination across weight-sync (optional)	repeated `ping`	`SessionKeeper`, *outside* the harness	during the sync barrier

Explicit `StopRuntimeSession` at episode end is the happy-path teardown, but a session can still be lost **mid-episode**: a long weight-sync pause between two turns can trip the 15-minute idle clock while the episode is still running and reclaim the session out from under it — exactly the failure mode the `HEARTBEAT=1` `SessionKeeper` heartbeat addresses (§"HEARTBEAT=1"). **One-sentence version:** the deploy driver owns the Runtime, the `generate` hook + its sandbox client own the session (launch on first invoke, terminate with `StopRuntimeSession` at episode end), and slime's training kernel owns neither.

5.5 Ending the session at episode end — explicit teardown, and what "end" means per substrate

We always end the session explicitly when the episode ends — and *what* gets ended differs by substrate, deliberately. The teardown is not inside the loop: `run_episode` (`harness.py`) runs *inside an already-open sandbox* and closes nothing itself (no `finally`, no `close` — it just returns the trajectory). The close fires because **every caller wraps the episode in a context manager** — `async with make_sandbox(...) as sb:` (`generate.py:146`; one fresh sandbox per boot attempt in `recovery.py:98`) — so the sandbox's `__aexit__` runs the instant the episode returns. One teardown path, exercised identically by all demos (`harness.py` is byte-identical across the set).

What `__aexit__` does depends on the backend the `SANDBOX` env selected:

- **Per-episode session (managed microVM, `agentcore-runtime`): stop the whole session.** `AgentCoreRuntimeSandbox.__aexit__` calls **`StopRuntimeSession(runtimeSessionId, agentRuntimeArn)`** (`agentcore_runtime_sandbox.py:287-324`; opt-out `AGENTCORE_STOP_SESSION=0`). Each episode minted its own `sbx-<uuid>` session, so ending the whole thing is exactly right — there is nothing to preserve.
- **Warm-shared instance (CPU/GPU Instances, `agentcore-ray-farm`): close only the episode's `sub_session`, never the shared session.** `RayFarmRuntimeSandbox.__aexit__` sends a single **`op=close` for the per-episode `sub_session`** and releases its client-side concurrency slot (`agentcore_runtime_sandbox.py:484-493`); it **never** calls `StopRuntimeSession`. Server-side, that wipes just that episode's working directory and returns the fractional-GPU Ray actor to the pool, while the warm CUDA context, the shared `runtimeSessionId`, and the instance stay alive for the next episode (`sandbox_ray_farm.py:378-385`, `:219-226`).
- **Default / local:** `AgentCoreSandbox.__aexit__` → `client.stop()`; `LocalSandbox` → temp-workspace cleanup (`sandbox.py:194`, `:110`).

Why we do it, and why it matters. A session is not free to leave dangling: it holds a microVM (or an Instance's actor slot) and counts against AgentCore's **concurrent-session quota**, and the only automatic reclaim is the **~15-minute idle timeout** — far too slow at rollout throughput, where leaked sessions throttle concurrency and inflate cost. Explicit `StopRuntimeSession` releases the slot **immediately** instead of waiting out the idle clock, and closing the per-episode workspace

guarantees the next episode starts from a **clean, isolated env** (no cross-episode file leakage). For the warm-shared path the *asymmetry is the whole point*: closing **only** the `sub_session` frees the episode's env and hands the next episode a fresh workspace **without re-paying the ~60–80 s instance cold-boot** — whereas stopping the shared session would defeat warm reuse (re-pay the boot every episode) and could tear the instance out from under sibling episodes still running on it. So the single "always end at episode end" discipline yields two correct behaviors: **kill the per-episode session** where the session *is* the instance, and **kill only the sub-session** where one warm instance is shared. (The complementary hazard — a session lost *mid-episode* to the idle clock during a long weight-sync pause — is what the `HEARTBEAT=1` `SessionKeeper` addresses.)

5.6 Warm instance shared by session — one hot Instance, a fresh env per episode

The episode-end teardown above hinted at a *second* session id — the per-episode `sub_session`. This subsection is why it exists.

Why we need it. AgentCore has two compute types (§"Two compute types"), and they cold-start on completely different timescales. A managed **microVM** boots in **~1–2 s**, so for CPU-microVM demos we *deliberately do not reuse* it — every episode mints its own `runtimeSessionId` (its own fresh microVM), which is cheaper, faster, and perfectly isolated for free (that is the per-episode-session path of the previous subsection). The **Instances** substrate — a capacity-provider EC2 box, the **CPU `m5`** (`demo_cpu_instance_pool`) and the **GPU `g6`** (the farm demos) — is the opposite: it takes **~60–80 s to cold-boot**. And AgentCore maps **one session to one whole instance** [D5], so the naive "fresh session per episode" model would (a) make the boot dominate wall-clock and (b) **scale the *number of instances* with concurrency** — one live g6 per in-flight episode. At GRPO rollout width that is not cost-feasible; it is the specific reason we built warm-instance reuse, and only for these two Instance substrates.

How we built it — two session ids, one image. Reuse is a two-part mechanism that needs *no* new API and *no* redeploy:

1. **One shared `runtimeSessionId` pins all episodes to one warm instance.** Every invoke for the run carries the same `B4_SHARED_RUNTIME_SESSION` (the client reads it at `agentcore_runtime_sandbox.py:422`). Because one session

= one instance, sharing the id means every episode lands on — and reuses — the **same warm box**, so the ~60–80 s cold-boot is paid **once per run**, not once per episode.

2. **A per-episode `sub_session` multiplexes a fresh, isolated env on that instance.** Each episode also carries a unique `sub_session` (`self._sub = ep-<uuid>`, `agentcore_runtime_sandbox.py:426`), sent on every open/exec/`close` (`:458`, `:488`). Server-side, **the same image** decides the path purely from that field: a request **with** a `sub_session` takes the in-process farm (`sandbox_server_agent.py:194-195` → `_handle_farm`, `:168`); a request **without** one takes the legacy single-session path — "one image serves all of series B" (`:143-147`). So switching a demo to warm-shared is a **client-only flip** (`SANDBOX=agentcore-ray-farm` + set `B4_SHARED_RUNTIME_SESSION`); the deployed Runtime is unchanged.

The in-process farm is what makes the sub_session an isolated env, and it differs by substrate:

- **GPU (`RayGpuFarm`, `sandbox_ray_farm.py`).** A small Ray cluster inside the g6 hands each `sub_session` a **fractional-GPU actor** — `ray.remote(num_gpus=B4_GPU_FRACT` (`:343`, fraction from `:294`; `0.5` packs two episodes on one card, `1.0` = whole-GPU). `open()` **leases** an actor by blocking on a free-actor queue — that block *is* the concurrency gate (`:358-367`) — and episode-end `close` wipes the workspace and **returns the actor to the pool** (`clear_session :219`, `_free.put :316`), keeping the warm CUDA context. Ray's `num_gpus` is only a *scheduling* fraction (`:31`), so an explicit **`flock` serializes `nvcc` compiles host-wide** (`_NVCC_LOCK_PATH :62`, `:100`) — without it, a storm of concurrent `cpp_extension` builds wedges a non-MIG L4 (§"Scaling, sharing, and network isolation"). The client's open-gate `B4_FARM_MAX_CONCURRENT` (`:167`) is set to the server's slot count so the two never disagree.
- **CPU (`CpuFarm`, `sandbox_cpu_farm.py`).** No warm GPU cache to protect, so it optimizes for **isolation instead of warmth**: each `sub_session` gets a private dir holding its `HOME`/`TMPDIR`/caches; every exec runs in its own Landlock jail (own dir only, no TCP, no cross-episode signals) with process and RAM caps, plus its own uid where the container has the capabilities; and close kills the process tree and deletes the dir (env plumbing in `sandbox_farm_env.py`). The shape is the same as the GPU farm's (a shared session plus `sub_sessions`), with the opposite cache policy. What this does and does not guarantee is spelled out in §"Isolation on a shared m5 — what is guaranteed, and what is

not".

Why it matters. The payoff is that **instance count is decoupled from concurrency**: one warm Instance serves an arbitrary number of concurrent episodes, each in its own clean env, with the cold-boot amortized to once. Measured on hardware, the CPU farm cut `m5` cold-boots from **201 → 1** across a 10-step run (§"Chapter 5 — a warm CPU Instance pool shared by isolated sessions (`demo_cpu_instance_pool`)"). It is also the *only* way to pack several episodes onto one physical GPU under AgentCore's "1 session = 1 GPU, no MIG, no time-slicing" reality — the fractional Ray farm is our own scheduler on top. And correctness is free: because the isolated env lives in the `sub_session` (not the instance), reuse never leaks state between episodes, and episode-end closes only the `sub_session` — never the shared session (previous subsection). This is now the **default for every Instance-based demo**, CPU and GPU alike; the microVM demos keep the fresh-session-per-episode model on purpose. Full mechanism write-up: `docs/sandbox-fresh-env-on-persistent-instance.md`.

5.7 One harness, many task families

The loop inside `run_episode` (`harness.py`) is **task-agnostic** — the same seed → turn → check → reward cycle serves every family; only the *prompt, seed files, and checker* change (`task_families.py`), dispatched by `metadata.task.family`:

```
for name, content in task.setup_files():                    # seed the workspace
    await sandbox.write_file(name, content)
await sandbox.write_file("run_checks.py", task.runner_source())
for _ in range(task.max_turns):                             # EPISODE_MAX_TURNS
↪  (default 3)
    text = await policy.complete(messages)                  # model turn (via proxy)
    await sandbox.write_file(task.artifact_filename, task.extract_submission(text))
    exit_code, out, err = await sandbox.exec("python3 run_checks.py", timeout=60)
    if exit_code == 0: solved = True; break
    messages.append(task.observation(exit_code, out, err)) # feed failure back
reward = task.reward(0 if solved else 1)                    # binary 1.0 / 0.0, from
↪  the in-sandbox checker
```

Datasets are built once and seeded per episode: each row carries `metadata.task` (family + spec), reference answers are dropped so they cannot leak into prompts, and `generate.py` rebuilds the `Task` from that metadata. The same code runs the checker for MBPP, SWE, SQL, math, tool-use, RAG, CTF, and the rest — the sandbox is the only thing that changes.

5.8 Reading the episode shape into a config

The families of §“Tasks frontier labs run in a sandbox” differ mostly in their **episode shape** — turns per episode, response length per turn, total episode length — and that shape is what you size the training config against. Two axes drive every knob. **Turns per episode** sets `EPISODE_MAX_TURNS` and, multiplied by the per-turn response, the *sequence length* the trainer and inference engine must hold: single-turn families (math, instruction following) need only a long `--rollout-max-response-len`; long-horizon ones (SWE, web/computer use, multi-agent) need a large **context window** and a **session that stays warm for the whole episode**. **Where the tokens are** sets which side you scale (§“Scaling the loop”): generation-heavy families (math, reasoning code) are *inference-bound* — long responses, trivial checker; observation-heavy families (SWE, RAG, web, tool use) push cost into *context growth and the sandbox*. Rule of thumb: budget context ≈ `max_turns × (response_per_turn + observation_per_turn)`, size `--rollout-max-response-len` to the per-turn response at the high end (reasoning traces), and set rollout-buffer memory from the total episode length so long-horizon samples do not overflow it.

5.9 The episode, end to end

Taking one MBPP episode:

1. `run.sh` sets `SANDBOX=agentcore`, `FAMILY`, `CONC`, `EPISODE_MAX_TURNS`, then `ray job submit ⋯ train_async.py --custom-generate-function-path ⋯generate.generate`.
2. slime hands a prompt to `generate()`, which rebuilds the Task and opens `make_sandbox("agentcore")` → **cold-start** the session’s microVM.
3. **Seed:** `write_file` `PROBLEM.md` + `run_checks.py`.
4. **Turn loop** (≤ `EPISODE_MAX_TURNS`): `policy.complete` via the proxy (`[sglang-gen]`) → extract `solution.py` → `exec("python3 run_checks.py")` (`[sbx-op]`) → on failure, feed the observation back.
5. `exit_code == 0` → solved; **reward = 1.0 / 0.0** from the in-sandbox checker.
6. `build_token_stream` → loss-masked `Sample` → buffer → Megatron step → weight sync to SGLang.

CPU → GPU is one env var: `SANDBOX=agentcore` (microVM, MBPP) → a GPU-Runtime ARN (Instances, KernelBench, `allclose` on cuda) — **zero change to the loop**.

5.10 Scaling, sharing, and network isolation

- **Scaling** follows §“Scaling the loop”: grow the side that dominates the per-op latencies, and balance by letting the SGLang router handle inference (cache-aware) while round-robining a pool of sandbox Runtimes (sticky per episode).
- **Sharing a GPU.** One session = one GPU, with no cross-session time-slicing [D5]. To pack several episodes onto one physical GPU you either run **many agents in one session** (they share the instance — no isolation) or run **your own fractional scheduler** (this project’s `RayGpuFarm`, `num_gpus=0.5` actors, with an `nvcc flock` because a non-MIG L4 wedges under compile storms). Isolated scaling means *more instances*; there is **no MIG on L4**.
- **Warm-instance reuse — solved for BOTH Instance substrates (CPU *and* GPU), and only where it matters.** The reuse question is **substrate-specific**: a managed **microVM** cold-starts in a second or two, so we **deliberately do NOT reuse it** — every episode mints its own session (its own fresh microVM), which is cheaper, faster, and perfectly isolated for free. The **Instances** compute type is the opposite: a capacity-provider EC2 box — the **CPU `m5`** (`demo_cpu_instance_pool`) *and* the **GPU `g6`** (the farm demos) — takes **~60–80 s to cold-boot**, so a fresh instance per episode makes the boot dominate wall-clock. For those two, and only those two, the project solves reuse with **one identical mechanism**: all episodes invoke under **one shared `runtimeSessionId`** (so they land on and reuse **one warm instance**, paying the cold-boot **once**) plus a **per-episode `sub_session`** that gets a **fresh, isolated environment** so reuse costs no correctness. The two substrates differ only in *what the sub_session multiplexes* and the *cache policy*: the **GPU** farm leases a **fractional-GPU Ray actor** per sub_session and keeps the compiled-CUDA-kernel cache **shared+warm** (the whole point of the warm actor; env isolation of `HOME`/`TMPDIR` is opt-in via `SANDBOX_ISOLATE_HOME=1`), while the **CPU** farm has no warm cache to protect so it **isolates everything** — its own workdir *and* `HOME`/`TMPDIR`/caches, wiped on close. Measured on hardware, the CPU farm cut `m5` cold-boots **201 → 1** over a 10-step run (§“Chapter 5 — a warm CPU Instance pool shared by isolated sessions (`demo_cpu_instance_pool`)”); the GPU farm is the same shared-session pattern, verified on the reserved L40S

GPUs in both farm demos — fractional packing (`demo_gpu_farm_fractional`) and whole-GPU-per-episode (`demo_gpu_farm_whole`, run twice) — each with `demo/boot_attempts` p50 = 1 (one warm actor per episode, no cold retries). Detail: `docs/sandbox-fresh-env-on-persistent-instance.md`.

- **Network isolation — pull, don't expose.** Every Runtime is *reached* by an outbound SigV4 `invoke_agent_runtime`; **no Runtime has an inbound port**. IAM/SigV4 is the auth; the `session_id` is routing, not auth. For `networkMode=VPC` the Runtime has no internet route, so it needs interface endpoints (`ecr.api`, `ecr.dkr`, `s3`, `logs`, `sts`, `bedrock-agentcore`), and the ENI must land in an AZ that serves `bedrock-agentcore` — miss one and create fails with a null reason [D10, D11].

5.11 Can AgentCore host every frontier sandbox?

Mapping the task families of §"Tasks frontier labs run in a sandbox" onto AgentCore, the only hard limit is GPU **fractioning** — not any missing capability (✓ native ·◑ needs a BYO container / egress):

Task / sandbox	✓?	How — and the limit
Code exec + unit tests (DeepSeek, Qwen) [2,3]	✓	Code Interpreter, or your image on a Runtime
Math / logic verifier	✓	stdlib checker in the CPU microVM
Tool-use env (Kimi K2) [5]	✓	Python tools; MCP via Gateway
Per-repo Docker image (SWE) [6]	◑	Runtime runs your OCI image, isolated per session — but the managed microVM is **arm64-only** and caps the image at **2 GB** (§3); slim it, defer installs to `CMD`, or use the amd64 Instances substrate
Browser / computer use	✓	managed Browser tool

Task / sandbox	✓?	How — and the limit
Networked RAG / web read	◑	egress — public, or VPC + endpoints [D11]
GPU kernel / CUDA (KernelBench) [7]	◑	Instances only; 1 session = 1 GPU; **no MIG on L4**

In this project the policy is always **Qwen3-4B**, trained by slime, with the harness in-process and AgentCore as the sandbox. What varies from demo to demo is only the **sandbox substrate** and the **task**. The project ships **seven main demos plus one experimental**, ordered as a storyline: which tasks, then the default sandbox, then custom images, then hiding the cold start, then shared Instances with isolation, then scaling, then GPUs. They are catalogued next (§"Demos — the storyline at a glance") and then deep-dived with measured results in §"Demos in depth".

6 Demos — the storyline at a glance

The demos live in `code/demos/`, all built to one design (`docs/demos-program.md`). The policy is always **Qwen3-4B** trained by **slime**, the **harness is in-process**, and **AgentCore is the sandbox**. Every folder is self-contained (its own vendored library, `deploy/`, data and logs) and ships the same six step scripts: `1_prepare_data.sh`, `2_build_image.sh`, `3_push_ecr.sh`, `4_deploy_runtime.sh`, `5_run.sh`, `6_cleanup.sh` (see `code/demos/README.md`). The set tells one story in eight chapters:

1. **The tasks** — what the literature trains in a sandbox: dataset, URL, environment dependencies, and whether a custom image is needed.
2. **Most tasks train on the default runtime** — the managed Code Interpreter, no image (`demo_cpu_tasks`).
3. **Custom images at scale** — one image and Runtime per task (`demo_cpu_terminalbench`).
4. **Hiding the container launch** — pre-open the next batches, and keep idle sessions alive with a heartbeat (the `warmstart/` probe in `demo_cpu_terminalbench`).
5. **A warm CPU Instance pool, shared by sessions, with isolation** (`demo_cpu_instance_pool`).
6. **Scaling the rollout on decoupled tiers** (`demo_cpu_scale_lanes`).

7. **GPU sandboxes** — one GPU per session, a fraction of one GPU, or one whole GPU per episode on a shared instance, with GPU isolation (`demo_gpu_single`, `demo_gpu_farm_fractional`, `demo_gpu_farm_whole`).
8. **Experimental** — one episode, many samples, with the real Claude Code CLI (`demo_cpu_branching`).

#	Demo	Sandbox substrate	Task	Run status
1–2	`demo_cpu_tasks`	managed **Code Interpreter** (default); own Runtime opt-in	any literature RLVR dataset, or a mix	**passed** on the Code Interpreter (2026-09-25): gsm8k 0.90, libs 1.0
3–4	`demo_cpu_terminalbench`	microVM, one image + Runtime **per task** (218)	Terminal-Bench	**passed** (2026-09-23): boot hidden, 4/382 solved; `warmstart/` probe live
5	`demo_cpu_instance_pool`	warm **m5 Instance** pool, Landlock jail per episode	DeepScaleR, > 2 GB image	pool + isolation probes passed; RL smoke **passed** 2026-09-25, eval 0.7875, 2 cold boots
6	`demo_cpu_scale_lanes`	256 pooled microVM sessions	DeepScaleR (fixed)	pool check ok (64 × 64, 2026-09-25); [PENDING: 4 : 256 : 256 RL smoke]; old 4 × 4 × 4 lanes passed 2026-09-23, eval 0.358

#	Demo	Sandbox substrate	Task	Run status
7	`demo_gpu_single`	one `g6.xlarge` (L4) per session	KernelBench	**trained** 10 steps; remote smoke [PENDING: root cause of the op=open 500s]
7	`demo_gpu_farm_fractional`	shared GPU instance, 0.5 + 0.5 of **one** GPU	KernelBench	**passed** (2026-09-25): eval 0.65
7	`demo_gpu_farm_whole`	shared GPU instance, one **whole** GPU per episode	KernelBench	passed (2026-09-21): 0.65 / 0.70
8	`demo_cpu_branching` *(exp.)*	microVM with node + `claude`	SWE bug-fix, real Claude Code CLI	3 blockers fixed, verified without a trainer; [PENDING: branching RL smoke fan_out and reward]

How to read the status column. Only `demo_cpu_tasks` (`DATASET=mbpp`, own-Runtime path) and `demo_gpu_single` carry committed **convergence-shaped** 10-step curves. Everything else is a **loop-verified smoke**: `NR=11` ⇒ 10 train steps + 1 held-out eval + one final checkpoint at iter 10. A smoke proves the plumbing and records a baseline solve rate; it is not a learning curve. No demo has yet been run to convergence (§“Experiment plan”). Several angles that used to be separate folders are now knobs of these demos and keep their old smoke results, attributed below: `HEARTBEAT=1` (every demo), `compare_backends.py` and the `mixed`/`libs` datasets (`demo_cpu_tasks`), and `SNAPSHOT_EVERY=1` plus the custom-Runtime probe (`demo_gpu_single`).

6.1 Chapter 1 — the tasks: what the literature trains in a sandbox

Every task below is a real RLVR dataset from the literature (or a synthetic family built to exercise one dependency), scored by a programmatic checker, not an LLM judge. The last column is the one the rest of the storyline turns on: **does the task need a custom Docker image, or does the default AgentCore sandbox already have its environment?**

Task (`DATASET` / chapter)	Dataset (URL)	Train / eval	Env deps of the checker	Custom image?
`mbpp`	google-research-datasets/mbpp	500 / 8	Python stdlib	no
`humaneval`	openai/openai_humaneval	140 / 24	Python stdlib	no
`gsm8k`	openai/gsm8k	200 / 40	Python stdlib	no
`math500`	HuggingFaceH4/MATH-500	160 / 40	Python stdlib	no
`deepscaler`	agentica-org/DeepScaleR-Preview-Dataset	400 / 40	Python stdlib	no
`gretelsql`	gretelai/synthetic_text_to_sql	300 / 40	`sqlite3`	no
`hotpotqa`	hotpotqa/hotpot_qa (distractor)	300 / 40	10 passage files in `./docs`	no
`xlam`	minpeter/xlam-function-calling-60k-parsed	300 / 40	Python stdlib	no
`mcp`	Team-ACE/ToolACE (MCP-style schemas)	300 / 40	Python stdlib	no
`libs` (datasci + scipy + sklearn)	synthetic (`task_families.py`)	72 / 6	pandas, scipy, scikit-learn	**no** — pre-installed in the Code Interpreter

Task (`DATASET` / chapter)	Dataset (URL)	Train / eval	Env deps of the checker	Custom image?
`mixed`	synthetic mix: math + SQL + CTF + datasci	174 / 9	stdlib + pandas	no
synthetic zoo (`math`, `swe`, `sql`, `ctf`, `rag`, …)	`build_family_datasets.py`	tiny	stdlib	no
Terminal-Bench (Ch. 3)	laude-institute/terminal-bench (~229 single-container tasks)	135 / 31 usable	a **pinned OS image per task** (packages, services, `/etc`)	**yes — one image per task**
DeepScaleR on a heavy stack (Ch. 5)	DeepScaleR (above)	400 / 40	torch + scientific stack, **> 2 GB image**	**yes — CPU Instance**
KernelBench (Ch. 7, GPU)	ScalingIntelligence/KernelBench	100 level-1 / 20 level-2	CUDA + `nvcc` + torch on a GPU	**yes — GPU Instance**
SWE bug-fix (Ch. 8)	synthetic SWE tasks	64 / —	node 22 + the `claude` CLI	yes

The pattern: **every dataset from the RL-for-reasoning and tool-use literature needs nothing beyond Python and its scientific stack**, so it runs on the default sandbox. A custom image is needed only for four reasons: a pinned per-task OS (Terminal-Bench), an image larger than the 2 GB microVM cap, CUDA, or a toolchain the default lacks (the Code Interpreter has no git and no internet, so nothing can be `pip installed` at run time).

6.2 Chapter 2 — most tasks train on the default runtime (`demo_cpu_tasks`)

`demo_cpu_tasks` is the reference demo. `SANDBOX=agentcore` (the default) runs every episode in a managed **AgentCore Code Interpreter** session: no Dockerfile, no ECR push, no Runtime to deploy or delete. You run `1_prepare_data.sh` and `5_run.sh`; steps 2, 3, 4 and 6 print `default Code Interpreter: no image/Runtime needed` and exit 0. `DATASET=<name>` picks any row of the Chapter 1 table, `DATASET=a,b,c` interleaves several into one stream (`eval/mix`), and `DATASET=mixed` / `libs` are prebuilt mixes.

The live environment probe (2026-09-25, us-west-2). The Code Interpreter is Python 3.12.13 on aarch64 with numpy 1.26.4, pandas 2.3.1, scipy 1.16.0, scikit-learn 1.5.0, sympy, torch 2.3.0 (CPU), statsmodels, networkx, matplotlib, requests and sqlite3, plus gcc 11.5 and node v24. It has **no git and no internet**. A Mock-Policy episode of **all 13 task families** scored reward 1.0 on it (`SANDBOX=agentcore python3 smoke_remote_families.py`), including the pandas/scipy/sklearn families that used to need three custom images.

How it works. Each episode opens its own Code Interpreter session (`StartCodeInterpreterSession`), runs its `write_file` / `exec` / `read_file` ops (`InvokeCodeInterpreter`), and stops the session when the episode ends. `AgentCoreSandbox` (`agentcore_slime_demo/sandbox.py`) works in the session's native writable CWD, since `writeFiles` accepts only relative paths, and opens the session off the event loop (`asyncio.to_thread`) so that concurrent episodes do not serialize. Each session is its own isolated microVM; no ARN and no IAM execution role are involved. Measured live per-op latencies: `start_session` 0.87–2.0 s, `write` ~82 ms, `exec` ~251 ms, `read` ~75 ms, `stop` ~92 ms.

Quotas (us-west-2 defaults, `aws service-quotas list-service-quotas --service-code bedrock-agentcore`):

Quota	Code	Default	This demo
Concurrent active Code Interpreter sessions	L-CAF6F552	1000	`S = CONC` × engines = 4 × 2 = **8** in flight
`StartCodeInterpreterSession` rate	L-6B09F374	30/s	about one per episode

Quota	Code	Default	This demo
`InvokeCodeInterpreter` rate	L-04A719E9	30/s	about 6–10 per episode, spread over turns

The invoke rate is the one that binds: at about 8 ops per episode and 20 s per episode, 30/s allows roughly 75 episodes in flight. Raise it before running with `CONC` ≳ 32.

Measured on the Code Interpreter.

- **`DATASET=gsm8k` smoke — PASSED** (2026-09-25, 11:15–11:35 UTC, `NR=11`; `logs/forkA-gsm8k_20260925_111527.*`). `rollout/raw_reward` per step 0.906, 0.969, 1.0, 0.938, 0.8125, 0.8125, 0.969, 1.0, 1.0, 1.0; held-out `eval/gsm8k` **0.90**; `perf/step_time` ~42–54 s; 425 Code Interpreter sessions opened and closed, 0 infrastructure errors. GSM8K is near-saturated for the base policy, so this is a **plumbing proof on the default runtime, not a learning curve**.
- **`DATASET=libs` smoke — PASSED** (2026-09-25, 12:34–12:53 UTC, `NR=11`; `logs/forkA-libs_20260925_123428.*`): pandas / scipy / sklearn on the default runtime with **no image**. `rollout/raw_reward` 1.0 on 9 of 10 steps (0.969 on the third); `eval/libs` **1.0**; `perf/step_time` 44.7–53.4 s; 349 episodes (datasci 132, scipy 109, sklearn 108); **1883 sandbox ops, all ok**. Per op: `start_session` p50 1.83 s / p95 3.50 s — the largest sandbox cost per episode, paid on every episode — `write_file` 69 ms, checker exec 1.34 s (it imports pandas/scipy/sklearn), `stop_session` 150 ms. No cold boot ≥ 5 s.
- **Honest caveat.** Both gsm8k and libs are **near-saturated** for Qwen3-4B (zero-variance groups on most libs steps), so these smokes prove the no-image path works end to end, not that the policy learns; harder task sets are needed for a curve.
- Eleven earlier 10-step smokes (2026-09-18→19) ran on the **own-Runtime path** (`mbpp` 0.69, `math` 1.00, `gsm8k` 0.90, `math500` 0.89, `deepscaler` 0.78, `humaneval` 0.85, `gretelsql` 0.60, `hotpotqa` 0.65, `xlam` 1.00, `mcp` 1.00, `mixed` 0.67 held-out solve rate; §“`demo_cpu_tasks` per dataset in depth”). The task, checker and reward are identical on both backends (`compare_backends.py` asserts it), so these carry over; re-running them on the Code Interpreter is [PENDING: CI smokes for the other DATASET values].

Why most tasks need no image. The sandbox contract is task-agnostic: `write_file` / `exec` / `read_file`. Everything task-specific — the prompt, the seed files, the hidden checker — is written into the session at seed time, not baked into an image. A math, a SQL and a tool-use episode are just different files in three identical microVMs. An image matters only when the *checker's environment* does, and the Code Interpreter's environment already covers the literature's CPU tasks.

The image-routing chain (for when a task does need an image). Each data row is self-describing (`metadata.task`). The chain is:

1. `metadata.task` → the task class, whose `image` ClassVar names the image it needs (`None` = any);
2. → `make_sandbox(image=⋯)`;
3. → `_runtime_arn_for_image`, which looks the tag up in the per-image ARN map (`AGENTCORE_SANDBOX_RUNTIME_ARN_BY_IMAGE`);
4. → otherwise round-robin over `AGENTCORE_SANDBOX_RUNTIME_ARNS`, sticky per episode.

AgentCore never sees the task; the routing is entirely client-side. Terminal-Bench (Chapter 3) uses a strict variant, in which an unmapped task is an error.

What `acs_tasks_cpu` is. It is the **opt-in own Runtime** for this demo (`SANDBOX=agentcore-runtime`). Steps 2–4 then build `deploy/Dockerfile.sandbox.cpu` (`python:3.12-slim` plus the sandbox server, **stdlib only**), push it to ECR `acs-tasks-cpu`, and register the Runtime `acs_tasks_cpu`. It exists for the backend comparison and the Runtime-count ablation, not because the tasks need it. Because it has no pandas, `DATASET=libs` scores 0 on it unless you add those libraries to the image. `compare_backends.py` runs the same episode on the Code Interpreter and on this Runtime and asserts the same reward; only the operations differ (who builds the image, which deps you get, the deploy cost).

Use case covered. RLVR post-training on the literature's datasets — code (MBPP/HumanEval [8]), math (GSM8K/MATH-500/DeepScaleR [2]), text-to-SQL, multi-hop RAG, tool/function calling, and data-science tasks (pandas/scipy/sklearn) — one at a time or mixed into one multi-family stream.

Why this demo. It answers the first question a customer asks: *do I have to build a sandbox image?* For most tasks, no — the default managed Code Interpreter already has the dependencies, so this is the lowest-ops entry point (no build, no deploy, no

cleanup). It is one demo, not one per dataset, because every dataset runs the same harness, sandbox and trainer and differs only in its data.

Design considerations.

- **Code Interpreter as the default**, own Runtime (`acs_tasks_cpu`) opt-in; steps 2/3/4/6 are no-ops on the default.
- **One harness for every task**: self-describing rows (`metadata.task`) rebuild the right task per prompt, and `task.image` routing picks a Runtime only when a task needs one.
- **Mixes are safe for GRPO**: `DATASET=a,b,c` interleaves datasets, and the advantage is normalized within a group of repeats of one prompt, so the families' reward scales never mix.
- **Heartbeat is a no-op on the Code Interpreter**; the session-timeout setting (`AGENTCORE_CI_SESSION_TIMEOUT`) is the lever instead.
- **Quotas**: the invoke rate (30/s) binds first; raise it before `CONC` $\gtrsim$ 32.
- **Rejected: a per-library image pool.** The former three-image pandas/scipy/sklearn pool was dropped, because the Code Interpreter already has those libraries (`DATASET=libs` runs them with no image).

6.3 Chapter 3 — custom images at scale (`demo_cpu_terminalbench`)

Terminal-Bench ships each task as its own container (a task `Dockerfile` plus a hidden pytest oracle), and the tasks act on the **whole OS**: they install packages, edit `/etc`, start services. So this demo builds **one image and one AgentCore Runtime per task** and gives every episode a **brand-new session — its own microVM, never reused**.

- **The pool.** `deploy/deploy_terminalbench.py` builds and pushes one arm64 image per task (ECR `acs-tb-*`) and registers one Runtime per task (`acs_tb_*`), writing `tasks/arn_by_image.json`. **218 of 229** deployed (7 builds failed, 4 images were too large). The oracle check (`validate_oracle.py`: an untouched environment must fail and the reference solution must pass) left **166 usable** (47 reference solutions fail on this substrate, 5 errored, 0 pass untouched), which gives a 135-train / 31-eval dataset.
- **Strict routing.** The episode's `task.image` (the task name) is looked up in `AGENTCORE_SANDBOX_RUNTIME_ARN_BY_IMAGE`. An unmapped task is an error;

it is never sent to a fallback Runtime, which would run it on another task's image.
- **The loop.** The tool-calling `MiniClaudeHarness` drives the policy through the in-process adapter (it needs SGLang's `qwen25` tool parser). The reward is the task's own hidden oracle, run by `grade.sh` after the agent's turns; the oracle is baked at `/opt/acs/oracle`, outside the workdir. REMOTE-ONLY.
- **Quota.** Active Session Workloads 5000 (L-3E5722B2), far above the 64 pre-opened sessions plus the running ones.

Smoke — PASSED (2026-09-23, `_smoke_logs/lanes_tb_0923/terminalbench.log`): 382 episodes in ~70 min, `Job succeeded`, 0 infrastructure errors. The cold start was **fully hidden**: `open_wait_ms` p95 0, open p50 0.66 s, 100 % warm hits. The price: `preopen_idle_ms` p50 7.3 min against an episode p50 of 21 s, so about **95 % of the pre-opened microVM time was idle**. The reward is sparse for a 4B policy: 4 solves in 382 episodes, `eval/terminalbench` 0.0, though 42 % of episodes pass at least one test. Next steps are `TB_PREOPEN_BATCHES=1` (cost) and an easy-first curriculum or partial-credit reward (signal).

Use case covered. Agentic terminal and SWE-style tasks where every task pins its own OS image — Terminal-Bench's ~229 containerized tasks [6], graded by each task's hidden tests.

Why this demo. It is the other side of Chapter 2: when a task *does* need a custom image, how do you run **hundreds** of them? One image and one Runtime per task, with strict per-task routing. It is its own demo because the substrate differs from `demo_cpu_tasks` in every respect that matters: per-task images, a fresh microVM per episode, and a tool-calling loop.

Design considerations.

- **A fresh microVM per episode, never reused.** The tasks mutate the OS, so no state may cross episodes or GRPO siblings.
- **Oracle validation before training.** `validate_oracle.py` drops tasks whose reference solution fails on this substrate (47 of them).
- **Look-ahead pre-open** hides the cold start, at the price of idle booted sessions (`TB_PREOPEN_BATCHES`, Chapter 4). The **heartbeat** is always on, so that pre-opened sessions survive.
- **Only single-container tasks** are poolable (one image ↔ one sandbox); multi-service tasks are skipped.

- **Quota**: the pre-opened plus the running sessions count against Active Session Workloads (5000).
- **The `warmstart/` probe** isolates the two cold-start mechanisms cheaply, without a trainer.

6.4 Chapter 4 — hiding the container launch: pre-open and heartbeat

A fresh session per episode means a cold start per episode: allocate a microVM, pull the task's image, boot it. Chapter 3 hides it with two mechanisms, both always on in `demo_cpu_terminalbench`; the `warmstart/` probe inside that demo measures each in isolation.

Use case covered. Any rollout that opens a fresh sandbox per episode (and so pays a container launch each time), and any run whose sandboxes sit idle across the trainer's weight-sync pause.

Why this probe. The Terminal-Bench smoke showed that pre-open works but wastes ~95 % of the booted time, which is too expensive to tune at full scale (~70 min, 229 Runtimes). The probe answers the two questions — *how much does pre-open save?* and *does a heartbeat keep a session alive?* — in about 7 minutes, for a few cents, on both backends. It is a probe folder inside `demo_cpu_terminalbench`, not a separate demo, because it tests that demo's two mechanisms.

Design considerations. MockPolicy episodes, with no GPU, model or trainer, so that only the sandbox is measured. A 60 s idle timeout, shortened from the 15-min default that training Runtimes keep, makes reclaim observable in minutes. The same episodes run on the own Runtime and on the Code Interpreter, because the two behave differently under a heartbeat. The pre-open depth is one batch here and `TB_PREOPEN_BATCHES` (default 2) in the real demo.

Pre-open the next one or two batches. slime hands the pool only `CONC` × `engines` episodes of the current batch and draws the next batch only when this rollout ends, so the pool alone cannot see far enough ahead. A **look-ahead data source** (`tb_lookahead_source.py`, wired with `--data-source-path`) replays slime's deterministic prompt order (a shuffled dataset walked in order, fixed seed per epoch) and, on every draw, tells the pool the episodes of the **next `TB_PREOPEN_BATCHES` batches** (default 2). The pool keeps that many upcoming episodes booted — `TB_PREOPEN_BATCHES` × `RB` × `NS` = 2 × 4 × 8 = **64** — each

on its own new session on its task's Runtime. When a worker claims one, the next upcoming episode starts booting; a misprediction is trimmed at the next draw and those episodes open on demand. `TB_PREOPEN_BATCHES=0` turns it off.

Heartbeat across the weight-sync stall. While the trainer syncs weights into SGLang, generation pauses and sessions send no invokes; an own-Runtime session idle past its timeout is reclaimed. A `SessionKeeper` (`session_keeper.py`) registers every open session and pings (`exec("true")`) any that has been idle past `WARM_PING_FRACTION` × `WARM_IDLE_TIMEOUT` (0.5 × 900 s). In `demo_cpu_terminalbench` it is always on; every other demo has it as the opt-in knob `HEARTBEAT=1` (default off, so a run is unchanged unless set).

The `warmstart/` probe isolates both mechanisms (`bash warmstart/run_warmstart.sh`; host-only, no GPU, no model; about 7 min and a few cents). The same M = 24 MockPolicy `sql` episodes run in batches of 6 (think time 2 s) against a stdlib Runtime `acs_warmstart_cpu` with a 60 s idle timeout, and against the Code Interpreter. Recorded run `warmstart/logs/warmstart_20260925_100103.*` (2026-09-25, us-west-2, all rewards 1.0):

Backend	Mode	Wait p50	Episodes that waited	Idle held	Makespan
own Runtime	cold	774 ms	24/24	0 s	31.9 s
own Runtime	pre-open	**5 ms**	8/24	75 s	25.5 s (−20 %)
Code Interpreter	cold	1715 ms	24/24	0 s	28.7 s
Code Interpreter	pre-open	**0 ms**	6/24	70 s	24.3 s (−15 %)

Backend	Heartbeat	Sessions surviving 150 s idle	Resume
own Runtime	on (14 pings)	**3/3**	104 ms, files intact

Backend	Heartbeat	Sessions surviving 150 s idle	Resume
own Runtime	off	0/3	**silently reclaimed**: the same session id returns a fresh, empty sandbox (948 ms)
Code Interpreter	on or off	0/3	expires at its session timeout regardless of pings

What the probe shows:

- **Pre-open takes the boot off the critical path; only the first batch pays it.** The cost is idle held: booted sessions waiting for work. It is the same trade-off the Terminal-Bench smoke's 95 % idle shows at full scale, which is why `TB_PREOPEN_BATCHES=1` is the next thing to try there.
- **Heartbeat only helps an own Runtime.** Without it, a reclaimed session *looks healthy* but has lost its files — the failure is silent, which is why Terminal-Bench's keeper is always on.
- **On the Code Interpreter, heartbeat is a no-op.** A session ends at the `sessionTimeoutSeconds` set when it starts, however much traffic it gets. `run.sh` warns and turns `HEARTBEAT` off for `SANDBOX=agentcore`, and the library logs one `[demo] heartbeat_skipped=code_interpreter` line. The lever there is `AGENTCORE_CI_SESSION_TIMEOUT` (default 3600 s, max 28800 s); every session is stopped when its episode ends, so a generous timeout costs nothing.

6.5 Chapter 5 — a warm CPU Instance pool shared by isolated sessions (`demo_cpu_instance_pool`)

Some checkers need an image over the microVM's 2 GB cap — here torch plus a scientific stack (`acs-sandbox-bigcpu`, ~4 GB). That image cannot be a microVM; it runs as your own Runtime on a **capacity-provider CPU Instance** (`m5.xlarge`). An

Instance cold-boots in 60–80 s, so a fresh Instance per episode is ruinous: the old per-episode-Instance run paid about **32 cold boots per step** (375 in total; `raw_reward` 0.66–0.97, `eval/deepscaler` 0.7625), and an earlier single-warm-instance run cut 201 boots to 1.

The warm pool. `SANDBOX=agentcore-ray-farm` with `SANDBOX_POOL=N` (default 2) mints N pool session ids per run (`cpupool-<run>-m00 ⋯ m0(N-1)`). AgentCore pins each `runtimeSessionId` to its own Instance, so the pool is **N warm m5s**, and the cold boot is paid N times per run instead of once per episode. Each episode is assigned to the **least-loaded** pool member (round-robin on ties) and runs as its own per-episode `sub_session` on it; closing an episode closes only its `sub_session`, never the pool session. One Runtime serves the whole pool, so the pool size is purely client-side. Idle pool Instances are reclaimed after the Runtime idle timeout (600 s). Runtime `acs_sandbox_bigcpu-ZM6bXj80yC` on capacity provider `acs_cpu_cp_bigimage-tNyZnyHYGr`.

Measured. An offline pool test split 9 episodes 5 / 4 over the two members, and the load counters returned to 0. A live probe (`SANDBOX_POOL=2`, MockPolicy, 6 episodes) used **2 distinct Instances** with **2 cold boots** (83.1 s and 83.6 s, in parallel), then warm opens of **121–328 ms**, reward 1.0 on all 6. The RL smoke is `smoke_pool2_20260925_113657`: 10 steps, job succeeded; 406 episodes on exactly 2 Instances (210 / 196), **2 cold boots** (only 4 episodes in step 1 waited, 63 s); warm op p50 117 ms / p95 261 ms; step time 38–219 s vs ~715–753 s before pooling; `raw_reward` 0.72–1.0; **`eval/deepscaler` 0.7875**.

6.5.1 Isolation on a shared m5 — what is guaranteed, and what is not

On a microVM, AgentCore guarantees the isolation: one session is one VM with its own kernel. Sharing one Instance between sessions gives that boundary up for warm reuse, so the farm (`sandbox_cpu_farm.py` + `sandbox_farm_jail.py`) puts back as much isolation as **one unprivileged kernel user** can.

Why not a uid per episode. Measured live on the m5 (2026-09-25), the AgentCore container is uid 0 only inside a user namespace (uid_map `0 100000 65536`), with `CapEff = CapBnd = 0`, `NoNewPrivs=1` and seccomp. `setuid`, `chown`, `unshare` and cgroup writes are all EPERM. The first hardened image (`:uidiso-20260925`, one uid per slot) failed every farm open on chown (HTTP 500), and its `uid_isolation: false` fallback did not trigger because it only checked `geteuid()==0`. What the kernel

(6.18) does allow an unprivileged process is **Landlock ABI 7**, so every exec runs in **its own Landlock domain**. `describe()["mode"]` reports `landlock` on AgentCore, and `uid+landlock` where the capabilities exist (local docker), which adds a uid per slot.

	microVM per session (AgentCore guarantees)	shared m5 `sub_session` — **enforced** by the farm	shared m5 — **still shared**
Filesystem	own root fs, discarded with the VM	Landlock: read+exec on the system dirs, read+write **only its own dir**; sibling dirs, farm root, `/tmp`, `/var/tmp`, `/dev/shm`, `/workspace`, `/root` all EACCES; HOME / TMPDIR inside the dir, `umask 077`; server `read_file` / `write_file` refuse `..`, absolute and symlink escapes	the image's read-only system files; `/dev/shm` if `SAND-BOX_FARM_SHM=allow` (needed by `multiprocessing`)

	microVM per session (AgentCore guarantees)	shared m5 `sub_session` — **enforced** by the farm	shared m5 — **still shared**
Process / user	own process tree	signals, `ptrace` and `/proc/<pid>/{environ,cmdline,mem}` scoped to the episode's Landlock domain; a subreaper kills the whole tree when each exec ends, so nothing outlives its exec	one uid (0 in the userns), one PID namespace: `ps` shows command lines
RAM / CPU	own allocation	watchdog (5–100 ms polling) kills the tree over 256 processes or 4 GiB RSS; FSIZE=1 GiB, CORE=0; per-op timeout	no per-episode cgroup: one episode can take the CPU, a fast spike between polls can OOM a neighbour
Network	own interface	Landlock TCP: every `connect` / `bind` denied (server port 8080, IMDS, internet); the server's `AWS_*` env is stripped	one netns; **UDP and DNS not covered**
Kernel	own guest kernel	—	**one kernel**: an exploit or a Landlock bypass breaks everything above
Blast radius	one session	—	**one instance**

Lifetime and keys. Close deletes the episode's dir. The `sub_session` id is kept off every command line (`/proc/*/cmdline` is world-readable, and the id is the key to the farm API) and is regex-validated against traversal. Costs: about +50 ms per exec, and a task that starts a server in one exec and uses it in the next must do both in one exec. Knobs: `SANDBOX_FARM_{NET,SHM,NPROC,FSIZE_MB,UID_ISOLATION}`.

Verified. `isolation_probe.py` has episode A plant a secret file, try `/tmp` and run a setsid'd daemon inside a long exec, while episode B tries to reach all of it. **Before the hardening**, B could read A's file by explicit path (`cat ../<A>/marker.txt`). **With the jail, 22/22 checks pass in all three settings** (2026-09-25): offline in `agentcore_p4` (`uid+landlock`), the image under `docker run --cap-drop ALL --security-opt no-new-privileges` (the AgentCore simulation), and **live on one warm m5** (Runtime v3, image `acs-sandbox-bigcpu:landlock-20260925`). Live, A and B had the same boot_id and both ran as uid 0 in `landlock-abi7`; B could not list the farm root or A's dir, read A's secret by path or via `../`, or write into A's dir; the server's file ops refused A's paths; B's `kill -0` and `cat /proc/<pid>/environ` of A's daemon were denied; `AWS_*` was not inherited; a 1.2 GB write stopped at 1 GiB and a 400-process fan-out was killed; `curl` to `127.0.0.1:8080` and `169.254.169.254` failed; A's daemon was gone once its exec ended. One MockPolicy episode of each of the 13 task families, run concurrently on the same warm m5 under the jail, all scored 1.0; the live warm exec p50 was 156 ms (117 ms in the pre-jail smoke).

When is it enough? It is **good enough for RLVR**, including a policy that stumbles into reward hacks: reading a sibling's `solution.py`, killing a neighbour's checker, calling the server port, or leaving a daemon that tampers with the next episode. It is **not** a defence against a kernel exploit, and it gives no guaranteed CPU/RAM or private network stack. For those, and for tasks that mutate the whole OS (Terminal-Bench), use a **microVM per session**: the Code Interpreter or a microVM Runtime.

Use case covered. Tasks whose checker needs a big image (> 2 GB) or heavy dependencies — a torch + scientific stack here — which must run on a CPU **capacity-provider Instance** (`m5`) rather than a microVM.

Why this demo. An Instance cold boot (~80 s) is too costly to pay per episode, so many sessions must share a few warm Instances. That raises the question the user asked to see answered: *how is isolation kept on a shared host?* It is its own demo because the substrate (Instances), the sharing model (a pool of warm sessions

with per-episode `sub_sessions`) and the isolation story all differ from the microVM demos.

Design considerations.

- **`SANDBOX_POOL=N`**, least-loaded assignment, one Runtime for the whole pool; idle Instances are reclaimed after 600 s.
- **A per-episode `sub_session` in its own Landlock jail** (table above; per-episode uids are impossible in AgentCore's capability-less container), plus kill-and-scrub.
- **Honest limits**: a shared kernel, network namespace and CPU/RAM. When a hostile-kernel boundary is required, use a microVM per session.
- **Rejected**:
  - a fresh Instance per episode, which paid ~32 cold boots per step;
  - a single warm Instance, which has no parallel capacity and whose fault domain is the whole run.

6.6 Chapter 6 — scaling the rollout on decoupled tiers (`demo_cpu_scale_lanes`)

This demo fixes the task (DeepScaleR math in coding shape) and the sandbox image, and scales the **rollout tiers themselves**, to make the §"Scaling the loop" law concrete: rollout throughput is `min(inference, sandbox, harness)`. The default is **4 SGLang engines (TP1, rollout GPUs 4–7) : 256 warm in-process harness workers : 256 persistent AgentCore microVM sessions**, and the three are **decoupled**: no worker is tied to an engine and no microVM to a worker. Each episode is joined from three pools when it starts. A free worker pulls it from one shared queue, borrows any free microVM from a free-list of sessions, and then takes the engine with the fewest in-flight episodes, which serves all of that episode's turns. `HARNESS_WORKERS`, `SANDBOX_SESSIONS` and `ROLLOUT_GPUS` each move one tier alone, and the log names the tier that binds: `qdepth` > 0 (harness), `sandbox_wait_ms` > 0 (sandbox), rising `engine_inflight` (inference).

Measured. A live pool check without a trainer (2026-09-25) ran the real pool against the real K = 2 Runtimes with fake inference, at W = M = 64 and 192 episodes: 1280 sandbox ops all ok, 64 sessions opened and stopped, every workdir fresh, episodes per engine 48 / 48 / 47 / 49, and the tiers mixed freely (a worker used up to 2 sessions and p50 2 engines; a session served up to 2 workers). `open_ms` p50 was

5.45 s; `sandbox_wait_ms` p95 11.8 s came only from the opening ramp. [PENDING: 4 : 256 : 256 RL smoke.] The previous topology, 4 × 4 × 4 pinned lanes, passed an AWS smoke on 2026-09-23 (`_smoke_logs/lanes_tb_0923/scale_lanes.log`: 4 opens for 400 episodes, `raw_reward` 0.19–0.50, `eval/deepscaler` 0.358). The offline sweep (`harness_scaling_demo.py`) shows the `min()` law directly (§"`demo_cpu_scale_lanes` in depth").

Use case covered. High-throughput rollout: an RL fleet that has already scaled inference and the sandbox and now hits the forgotten third tier, the harness driver.

Why this demo. It shows `throughput = min(inference, sandbox, harness)` with the three tiers as **independent pools**: 4 SGLang engines, 256 in-process harness workers and 256 persistent microVMs. Harness and sandbox scale for different reasons (Python concurrency vs microVM quota and cost), so they must be two separate numbers; pinning them together, as the old 4 × 4 × 4 lanes did, hides which one binds. It is its own demo because it holds the task and sandbox image fixed and varies only the topology, which no other demo does.

Design considerations.

- **Three pools, joined per episode**: pull from a shared episode queue (harness), a free-list of sessions (sandbox), least-loaded engine with rotating ties (inference). The sandbox is taken first, so an episode never holds an engine slot while it waits for a microVM.
- **Isolation of a reused microVM**: a session serves one episode at a time; each episode gets a fresh `/workspace/ep-<hex>` that is `rm -rf`'d before the session goes back on the free list; different sessions are different microVMs. Not reset between episodes of one session: `/tmp`, packages, stray processes. That is safe for DeepScaleR, which writes only its workdir; OS-level tasks use `SANDBOX_SESSIONS=0` (a fresh session per episode, as in Chapter 3).
- **Opening 256 microVMs**: all sessions open once, paced at `SANDBOX_OPEN_RATE` = 20/s against the 25 new-sessions/s quota (ready in about 13 s plus one boot); a failed open shrinks the pool instead of failing the run; all are stopped at exit.
- **The harness process must not be the silent ceiling**: one boto3 client with `max_pool_connections` = 272 (default 10) and a 288-thread default executor (the stock pool caps at 32); one `aiohttp` session with no connection limit.
- **Admission**: the stock driver keeps one batch in flight, so `RB` × `NS` = 32 × 8 = 256 episodes per step, and `CONC = ceil(W / E)` = 64.

- **Honest limit**: in-process workers lift the IO ceiling, not the GIL; at W = 256 the adapter's CPU-bound trajectory flattening is the part to watch.

6.7 Chapter 7 — GPU sandboxes: one GPU, a fraction of a GPU, a whole GPU per episode

GPUs exist on AgentCore only on **capacity-provider Instances**, and **one session = one whole Instance**. The three GPU demos are three answers to the question *how many episodes share a GPU?* All three train KernelBench: the policy writes a CUDA `ModelNew`, and the checker rebuilds the reference with matched seeds, asserts `allclose`, and reports a speedup. The two farm demos serve the Ray farm from the box's own GPUs over the identical `/invocations` contract (`code/deploy/run_local_gpu_runtime.sh fractional|whole`). slime keeps the other GPUs; the `SLIME_CUDA_VISIBLE` / `SLIME_NUM_GPUS` knob is a kept, env-only GPU-subset pin whose default (unset) leaves the 8-GPU placement unchanged.

6.7.1 `demo_gpu_single` — one warm custom GPU Runtime

Use case covered. GPU-in-the-loop verification: RL on CUDA kernels (KernelBench [7]), where the reward itself requires running untrusted code on a real GPU, isolated from the training state.

Why this demo. A GPU exists only on Instances; the managed Code Interpreter has none (`gpu_torch_smoke.py` shows `cuda_available=False` there and `True` on a custom g6 Runtime). This is the simplest GPU sandbox — one custom Runtime, one `g6.xlarge` (L4, amd64, ECR `acs-sandbox-gpu`), and nothing shared on the GPU — and the baseline the two farm demos pack against.

Design considerations.

- **Capacity provider `acs_gpu_cp`** with a warm-shared session, so the ~207 s cold boot is paid once and then reopens take ~150–190 ms.
- **torch pip-installed at container start** (~90 s) to stay under the image cap.
- **The trainer box keeps its GPUs for slime** (4 train + 4 inference); only the checker is remote.
- **`SNAPSHOT_EVERY=1`**: an application-level snapshot of the workspace to a store you own, restored after an instance recycle. There is no usable VM/EBS GPU snapshot (`docs/gpu-snapshot.md`).

- **`LOCAL_GPU=1`**: an on-box option with a synchronous checkpoint save.

Measured.

- The committed 10-step run: `raw_reward` ~0.28–0.94, `eval/kernelbench` 0.65. exec p50 8.5 s ≫ generation 2.5 s, so the demo is sandbox-bound.
- Earlier smokes:
  - the GPU-farm smoke, 0.525;
  - `SNAPSHOT_EVERY=1`, 0.625, with 3 files saved and restored per episode;
  - the custom-Runtime angle, 0.65.
- A new remote smoke (2026-09-25, `logs/smoke-remote_20260925_113622.*`) completed **4 steps**:
  - `raw_reward` 0.531, 0.469, 0.25, 0.5625;
  - one 207 s cold boot, then warm opens of 150–190 ms;
  - sandbox op p50 148–177 ms, p95 5.4–17 s.
- It then **failed** (rc=1) on `InvokeAgentRuntime` `RuntimeClientError` 500s during op=open retries. Root cause: [PENDING: root cause of the demo_gpu_single remote-smoke op=open 500s].

6.7.2 `demo_gpu_farm_fractional` — two episodes on one GPU

Use case covered. Many light GPU episodes — small kernels that use a fraction of a card — packed onto one GPU.

Why this demo. It raises GPU utilization and cost efficiency, and it works around capacity. Isolating every episode on its own g6 needs N instances at once, which stalls on a capacity-starved fleet; the fractional farm needs one GPU. It is separate from `demo_gpu_farm_whole` because its trade-offs (a shared card, a shared fault domain) are the opposite.

Design considerations.

- **The farm.** All episodes invoke one GPU Runtime under a single shared `runtimeSessionId`. Inside it, a Ray farm leases one fractional actor per episode `sub_session` (`B4_FARM_GPUS=1`, `B4_GPU_FRACTION=0.5` → two actors on GPU 6).

- **The per-process memory cap** (`B4_PER_SANDBOX_MEM_FRACTION=0.5`) is **co-operative**.
- **No MIG on L4 / L40S**, so the card is one shared fault domain.
- **Wedge mitigation**: the non-MIG GPU wedges under sustained `nvcc` load, which the `nvcc flock` and `fraction=0.5` mitigate.
- **Admission control**: a client-side open-concurrency gate caps opens at the farm's slot count.

Measured (2026-09-25, `logs/smoke-local-fractional_20260925_095218.*`, ~83 min, slime on GPUs 2–5 because GPUs 0 and 1 were busy):

- `Job succeeded`, `eval/kernelbench` **0.65** (0.775 on 2026-09-21);
- `raw_reward` 0.72, 0.59, 0.22, 0.69, 0.72, 0.19, 0.72, 0.75, 0.63, 0.44;
- 359 episodes split 181 / 178 across the two actors, with lease wait always 0;
- exec p50 9.0 s / p95 19.8 s, warm open 4.2 ms;
- checker: PASSED 325 (mean 0.64), IMPORT_ERROR 18, NO_OUTPUT 16.

6.7.3 `demo_gpu_farm_whole` — one whole GPU per episode, on one shared instance

Use case covered. Episodes that need a full GPU each — heavier kernels, or checkers that must not contend for memory — on a multi-GPU instance.

Why this demo. An AgentCore session is a whole instance, so the native API cannot give one GPU of a multi-GPU box to each session. Two session ids get two instances, not two GPUs of one. The Ray farm fixes that: one shared session, and one whole GPU per `sub_session`.

Design considerations.

- **Same farm as the fractional demo**, with `B4_FARM_GPUS=2` and `B4_GPU_FRACTION=1.0` (GPUs 6, 7).
- **Capacity = the GPU count.** `CONC` gating queues overflow episodes in-process, instead of timing them out at the server.
- **No memory contention and no co-tenant wedge on a card**, at the price of one GPU per concurrent episode.

Measured.

- Two 10-step smokes (2026-09-21) scored `eval/kernelbench` **0.65** and **0.70**.

- A 2026-09-19 run failed when the farm container died; that is fixed since with `--shm-size` and `--restart=on-failure`.
- Not re-run since the split into its own folder: [PENDING: demo_gpu_farm_whole re-run after the split].

6.7.4 GPU isolation — what a shared GPU instance guarantees

The farm's host-side isolation is the CPU pool's uid mode (the Landlock jail is not ported to the GPU farm yet): every `sub_session` runs as its own uid in a private `0700` dir with `prlimit` caps and kill-and-scrub on close (`sandbox_ray_farm.py`). The GPU adds its own layer:

Layer	Enforced	Still shared
CUDA context	a separate CUDA context per actor process; the driver keeps their memory apart	the same physical GPU and driver
GPU memory	`B4_PER_SANDBOX_MEM_FRACTION` via `sitecustomize` — **cooperative**, not a hard limit	a process can bypass it
Caches	`SANDBOX_ISOLATE_HOME=1` gives each episode its own CUDA / compile cache	by default the warm cache is shared: faster, but a poisoning risk
Faults	`nvcc flock` and `fraction=0.5` mitigate the compile-storm wedge	**no MIG on L4 / L40S**: a GPU fault or wedge hits every co-tenant
Kernel / host	as the CPU pool	one kernel, one instance

So a fractional GPU is **software isolation on shared hardware**. It is fine for RLVR on kernels you trust not to attack the driver; a whole GPU per episode (`demo_gpu_farm_whole`) removes the memory contention and the co-tenant wedge; only a separate instance per session (`demo_gpu_single` with one session per episode) gives a hardware boundary. Live GPU isolation check (2026-09-25, local farm on GPU 6): slot A and slot B ran as separate uids 20000 / 20001; B was denied A's files, dirs and home; `../`, absolute and symlink escapes were refused; 4

separate CUDA contexts, the 0.5 memory cap held in both slots; closing A left no process and removed its dir. On the AgentCore g6 (no capabilities) the farm falls back to one shared uid.

6.8 Chapter 8 (experimental) — one episode, many samples: `demo_cpu_branching`

The one demo whose harness is the **real Claude Code CLI**, not the in-process mini-agent. The CLI runs next to slime, wired to its `AnthropicAdapter`, while its tools run in a remote CPU Runtime whose image bakes node 22 and `claude` (`deploy/Dockerfile.sandbox.cli`). It provokes the two CLI behaviours that split one episode into several branches — sub-agents (the `Task` tool) and context compaction — so one rollout fans out into a `list[Sample]`. A custom generate hook (`claude_code_generate.py`) linearizes the trajectory tree, and a custom advantage (`rollout_advantage.py`) baselines over **distinct episodes, not leaves** (§"When one episode becomes many samples"). **Status: no longer blocked** (details below the design considerations); the RL result is [PENDING: branching RL smoke fan_out and reward].

Use case covered. A real coding-agent CLI (Claude Code) as the harness, where one episode fans out into sub-agents and compacted contexts — a multi-trajectory episode.

Why this demo. Every other demo maps one episode to one sample. This one tests what happens when an episode becomes a tree of samples, and whether the update stays unbiased (it must not reward an episode for spawning more leaves).

Design considerations.

- **Rollout-level GRPO**: the advantage is baselined over episodes, with a rollout-level token-mean loss.
- **The CLI runs next to slime; only its tools run in the sandbox**, a CLI-capable Runtime.
- **`MODE=A`, `dp_size=1`** (TP=2 × PP=2), with a synchronous checkpoint save.
- **The CLI needs the tool parser and a per-turn cap above Qwen3's `<think>`**: `run.sh` passes `--sglang-tool-call-parser qwen25 --sglang-reasoning-parser qwen3`, and the per-turn cap is `MAX_RESPONSE_LEN` (default 2048).
- **Kept experimental** until an RL smoke shows fan-out and a non-zero reward.

Status — three blockers, all fixed.

1. **The CLI's root guard.** Fixed with `IS_SANDBOX=1` (2026-09-19).
2. **One-turn episodes with reward 0.** The 2026-09-19 smoke exited 0 with `fan_out=1` and reward 0 because `run.sh` lacked `--sglang-tool-call-parser qwen25 --sglang-reasoning-parser qwen3`: the adapter returned Qwen3's `<tool_call>` as a plain `end_turn` text reply, so the CLI stopped after one turn. Fixed 2026-09-25.
3. **CLI exit 1, "response exceeded the output token maximum".** A turn hit the 1024-token per-turn cap (`stop_reason=max_tokens`). The cap is now `MAX_RESPONSE_LEN`, default 2048.

Verified without a trainer: the session starts, `claude` 2.1.278 runs as root, the dial-back to `:18021` works, the parser extracts the tool call, and the checker scores 0 / 0 / 1. The RL result is [PENDING: branching RL smoke fan_out and reward].

7 Demos in depth — architecture, pipeline, lifecycle, and measured results

This section is the deep dive behind the storyline chapters. Because the training demos share one architecture — in-process harness, AgentCore sandbox, slime trainer — the first subsection establishes the **shared skeleton** once (the pipeline, the step-by-step, the lifecycle, the scaling method, and the metric glossary); the second covers how to **size the two config knobs** every demo exposes; the third is the **experiment plan** to convergence. The per-demo subsections then give only what differs, with the measured results: `demo_cpu_tasks` (`DATASET=mbpp`, `DATASET=math`, the other datasets together, and the mixes), `demo_gpu_single`, the two Ray-farm demos (`demo_gpu_farm_fractional` and `demo_gpu_farm_whole`, which share one topology and one diagram), the two resilience mechanisms (`HEARTBEAT=1` and `demo_gpu_single SNAPSHOT_EVERY=1`), and `demo_cpu_scale_lanes`. `demo_cpu_terminalbench`, `demo_cpu_instance_pool` and `demo_cpu_branching` are covered fully in their storyline chapters above.

A note on grounding: the numbers below are read from this project's own committed run artifacts (`train.csv`, the ablation CSVs under `demo_cpu_tasks/logs/`, and the `[sbx-op]`/`[sglang-gen]` records in the smoke and GPU logs). They come from

short runs (≈10 training steps, or smoke runs), so treat them as *shape and order-of-magnitude*, not as converged science. Where a metric is only *instrumented* — emitted by the code but not yet populated in a committed run — this is said explicitly rather than filled with an invented value.

7.1 The shared skeleton — one pipeline, one set of metrics

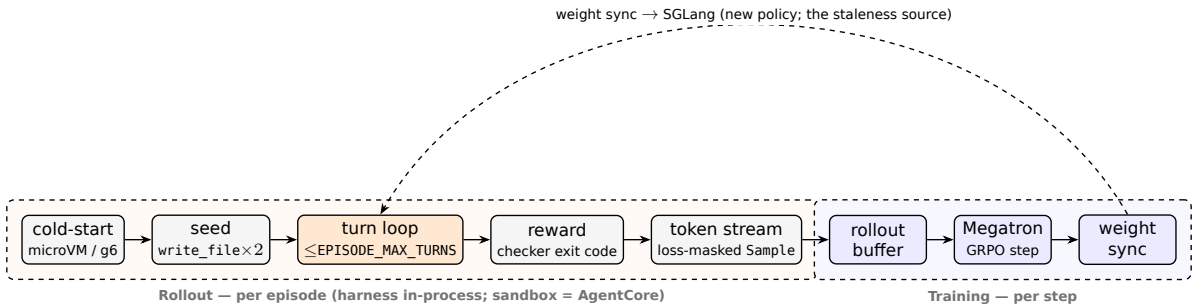


Figure 8: The demo pipeline, shared by all four training demos. An episode cold-starts its sandbox session, seeds the workspace, then runs the **turn loop** — each turn a generation through the proxy (`[sglang-gen]`) followed by a checker exec in the sandbox (`[sbx-op]`). The checker's exit code is the reward; `build_token_stream` emits a loss-masked `Sample` into the rollout buffer; Megatron takes a GRPO step and **weight-syncs** the new policy back to SGLang — the point at which rollouts generated on an older policy version become *stale*. The two labeled latencies are the two things the scaling decision (§"Scaling the loop") is made from.

Architecture. Every training demo is the topology of §"The call path": slime's `generate()` hook runs the agent loop in-process, each **model turn** goes through the proxy (which returns the exact sampled tokens + logprobs — the TITO property), and each **tool call** becomes one `invoke_agent_runtime` to an AgentCore sandbox where the checker produces the reward. The four training demos differ only in the sandbox ARN (CPU microVM vs GPU instance) and the task.

The four configs — trainer, inference, harness, sandbox. The training demos fix three of the four components and vary only the sandbox:

Component	Where it runs	Config (shared across the training demos)
Trainer	4 GPUs on the 8×L40S box	Megatron-LM, GRPO; `global_batch_size=32`; async checkpoint save; `ACTOR_GPUS=4`
Inference	4 GPUs on the same box	SGLang, 2 engines (2 GPUs each), `sglang-mem-fraction-static=0.7`, one **cache-aware router**; `ROLLOUT_GPUS=4`
Harness	in-process (CPU) next to the trainer	`run_episode`; fully-async `MODE=B` (deep buffer); `CONC=4` rollout concurrency; `INFER_CONC=0`; `EPISODE_MAX_TURNS=3`
Sandbox	**the only variable** — remote AgentCore	CPU microVM (arm64) or GPU `g6` instance (amd64); one `runtimeSessionId` per episode; the `SANDBOX` env var selects it

Each per-demo subsection states that demo's **sandbox** row; the trainer, inference, and harness rows are identical unless noted. This is the disaggregation of §"Agentic RL and its reference architecture" made concrete: GPUs go to training and inference, the harness rides cheap CPU in-process, and the sandbox is the isolated remote seam.

Step by step (one episode). Cold-start the session's microVM (or launch the g6) → seed `PROBLEM.md` + `run_checks.py` → the turn loop (≤ `EPISODE_MAX_TURNS`): generate via the proxy (`[sglang-gen]`), write the artifact, exec the checker (`[sbx-op]`), feed any failure back → reward from the checker's exit code → `build_token_stream` → a loss-masked `Sample` → the rollout buffer → a Megatron GRPO step → weight sync back to SGLang.

Lifecycle. One session = one episode: the first invoke cold-starts the microVM (or launches the g6), the same `runtimeSessionId` re-routes to the warm VM, and the session is reclaimed on idle (~15 min CPU / 600 s GPU) or on max lifetime (8 h microVM / up to 14 d instance). Under asynchronous training, the pause during weight sync can trip the idle clock — which is what the resilience demos address.

Scaling method. Following §“Scaling the loop”: read `perf/sandbox_ms_*` against `perf/sglang_gen_ms_*`, then grow the dominant side — inference (more SGLang replicas behind the **cache-aware** router) when generation dominates, or the sandbox (more microVM sessions, or a pool of Runtime ARNs round-robined **sticky per episode**; on GPU, the Ray farm) when the checker dominates. The demos are chosen so both cases actually occur: MBPP/math are generation-bound, KernelBench is sandbox-bound.

What is measured — and what is only instrumented. The metric glossary the deep-dives use:

Metric	Meaning	Status in these demos
`rollout/raw_reward`, `eval/<task>`	mean binary checker reward per step; held-out solve rate	**measured** (per step + eval)
`perf/actor_train_tok_per_s`, `perf/actor_train_tflops`	trainer throughput and achieved FLOPs	**measured**
`perf/mfu`	model FLOPs utilization = TFLOPS ÷ 362 (L40S peak)	**measured** (ablation CSVs)
`perf/wait_time_ratio`	fraction of the step the trainer waits on rollout — the pipeline-balance / GPU-idle proxy	**measured**
`perf/goodput_tok_per_s`	throughput × (1 − discard ratio)	**measured**, but = throughput (discard = 0)
`train/pg_clipfrac`	PPO/GRPO clip fraction	**measured**, uniformly 0.0
`[sglang-gen]`, `[sbx-op]` p50/p95	**per-turn inference latency** (generation) and per-op sandbox latency	**measured** in the run logs

Metric	Meaning	Status in these demos
`perf/step_time`	per-step wall-clock — **batch training** + rollout	**measured** (≈ 23–78 s/step, `demo_cpu_tasks` `DATASET=mbpp`; 110–804 s/step on `demo_gpu_single`, where the remote-g6 checker dominates)
weight-sync latency	broadcasting new weights trainer → SGLang each step	**not separately recorded** here (folded into `step_time`); ≈ 20–30 s for a 1T model at frontier scale [29]
staleness (`v_use` − `v_gen`), discard/staleness ratio, importance-sampling ratio (ρ, TIS), `num_turns`, buffer depth, samples/s, direct GPU utilization	off-policy and horizon signals	**instrumented or configured only** — no value recorded yet

The last row matters: the code emits `[sbx-op]`/`[sglang-gen]` and the derived columns exist, but staleness, discard ratio, and the importance-sampling ratio are described in `docs/observability.md` as slime keys we would read — they are not populated in any committed run here, so no number is claimed for them below. Trainer-vs-inference GPU utilization is likewise not directly sampled; the split is *configured* (4 trainer + 4 inference GPUs), and `perf/wait_time_ratio` is the recorded proxy.

Two tiers: the general set, plus one per-demo signal. Every demo records the *general* set above (it is the same `metrics.py` and the same in-process harness across every demo, so the pipeline-shape numbers are comparable demo-to-demo). On top of that, each demo emits exactly the signal that makes *its own point* legible, through one unified channel: a `[demo] k=v k=v ⋯` line that `metrics.py` parses generically (numeric keys → count/mean/p50/p95/max; categorical keys → value-counts; plus mean-reward grouped by each categorical

key), writing a <log>.demo.csv alongside the train/rollout CSVs. For the ~18 run_episode training demos the harness emits one [demo] line per episode carrying task/family/reward/solved/turns/boot_attempts/status/checker (the checker stdout compressed to a space-free class token — PASSED, IMPORT_ERROR, FAIL_<ExcType>, ...), so a mixed-task run yields reward-by-family and a failure-mode histogram for free. The bespoke demos add their defining metric on the same channel: demo_cpu_branching logs per-task fan_out; HEARTBEAT=1 logs one line per heartbeat fired in any demo (>0 with keep-warm on, 0 in the A/B baseline — the whole point of the knob); demo_cpu_scale_lanes logs the driving worker, pull-qdepth, the chosen engine and its engine_inflight, and the borrowed sandbox_session/sandbox_slot with its sandbox_wait_ms; demo_gpu_single with SNAPSHOT_EVERY=1 logs files restored/saved at each episode boundary (the rework a recycle would otherwise cost); and the GPU-farm demos log the per-episode mem_fraction/waited_s/cuda_visible of each fractional sub-session.

Two corrections to the general set land the same way for every demo. Sandbox latency is split into **warm data ops** (perf/sandbox_ms_*) versus **session-open ops** (perf/sandbox_open_ms_*), because on an Instance substrate a session open can be a 60–80 s cold boot while a warm reopen is ~150 ms — averaging them together would hide exactly the warm-reuse win this project is about. And a **cold boot** is counted only when a session-open op actually exceeds COLD_BOOT_MS (5 s): perf/sandbox_coldboot_count. On the demo-22 warm-farm log that split reads correctly as 8 real cold boots against 416 warm reopens — the one-shared-runtimeSessionId reuse the farm exists to demonstrate.

7.2 Sizing the config — response length per turn, and turns per episode

Every demo exposes two knobs that follow directly from the episode shapes of §“Tasks frontier labs run in a sandbox”. Getting them right is what makes a run efficient rather than wasteful or broken.

Max response length per turn (--rollout-max-response-len). This is the per-turn generation budget. Set it from the **p95 of the policy’s own per-turn response length** for your family (from §3: math reasoning traces run 1k–8k and up to 32k tokens; a raw code submission is 100–800 but a reasoning model prepends thousands; a CUDA kernel is 200–1,000; a tool call is 50–300). Size to that p95 with headroom. **Too small** truncates the reasoning trace mid-thought: the

checker then scores an incomplete artifact (depressing reward), and the truncated sequence is still trained on (a biased gradient). **Too large** wastes KV cache and slows decode, because SGLang reserves capacity per sequence. In these demos `enable_thinking=False` keeps Qwen3-4B's per-turn responses short, so a modest budget suffices; a reasoning policy on hard math would need a far larger one — measure before committing.

Max turns per episode (`EPISODE_MAX_TURNS`, default 3). This caps how many generate→exec rounds an episode may take. It is the single biggest lever on episode cost: each extra turn adds one full generation *and* one sandbox round-trip, and it grows the context every subsequent turn must attend over — so per-episode cost ≈ `turns × (generation + exec + observation)`, and the trainer's ref/actor forward passes scale with the resulting sequence length. More turns raise the *ceiling* on reward (more chances to read a failure and fix it) but lengthen episodes, which under asynchronous training means **more staleness** (a long episode spans more weight-sync barriers) and more sandbox operations. Set it from the family's horizon: single-turn families (math, instruction following) → 1; iterative code/kernel/SQL → 3–5; long-horizon SWE / web / multi-agent → 15–50+, at which point a **warm, long-lived session** (§"HEARTBEAT=1") becomes necessary. It is controlled per run as an env var in `run.sh`, threaded into the harness as `max_turns` — no redeploy needed. These demos use `EPISODE_MAX_TURNS=3` and rollout concurrency `CONC=4`.

7.3 Experiment plan — training each demo to convergence

A demo is "done" only when it shows **learning to convergence**, not a smoke run (`docs/demos-program.md`). What is committed today is deliberately short: `demo_cpu_tasks` (`DATASET=mbpp`) and `demo_gpu_single` have convergence-shaped ~10-step runs with committed eval curves (enough to read the bottleneck, not to converge), and the 2026-09-18→19 smoke-plan batch (`code/demos/_smoke_logs/`) has since re-run the whole CPU set — the other nine `DATASET=` variants of `demo_cpu_tasks` and the scale/resilience/variant CPU trainers — at `NR=11` (10 train steps + 1 held-out eval + one final checkpoint at iter 10), a loop-verified smoke with a baseline held-out solve rate, not a learning curve; and the 2026-09-21 GPU sweep ran **every GPU configuration** (all three `demo_gpu_single` angles and both farm modes, today `demo_gpu_farm_fractional` and `demo_gpu_farm_whole`) at the same `NR=11` budget on the reserved L40S (`Job succeeded` + checkpoint iter 10 each), so every training demo now has at least a loop-verified smoke — none

has yet been run to convergence. The plan to convergence has the same shape for every training demo and differs only in the step budget and which side gets scaled once the bottleneck is measured.

The convergence bar (all training demos). (1) Train enough steps that the **held-out eval solve rate rises then plateaus** — log `eval/<task>` every `EVAL_INTERVAL` steps, not the default ~10-step run. (2) `rollout/raw_reward` and `eval/<task>` trend up then flat on the folder's `train.png`. (3) `train/grad_norm` and `train/kl_loss` stay sane (the policy is moving, not diverging). (4) Only then apply the §"Scaling the loop" decision: scale the *measured* bottleneck and show the `perf/step_time` improvement — confirming the other side would not have helped.

Because the metric that decides scaling (`perf/sandbox_ms_*` vs `perf/sglang_gen_ms_*`) is measured *per demo*, the experiment plan is per demo too — the bottleneck column below is what each demo's own latencies (in its subsection) already show:

Demo	Eval metric	Measured bottleneck	Scale	Convergence note
`demo_cpu_tasks · mbpp`	`mbpp`	generation (sandbox 0.12 s ≪ gen 1.76 s)	inference — SGLang replicas	run past 10 steps until `eval/mbpp` plateaus
`demo_cpu_tasks · math`	`math`	generation (trivial checker, long traces)	inference	long-reasoning → largest `--rollout-max-response-len`
`demo_gpu_single`	`kernelbench`	remote g6: sandbox (exec 8.5 s ≫ gen 2.5 s); local farm: gen ≈ sandbox	sandbox — more g6 or the farm	budget the ~90 s torch + g6 cold start

Demo	Eval metric	Measured bottleneck	Scale	Convergence note
demo_gpu_farm_fractional	kernelbench	sandbox, packed (warm farm → gen-bound locally)	Ray fractional actors (0.5+0.5) per GPU	smoke passes end-to-end; now run to convergence
demo_gpu_farm_whole	kernelbench	sandbox, one whole GPU/episode	more whole-GPU Ray actors	smoke passes end-to-end; now run to convergence

That table says *what* to scale; the protocol below says *how far to run* each demo to earn the right to scale — a concrete step budget (NR), eval cadence (EVAL_INTERVAL), a target held-out plateau, and a wall-clock estimate. All share the committed config (global_batch_size=32, CONC=4, EPISODE_MAX_TURNS=3):

Demo	Steps (NR)	Eval every	Held-out start → target plateau	Est. wall-clock*
demo_cpu_tasks·mbpp	150–200	10	eval/mbpp 0.56 → ≥ 0.80	~1–4 h
demo_cpu_tasks·math	150–200 /split	10	eval/gsm8k 0.90 → flat; MATH-500 / DeepScaleR climb from cold	~2–6 h /split
demo_gpu_single	100–150	10	eval/kernelbench 0.53 → ≥ 0.80	~4–24 h
demo_gpu_farm_fractional	100–150	10	eval/kernelbench 0.65–0.78 → ≥ 0.85, two 0.5-GPU actors on **one** GPU	≈ single-GPU/step, ÷ packing
demo_gpu_farm_whole	100–150	10	eval/kernelbench 0.65–0.70 → ≥ 0.85, one whole GPU/episode	≈ single-GPU/step

* *Wall-clock is each demo's measured `perf/step_time` (§ metric glossary) times the step budget — a planning extrapolation, not a measured convergence time. The step budgets (chosen so MBPP's 500-row / KernelBench's 100-row splits see ~10 epochs at GBS=32) and the target plateaus are likewise **targets**: no demo has yet been run to plateau — the committed runs are the ~10-step convergence-shaped `demo_cpu_tasks` (`DATASET=mbpp`) / `demo_gpu_single`, the 10-step (`NR=11`) smoke-plan batch across the CPU set (the other `DATASET=` variants and the scale/variant trainers), and the 2026-09-21 `NR=11` GPU sweep across every GPU configuration. The GPU range is wide because `perf/step_time` spans ~110–800 s: on the remote g6, cold start (~90 s torch install + ~123 s first ping) and per-episode checker variance dominate; on the local L40S farm the sweep's `perf/step_time` ran a tighter ~130–440 s (warm actors, no g6 launch), rising within each run as later steps train on longer accumulated sequences.*

The **eight other real-dataset `DATASET=` values** (§"`demo_cpu_tasks` per dataset in depth") inherit the `DATASET=mbpp` plan unchanged — they are generation-bound (cheap checker, `math`/`coding`/`sql`/`rag`/`tool use`), so each converges by running past the smoke budget while tracking its own `eval/<task>` every 10 steps and, if generation dominates, adding SGLang replicas. Concretely: `NR ≈ 100–200` (scaled to each 140–400-row split, so ~10 epochs at `GBS=32`), `EVAL_INTERVAL=10`, ~1–3 h each on CPU — the same 23–78 s/step envelope as `DATASET=mbpp`. Running these eight to convergence is `demo_cpu_tasks`' open work item.

7.4 `demo_cpu_tasks` in depth — `DATASET=mbpp` (MBPP, the default)

- **Task / harness:** `DATASET=mbpp` (the default) — MBPP multi-turn Python repair; task-agnostic `run_episode`, in-process.
- **Dataset:** MBPP 500 train / 8 eval (`build_dataset.py`), plus family seed sets in `data/`.
- **Runtime:** this committed run used the own CPU Runtime (`sandbox_server_agent.py`, arm64, `deploy/Dockerfile.sandbox.cpu`, `SANDBOX=agentcore-runtime`); the demo's default is now the managed Code Interpreter (`SANDBOX=agentcore`, Chapter 2), with the same task, checker and reward. All 8 L40S serve the policy (4 train + 4 inference); 0 env GPUs.
- **Reward:** run the candidate against hidden MBPP asserts; 1.0 iff exit code 0.
- **Sandbox scaling:** round-robin a fleet of sandbox Runtime ARNs (`AGENT-CORE_SANDBOX_RUNTIME_ARNS`, `deploy_cpu_sandbox_fleet.py`, `run_runtime_ablation.s`

sticky per episode.

Measured results. Over the committed 10-step run (`train.csv`), `rollout/raw_reward` moved in the ~0.5–0.94 band (0.875 at step 1, dipping to 0.5 at step 9 — it never reaches 1.0) and held-out `eval/mbpp = 0.5625` at step 9. Trainer throughput `perf/actor_train_tok_per_s` ≈ 1.5k–3.3k tok/s, `perf/actor_train_tflops` ≈ 8.9–20.4, `perf/step_time` ≈ 23–78 s, `perf/wait_time_ratio` ≈ 0.39–0.84, and `train/pg_clipfrac = 0.0` at every step. The runtime-count ablation (`logs/ablation_summary.csv`, sweeping 1/2/4 sandbox Runtimes) adds `mean_mfu` ≈ 0.018–0.043 (2–4 % of the L40S 362-TFLOPS peak), `mean_goodput_tok_per_s` ≈ 1.1k–2.6k (equal to throughput — discard ratio 0), and a wider `wait_time_ratio` spread. Real latencies (from `smoke_realaws.log`): **sandbox** ops p50 123 ms / p95 900 ms (`write_file` p50 94 ms, exec p50 123 ms, `ping` p50 546 ms); **SGLang** decode p50 1.76 s / p95 4.1 s. Reading them: generation (1.76 s) ≫ sandbox (0.12 s), so this family is **generation-bound** — scale inference, exactly as §3 predicts for a cheap-checker code task. **Plan to convergence:** run `NR` ≈ 150–200 steps (MBPP is 500 rows ≈ 16 steps/epoch at GBS=32, so ~10 epochs) with `EVAL_INTERVAL=10`, until `eval/mbpp` climbs from its 0.56 start to a ≥ 0.80 plateau — an estimated ~1–4 h at the measured 23–78 s/step; then add SGLang replicas (the generation-bound side) and confirm `perf/step_time` drops while `eval/mbpp` holds.

7.5 `demo_cpu_tasks` in depth — `DATASET=math` (math word problems)

- **Task / harness:** `DATASET=math` (`FAMILY=math`); write `solution.py`, numeric checker; same in-process loop.
- **Dataset:** real HF slices via `real_datasets.py` — GSM8K 200/40, MATH-500 160/40, DeepScaleR 400/40.
- **Runtime / reward / scaling:** identical to `DATASET=mbpp` (CPU microVM; 1.0/0.0 on the checker's exit code; scale inference).

Measured results — smoke only. The committed artifacts are hardware *smoke* runs, not convergence runs. The 2026-09-18→19 smoke-plan batch ran each math family for the full 10-step budget (`NR=11`) with a single held-out eval: `DATASET=gsm8k` `eval/gsm8k = 0.90`, `DATASET=math500` `eval/math500 = 0.89`, `DATASET=deepscaler` `eval/deepscaler = 0.78` (and `DATASET=math` itself `eval/math = 1.00`), each with `rollout/raw_reward` moving across ten training rollouts. Both carry real `[sbx-op]`/`[sglang-gen]` records. Treat these as a *sanity*

signal that the loop and the numeric checker work, not as a learning curve — the README states the convergence bar is deliberately not met by a smoke run. The family is nonetheless clearly **inference-bound**: the reasoning traces dominate wall-clock while the numeric checker is milliseconds, so the `[sglang-gen]` records — not the `[sbx-op]` records — set the step time, and the scaling answer is more SGLang capacity behind the cache-aware router. **Plan to convergence:** run `NR ≈ 150–200` per split with `EVAL_INTERVAL=10`, tracking `eval/math` on GSM8K / MATH-500 / DeepScaleR (`DATASET=gsm8k/math500/deepscaler` package exactly these) — `eval/gsm8k` starts near its 0.90 smoke value and should flatten quickly, while MATH-500 and DeepScaleR climb from cold; size `--rollout-max-response-len` to the reasoning-trace p95 and scale SGLang, since the bottleneck is generation. Estimate ~2–6 h/split — longer than `DATASET=mbpp` because the reasoning traces are longer.

7.6 `demo_gpu_single` in depth — KernelBench on a GPU Runtime instance

- **Task / harness:** write a CUDA `ModelNew`; the harness is in-process, only the checker is remote.
- **Dataset:** KernelBench 100 level-1 train / 20 level-2 eval (`build_dataset.py --dataset kernelbench`).
- **Runtime:** one GPU AgentCore Runtime on the `acs_gpu_cp` capacity provider (a real `g6.xlarge` / L4, amd64, `deploy/Dockerfile.sandbox.gpu`, ECR `acs-sandbox-gpu`); `SANDBOX=agentcore-runtime`. **torch 2.5.1 is pip-installed at container start (~90 s cold)** to dodge the 2048 MB image cap. The trainer box's 8 GPUs all go to slime (4 train + 4 inference); the checker's GPU is the remote g6.
- **Reward:** rebuild the reference on CUDA with matched seeds, assert `allclose`, report speedup; failure text feeds the next turn.
- **Sandbox scaling:** more g6 instances (isolated) or the Ray farm of `demo_gpu_farm_fractional` / `demo_gpu_farm_whole` (packed).

Measured results. Over the committed 10-step run, `rollout/raw_reward` moved in the ~0.28–0.94 band (0.81 at step 1) and `eval/kernelbench = 0.65` (CSV) / 0.725 (log). Trainer throughput `perf/actor_train_tok_per_s ≈ 2.9k–4.0k` tok/s, `perf/actor_train_tflops ≈ 19–26`, `perf/step_time ≈ 110–804 s`. `perf/wait_time_ratio ≈ 0.79–0.96` — high, meaning the trainer spends most of

the step waiting on rollout + the remote GPU checker. Real latencies (from the GPU run log): **sandbox** exec p50 8.5 s / p95 17.3 s / max 24 s; the first `ping` p50 ≈ 123 s (the g6 EC2 cold launch, amortized once per session); `write_file` p50 170 ms. **SGLang** decode p50 2.5 s / p95 6.4 s. Reading them: exec (8.5 s) ≫ generation (2.5 s), so KernelBench is **sandbox-bound** — scale the sandbox (more instances, or the Ray farm), exactly as §3 predicts. The high `wait_time_ratio` is the same story from the trainer's side. **Plan to convergence:** run `NR` ≈ 100–150 steps (KernelBench is 100 train rows) with `EVAL_INTERVAL=10`, until `eval/kernelbench` plateaus above its ~0.65–0.73 start toward ≥ 0.80; budget the one-time ~90 s torch install + ~123 s g6 cold launch per session, and expect ~4–24 h given the 110–804 s/step range; because it is sandbox-bound, scale the sandbox (more g6 instances or the Ray farm) rather than inference. The end-to-end path was then re-verified at the **full 10-step budget** on the local L40S farm (2026-09-21 GPU sweep, `NR`=11): `Job succeeded`, final checkpoint iter 10, `eval/kernelbench` **0.525**, `raw_reward` 0.75→0.5 across the 10 noisy steps, `demo/solved_mean` 0.52 on the last step. On this local run the latencies *invert* the remote-g6 story: the warm farm makes exec cheap, so `perf/sandbox_ms_mean` (~2.0–3.0 s) is now **comparable to** `perf/sglang_gen_ms_mean` (~3.0–3.6 s) rather than dwarfing it — KernelBench is only sandbox-bound when the checker pays a cold remote instance; warm-shared, generation is the co-equal cost. This demo's eval is the lowest of the GPU set (0.525) and its eval responses are the longest (mean 924 tokens, max 3072), which is the same fact from two sides — longer generations, more truncation pressure, a harder time landing a correct kernel in the turn budget. Note it keeps `--async-save`: unlike the farm demos, its persistent-async-ckpt worker detects the unfinalized save and *degrades gracefully to a synchronous finalize* rather than crashing on the `pidfd_getfd` seccomp block, so it needs no explicit sync-save change.

7.7 `demo_gpu_farm_fractional` and `demo_gpu_farm_whole` in depth — packing episodes onto GPUs

- **What differs from `demo_gpu_single`:** one line — `SANDBOX=agentcore-ray-farm`. The sandbox image bakes Ray (`deploy/Dockerfile.sandbox.gpu.shared`); the smokes serve it on the box's own GPUs via `run_local_gpu_runtime.sh fractional|whole`, and it deploys remotely as `acs_sandbox_gpu_b4`. Farm knobs (`B4_GPU_FRACTION`, `B4_MAX_SANDBOXES`, `B4_PER_SANDBOX_MEM_FRACTION`)

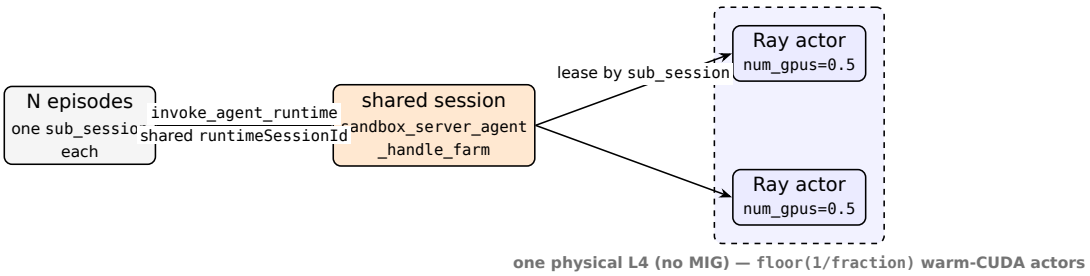


Figure 9: The Ray GPU farm of `demo_gpu_farm_fractional`. All concurrent episodes invoke *one* AgentCore GPU Runtime under a single shared `runtimeSessionId`; inside it, `_handle_farm` leases one fractional-GPU Ray actor per episode (keyed by a per-episode `sub_session`). With `num_gpus=0.5`, two warm-CUDA actors share the one GPU; at `fraction=0.25`, four. Because L4/L40S offer no MIG, isolation is software-only (per-process memory caps + an `nvcc flock`, since a non-MIG GPU wedges under sustained compile storms). `demo_gpu_farm_whole` sets `fraction=1.0` across two GPUs, so each actor owns a whole GPU instead of a slice.

are runtime env on the g6, retunable without a rebuild; `INVOKE_MAX_RETRIES` is raised for the minutes-long cold launch.

- **`demo_gpu_farm_fractional` vs `demo_gpu_farm_whole`:** the *only* difference is two farm knobs (two folders since 2026-09-25). `fractional` runs `B4_GPU_FRACTION=0.5` with `B4_FARM_GPUS=1` → `floor(1/0.5)=2` actors packed onto **one** physical GPU (`demo/mem_fraction` p50 0.5); `whole` runs `B4_GPU_FRACTION=1.0` with `B4_FARM_GPUS=2` → two actors each owning a **whole** GPU (`demo/mem_fraction` p50 0.95). Same image, same code, same contract — the pair shows both points of the packing spectrum on the same farm.
- **Task / dataset / reward:** identical to `demo_gpu_single` (KernelBench, `allclose`/speedup).

Results — measured, full 10-step smokes (2026-09-21, plus a 2026-09-25 fractional re-run). Both farm configurations pass the **full `NR=11` budget** on the reserved L40S (the `Dockerfile.sandbox.gpu.shared` image served locally over the identical `/invocations` contract; slime on the remaining GPUs), each reaching `Job succeeded` with a valid final checkpoint iter 10:

- **`demo_gpu_farm_fractional`** (run then as `demo_gpu_shared_single`; `fraction=0.5` → two 0.5-GPU actors on **one** GPU): `eval/kernelbench` **0.775**

— the **highest of the GPU set** on this sweep — with demo/solved_mean 0.708 on the last step, raw_reward 0.625→0.375 across the 10 noisy steps, demo/mem_fraction p50 0.5 (the fractional cap doing its job) and demo/cuda_visible 0 (no whole device is pinned, exactly as fractional sharing intends). perf/sandbox_ms_mean ~2.2–3.1 s, perf/sglang_gen_ms_mean ~3.0 s. The 2026-09-25 re-run in its own folder scored eval/kernelbench 0.65 over 359 episodes (Chapter 7).

- **demo_gpu_farm_whole** (run then as demo_gpu_shared_multi; fraction=1.0 → two whole-GPU actors on **two** GPUs): eval/kernelbench **0.65** (a second run of the same config, as demo_gpu_local, scored 0.70), demo/solved_mean 0.625, raw_reward 0.75→0.5, demo/mem_fraction p50 0.95 (whole-GPU actor).

Both ran with **balanced sandbox open==close counts and demo/boot_attempts p50 = 1** — the client-side **open-concurrency gate** caps opens at the farm's slot count (2), so overflow episodes queue in-process instead of hammering the server gate into a ~15-min invoke timeout. This is the fix that closed the old wedge (a lease leak in a stale vendored sandbox copy where op=close never fired in local-GPU mode, starving the 2-slot farm after two episodes — see §7's sweep note). Getting the shared farm clean also required **synchronous checkpoint save**: this container's seccomp blocks pidfd_getfd, which slime's persistent async-ckpt child needs for the CUDA storage fds, so --async-save hangs the job even at PP=1 (the save writes a 53 GB torch_dist checkpoint that takes ~7 min to fsync — the log *looks* frozen but is flushing).

Reading the two results together. fractional (0.5+0.5, one GPU) *out-scored* whole (two GPUs) on this short sweep — 0.775 vs 0.65 — which is noise over 10 steps, not a real quality gap, but it does make the central point concrete: **packing two episodes onto one physical GPU costs nothing measurable in reward** here, while freeing a whole GPU back to the fleet. The fractional *packing* itself was already validated on hardware (a non-MIG L4/L40S wedges under sustained nvcc load, which fraction=0.5 plus the nvcc flock mitigate). The demo's **primary motivation is capacity, not just cost**: the isolated-per-session path needs *N whole g6 instances at once*, which stalls on a capacity-starved fleet; fractional packs N episodes onto **one** GPU with software-only isolation, and whole is the intermediate when whole GPUs are available but you still want one shared session. **Plan to convergence:** run the same NR ≈ 100–150 / EVAL_INTERVAL=10 protocol as demo_gpu_single and compare goodput-per-GPU: fractional should match

`demo_gpu_single`'s per-episode reward at ~half the GPUs (the packing win), `whole` at parity per GPU.

7.8 `HEARTBEAT=1` and `demo_gpu_single SNAPSHOT_EVERY=1` in depth — surviving the pause and the recycle

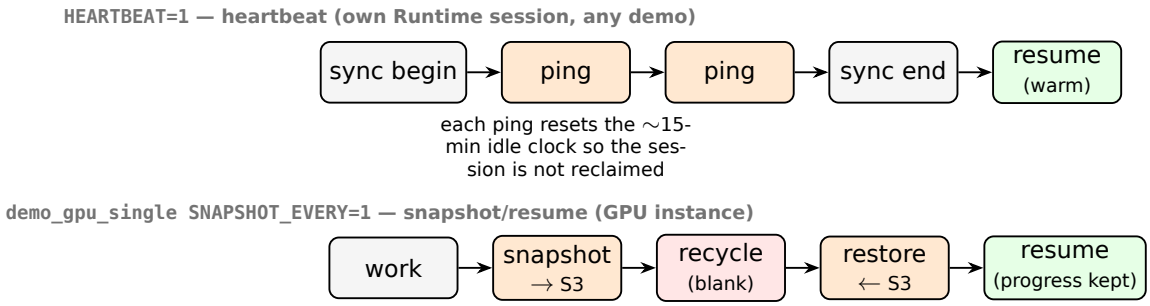


Figure 10: The two resilience levers. **Heartbeat** (top) keeps a *live* session warm across the seconds-long weight-sync pause by resetting the idle clock — no state is lost. **Application-level snapshot** (bottom) restores a *lost* session after a genuine instance recycle by serializing the workspace to S3 and restoring it onto a fresh session — because AgentCore offers no VM/EBS snapshot. Heartbeat prevents loss; snapshot recovers from it.

Neither offline probe trains a policy; both run in seconds and demonstrate a mechanism the training demos compose. The heartbeat has since been measured live (Chapter 4): on an own Runtime it kept 3/3 idle sessions alive with files intact, against 0/3 without it; on the Code Interpreter it has no effect, because a session ends at its start-time timeout.

- **`HEARTBEAT=1`** (probe `demo_cpu_tasks/heartbeat_demo.py`; the live measurement is the `warmstart/` probe of Chapter 4) — the reusable `SessionKeeper` (a process-wide registry + one background heartbeat loop driven by the trainer's `sync_begin()`/`sync_end()` barrier) and `KeptSandbox` (wraps any `Sandbox`), in `session_keeper.py`. The three-act arc: ACT 1 naive "stop invoking" → the session is idle-reclaimed; ACT 2 the heartbeat keeps it warm across an arbitrarily long pause while the gate holds new invokes during sync; ACT 3 recover-by-reseed if a session dies anyway. The heartbeat lives **outside** the harness by design, so no harness grows its own keepalive; with `HEARTBEAT=1` each demo's `generate` wraps its sandbox in `KeptSandbox` (`maybe_keep_warm`), and `demo_cpu_scale_lanes` also registers its pooled persistent sessions.

- **demo_gpu_single SNAPSHOT_EVERY=1** — snapshot_workspace() / restore_workspace() in snapshot_store.py, backend-agnostic (LocalSnapshotStore offline, S3SnapshotStore for production — no mount, AZ-agnostic). The A/B arc: RUN A (no snapshot) redoes from step 0 after a recycle; RUN B snapshots before the recycle and restores onto a fresh session, printing the rework saved (e.g. "6 vs 4 steps → 2 saved"). This is explicitly *not* a VM/EBS snapshot — see docs/gpu-snapshot.md for why an application-level snapshot is the only feasible option on AgentCore. What is committed is the **offline simulation** (the A/B arc above, ~1 s, LocalSnapshotStore). The demo also documents a **REAL_SNAPSHOT=1 mode** that would run the same arc against a live local Runtime container — closing session #1, opening a fresh blank session in a new workdir, and restoring the workspace onto it over real /invocations calls — but this real-recycle path is **a recipe only, not yet verified on hardware** (both run.sh and the README say so, and the snapshots/ directory holds no committed artifact). So the mechanism is demonstrated in simulation; the on-hardware recycle is honest open work.

7.9 demo_cpu_scale_lanes in depth — 4 SGLang engines : 256 harness workers : 256 microVMs

This demo makes the §"Scaling the loop" law operational. It holds the task and sandbox image fixed and scales the rollout as **three decoupled pools** — E SGLang engines, W warm in-process harness workers, M persistent microVM sessions — to show that rollout throughput is min(inference, sandbox, harness) and that the driver is the third dial. The default 4 : 256 : 256 is balanced for a 256-episode step; changing one pool alone moves the ceiling to it. Its topology differs from the training demos (three pools joined per episode), so it carries its own diagram.

- **Task / harness:** DeepScaleR math in coding-shape (write solution.py setting a module-level answer; a numeric-assert checker scores it, RLVR, no LLM judge). The real integration drives the MiniClaudeHarness (write_file/bash/read_file over the Anthropic Messages API to slime's in-process adapter); the offline sweep drives the family-agnostic run_episode.
- **Dataset:** DeepScaleR 400 train / 40 eval (data/deepscaler_{train,eval}.jsonl), numeric-answer subset.
- **Two artifacts:** an **offline** scaling-law sweep (harness_scaling_demo.py: real run_episode on a LocalSandbox, inference and sandbox modeled as a

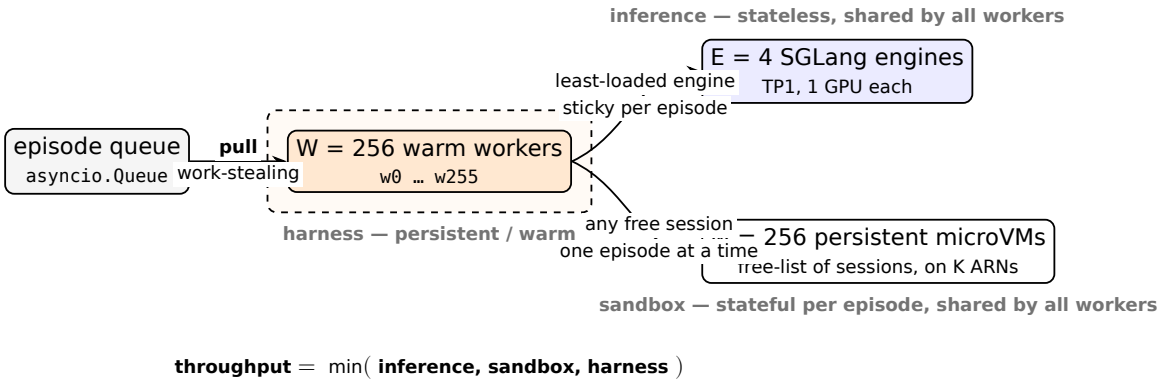


Figure 11: The decoupled tiers of `demo_cpu_scale_lanes` (default 4 : 256 : 256). **Harness**: `W = HARNESS_WORKERS` warm workers pull episodes off a shared queue — work-stealing balances by *availability*, robust to the wide, variable latency of multi-turn agentic episodes. **Sandbox**: the worker borrows any free session from a free-list of `M = SANDBOX_SESSIONS` persistent microVMs (slot `s` on ARN `s mod K`), works in a fresh `ep-<hex>` workdir, and returns the session after deleting it. **Inference**: the episode then takes the SGLang engine with the fewest in-flight episodes (ties rotate), and all its turns stay there. The three sizes are independent; raising one alone moves the ceiling to the next-scarcest tier.

concurrency ceiling + service time, `W` swept — no AWS/GPU, seconds) and the **real slime integration** (`harness_pool.py` behind `--custom-generate-function-path agentcore_slime_demo.harness_pool_generate.generate`, with `HARNESS_WORKERS` / `SANDBOX_SESSIONS` / `ROLLOUT_GPUS` as the three independent knobs; `SANDBOX_SESSIONS=0` = fresh session per episode). Every episode logs `[demo] worker=⋯ engine=⋯ engine_inflight=⋯ sandbox_session=⋯ sandbox_slot=⋯ session_episode=⋯ sandbox_wait_ms=⋯ qdepth=⋯`, `router.csv` gains a `sandbox_slot` column, and `metrics.py`'s `check_stickiness` prints the decoupling report (episodes per engine; sessions per worker, engines per worker, workers per session) and fails only if a session spanned two Runtimes; requires `--sglang-tool-call-parser qwen25 --sglang-reasoning-parser qwen3`, the tool-parser fix without which the mini-harness tool loop scores a structural zero).

Measured results — offline sweep, plus a loop-verified slime smoke. The scaling-law numbers below are from the offline model (`harness_scaling_demo.py`) on the dev box — measured wall-clock throughput, with the per-tier ceilings computed from the service times observed during the run. With the inference ceiling

set to CINF=4 in-flight @ ~100 ms (≈ 40 ep/s) and the sandbox to CSBX=8 @ ~50 ms + the real checker subprocess (≈ 80 ep/s), throughput rose ~linearly with W while the **harness** bound — 5.3, 10.5, 20.9, 29.6, 36.5 ep/s at W = 1, 2, 4, 6, 8 — then **plateaued at ~37 ep/s** once **inference** bound (raising W to 12 bought little more: 36.6 ep/s, and the committed sweep marks W=8 and 12 as inference-bound). The min() law is visible directly: harness-bound at small W, then the ceiling relocates to inference, the next-scarcest tier — to push past ~40 ep/s you would have to lift CINF too, exactly the joint-scaling point of §2. Every episode solved (48/48), so the checker path is real, not mocked away. The sweep is an **unseeded offline simulation**: absolute ep/s are machine-dependent and vary run to run — treat them as an *illustration of the min() law*, not a slime rollout-throughput measurement. Separately, the real slime integration was **loop-verified on hardware before the lanes rework**: a smoke run (_smoke_logs/demo_cpu_harness_scale.*, 2026-09-18, old topology — 2 TP2 engines behind the router, a fresh session per episode round-robined over two Runtimes) reaches Job succeeded at ckpt 10 with eval/deepscaler 0.33 — proof the warm-worker pool drives the real harness end-to-end. The **4 × 4 × 4 lane topology** (2026-09-23) then passed an AWS smoke (eval/deepscaler 0.358, 4 opens for 400 episodes). The **decoupled 4 : 256 : 256 pool** (2026-09-25) is verified by a live pool check without a trainer (W = M = 64, 192 episodes: 1280 ops ok, engines 48/48/47/49, tiers mixing freely); [PENDING: 4 : 256 : 256 RL smoke].

Honest scope. The offline sweep models inference/sandbox as capacity + latency; it demonstrates the concurrency (IO-bound) ceiling that warm asyncio workers lift, not the CPU/GIL ceiling of a single process (tokenization, trajectory flattening) — passing that needs separate processes or dedicated harness Runtimes, the placement question §2 leaves open. The real training path is wired and documented; the pre-rework ckpt-10 smoke and the 2026-09-23 lane smoke verified the loop, and the decoupled pool still needs its own RL smoke. Reusing a microVM across episodes is safe here only because the DeepScaleR checker reads nothing outside the episode's own workdir — tasks that change the OS stay one fresh microVM per episode (demo_cpu_terminalbench). Like the DATASET= variants it still has no committed *convergence* run, which is the open work.

7.10 `demo_cpu_tasks` per dataset in depth — the eight other real-dataset values

The eight remaining real-dataset values of `DATASET=` (each formerly its own `demo_cpu_<dataset>` folder, cloned from the MBPP one) run the **same** in-process harness, CPU AgentCore microVM sandbox, slime trainer and shared skeleton (§"The shared skeleton"). Only two things differ per value: the **`FAMILY`** and the **real Hugging Face dataset**. So everything the `DATASET=mbpp` deep-dive establishes — pipeline, the four configs, lifecycle, metric glossary — applies to each unchanged, and three properties are **shared by all eight**, stated here once so each subsection below need only give its own dataset, checker, and measured number:

- **What was run (shared).** Each is data-prepared by `prepare_data.sh`, and each was driven end-to-end by the 2026-09-18→19 smoke-plan batch as a **full 10-step run** (`NR=11` ⇒ 10 train steps + 1 held-out eval + final checkpoint at iter 10; per-run `.log`/`.csv`/`.eval.csv`/`.png` under `code/demos/_smoke_logs/`, still named after the former folders, e.g. `demo_cpu_mcp.*`) against the *real* CPU sandbox — `Job  succeeded`, ckpt 10 for all eight. This exercises the loop and checker and records a **baseline held-out solve rate**, but ten steps is still well short of convergence, so the `eval/<task>` numbers below reflect the **base Qwen3-4B policy** through the harness, *not* learning: they are **smoke baselines, not curves.**
- **Bottleneck (shared).** All eight are **generation-bound** (§3, cheap-checker CPU families): the checker is milliseconds, the policy's response is the wall-clock, so `[sglang-gen]` — not `[sbx-op]` — sets step time. **Scale inference (SGLang replicas behind the cache-aware router), never sandboxes.**
- **Plan to convergence (shared).** Run **`NR` ≈ `100`–`200`** steps (scaled to each 140–400-row split, ~10 epochs at `GBS=32`), `EVAL_INTERVAL=10`, at the same **23–78 s/step** CPU envelope (~1–3 h each), tracking the demo's own `eval/<task>` from its smoke baseline toward a plateau. Running all eight to convergence is `demo_cpu_tasks`' open work item.

7.10.1 `DATASET=gsm8k` — grade-school math word problems

- **Task / dataset:** `FAMILY=math`; GSM8K (`openai/gsm8k`), 200 train / 40 eval. Write `solution.py`, one integer answer.
- **Checker / reward: exact integer match** against the gold answer; 1.0/0.0.

- **Measured (smoke):** `eval/gsm8k` **0.90** (10-step, `NR=11`) — the base policy already solves most of this easy split, so its curve should **flatten quickly**; it is the dataset set's sanity anchor.

7.10.2 `DATASET=math500` — competition-style math

- **Task / dataset:** `FAMILY=math`; MATH-500 (`HuggingFaceH4/MATH-500`), 160 train / 40 eval.
- **Checker / reward: numeric match** (tolerance 1e-4) on the extracted answer.
- **Measured (smoke):** `eval/math500` **0.89** (10-step, `NR=11`) — harder than GSM8K, so more headroom to climb; the longer reasoning traces make its `--rollout-max-response-len` the one to size up first.

7.10.3 `DATASET=deepscaler` — RL-scale math (largest split)

- **Task / dataset:** `FAMILY=math`; DeepScaleR (`agentica-org/DeepScaleR-Preview-Dataset`), 400 train / 40 eval — the **largest** fan-out split, so nearest to a real epoch budget.
- **Checker / reward: numeric match** (tolerance 1e-4).
- **Measured (smoke):** `eval/deepscaler` **0.78** (10-step, `NR=11`) — the lowest of the math three (hardest cold), the most room to show learning; also the workload `demo_cpu_scale_lanes` reuses.

7.10.4 `DATASET=humaneval` — function-synthesis coding

- **Task / dataset:** `FAMILY=coding`; HumanEval (`openai/openai_humaneval`), 140 train / 24 eval. Write a function body.
- **Checker / reward: the row's own `check()` unit test** run in the sandbox; 1.0 iff it passes.
- **Measured (smoke):** `eval/humaneval` **0.85** (10-step, `NR=11`). Same shape as `DATASET=mbpp` (MBPP) but with the canonical HumanEval tests — a real code-execution checker rather than a numeric assert.

7.10.5 `DATASET=gretelsql` — text-to-SQL

- **Task / dataset:** `FAMILY=sql`; Gretel synthetic text-to-SQL (`gretelai/synthetic_text_to_sql`), 300 train / 40 eval. Emit one SQL query.
- **Checker / reward: result-set equality** — the query is executed against **`sqlite3`** (Python stdlib, so no extra image dep) and its rows compared to gold.
- **Measured (smoke):** `eval/gretelsql` **0.60** (10-step, `NR=11`). Cheapest checker of all (an in-process SQLite query), textbook generation-bound.

7.10.6 `DATASET=hotpotqa` — multi-hop RAG

- **Task / dataset:** `FAMILY=rag`; HotpotQA (`hotpotqa/hotpot_qa`, distractor; a 10-passage corpus per question), 300 train / 40 eval.
- **Checker / reward: SQuAD-normalized exact match** of the answer span.
- **Measured (smoke):** `eval/hotpotqa` **0.65** (10-step, `NR=11`; up sharply from the earlier `NR=3` baseline of 0.21, so its cold-start solve rate is dataset-slice-sensitive — read the single number cautiously). Still an interesting convergence target (multi-hop over 10 passages). **Note:** it is the one mild exception to "generation-bound" — the 10-passage observations grow the *context* each turn, but the checker is still trivial, so it is generation/**context**-bound, still not sandbox-bound.

7.10.7 `DATASET=xlam` — function-calling / tool use

- **Task / dataset:** `FAMILY=tool use`; xLAM-60k (`minpeter/xlam-function-calling-60k-parsed`), 300 train / 40 eval. Emit the tool call(s) for a request.
- **Checker / reward: arg-normalized gold-call match** (order-insensitive over arguments), RLVR.
- **Measured (smoke):** `eval/xlam` **1.00** (10-step, `NR=11`) — the base policy already nails this smoke slice, so this demo's value is verifying the tool-call parsing/checker path end-to-end more than showing a climb.

7.10.8 `DATASET=mcp` — MCP-style tool schemas

- **Task / dataset:** `FAMILY=tool use`; ToolACE (`Team-ACE/ToolACE`, MCP-style schemas), 300 train / 40 eval — the same family as `xlam` but against MCP-shaped tool definitions.
- **Checker / reward: arg-normalized gold-call match**, RLVR.
- **Measured (smoke):** the 10-step smoke-plan run (`_smoke_logs/demo_cpu_mcp.*`, from the former per-dataset folder, `NR=11`) recorded **`eval/mcp 1.00`** on the held-out set — a *baseline solve rate*, not convergence: against a small tool-call gold set the base policy already matches, so read it as "the checker and MCP-style tool loop run end-to-end," not as a learned gain. The shared plan above applies.

7.11 `demo_cpu_tasks` mixes in depth — one interleaved multi-family stream

Same demo and skeleton as the single-dataset runs (in-process harness, CPU microVM sandbox, slime), but a mix differs in a dimension none of the others touch: the **data composition**. Instead of pinning one family per run, `DATASET=a,b,c` interleaves those datasets' files round-robin, and the prebuilt `DATASET=mixed` (`build_mixed_datasets.py`) is a single `mixed_train.jsonl` / `mixed_eval.jsonl` that **interleaves math + SQL + CTF + datasci** — so every GRPO rollout step draws a **multi-family batch**, the setting a generalist policy is actually trained in.

- **Why no architectural change is needed.** Each row is **self-describing** (`metadata.task = {family, task_id, prompt, spec, ⋯}`), and `generate._task_from_sample` rebuilds the right `EpisodicTask` subclass **per prompt**, so `run_episode` already dispatches prompt, seed files, checker, and reward per row (§"One harness, many task families"). The sandbox is task-agnostic, so a math, a SQL, and a CTF episode are just different files in three identical, isolated microVMs. And crucially (§"`demo_cpu_tasks` mixes"), **one Runtime, many sessions** still holds for a stdlib-only mix — its families share the one generic image, so mixing multiplies *sessions*, not *Runtimes*; `DATASET=mixed`'s `datasci` adds exactly one pandas Runtime, picked per episode by the task's `image` tag.
- **Why the reward scales don't clash.** GRPO normalizes advantage **within a group** of `n_samples_per_prompt` repeats of the *same* prompt — hence the

same family — so the different families' reward magnitudes are never compared directly; only the prompt **sampler** mixes (across groups within a step). The builder offers a balanced round-robin interleave or a proportional shuffle, and replicates the pool so one epoch spans the run (keeping save-only-at-end intact).

- **Run status — honest.** The mixed dataset is now **built** (the builder produced a 174-row `mixed_train.jsonl` / 9-row `mixed_eval.jsonl` interleave), and a **smoke run (as the former `demo_cpu_mixtask`) reaches `Job succeeded` at ckpt 10** with `eval/mixed` **0.67** (raw_reward 1.0 rollouts present). So the multi-family stream is now **loop-verified end-to-end** — a step past the single-dataset smokes — but it is a short smoke, not convergence, and it emits only a **single aggregate `eval/mixed`**, not the per-family breakdown below.
- **What it would measure.** Because it is generation-bound like the CPU families it mixes, the scaling story is the single-dataset runs' (scale inference). Its *distinct* signal — still open — is a **per-family `eval/<family>` breakdown** rising together under one shared policy, the evidence that a single run is training a generalist rather than trading one family off against another; the committed smoke reports only the aggregate. Note the pre-built `mixed` seeds are the **hand-authored templated `task_families.py` families**, not the real HF datasets, so it demonstrates the multi-family training *mechanics*, not dataset-scale coverage; a comma list of real datasets (e.g. `DATASET=gsm8k,gretelsql,hotpotqa,xlam`) is the dataset-scale version, not yet run. Emitting the per-family eval split and running it to a multi-family plateau is its open work (§"Experiment plan").

8 Open problems in production agentic RL — infrastructure and algorithms

The workflow in this report is deliberately small — one policy, an in-process harness, a per-episode sandbox. Running agentic RL at *frontier* scale surfaces a set of recurring hard problems that the open reports (2024–2026) now describe in detail. This section catalogs them in two groups — infrastructure/systems and algorithms — with the mitigations people actually ship, and notes where this project's design already meets one and where it remains open.

8.1 Infrastructure and systems

Long-tail stragglers and partial rollouts. Rollout lengths are heavily right-skewed, so a synchronous step idles the whole cluster waiting on the last few long trajectories — and generation is more than 90 % of RL runtime [20]. Mitigations: *partial rollouts* that cap output and resume the unfinished portion next iteration (Kimi k1.5 [17]; Kimi K2 [5]); *active partial rollouts* that over-provision and recycle interrupted requests (APRIL, +22–44 % rollout throughput, now in slime [28]); *interruptible generation* that resumes decoding after a weight update (AReaL [18]); *skewness-aware routing* of long prompts (StreamRL [20]); and *dynamic sampling* that drops zero-gradient groups (DAPO [14]).

Weight-sync cost and staleness. Trainer and inference are separate backends; after each optimizer step, new weights must reach every rollout replica. Synchronously this is a serialization bubble; at 1T scale it moves hundreds of GB. Mitigations: *fast weight transfer* (Moonshot's checkpoint-engine updates all of Kimi-K2's ~1T parameters in ~20 s [29]); *zero-redundancy resharding* between train and generate layouts (veRL / HybridFlow [19]); and *decoupled, "one-step-off" async RL with a bounded staleness* and a staleness-aware objective (AReaL, up to ~2.77× [18]; Asynchronous RLHF [21]). *This project uses slime's fully-async rollout (MODE=B), so staleness is the accepted cost — and it is exactly the signal that is instrumented-yet-unrecorded here (§"The shared skeleton").*

Goodput vs MFU, and placement. An iteration interleaves a bandwidth-bound, long-tailed generation phase with a compute-bound training phase, so one engine's GPUs idle while the other works — which is why MFU is misleading for RL and practitioners reason in *goodput* (useful samples per GPU-second). *This project's measured MFU of 2–4 % (§`demo_cpu_tasks`, `DATASET=mbpp`) is a concrete instance of that gap.* Mitigations: *colocate-with-offload* (veRL / HybridFlow, the frontier default [19], used by Kimi K2 [5]) vs *disaggregated placement* for heterogeneous or elastic hardware (StreamRL, up to 2.66× [20]; slime supports both). *Here the 4-train + 4-inference colocation is the simple default; disaggregation is the lever once the sandbox or a second cluster enters the picture.*

Scaling execution sandboxes to tens of thousands. Every step runs untrusted code in an isolated sandbox, so the sandbox layer — not the GPU — often becomes the bottleneck: constant create/destroy, strong isolation, real startup cost. Kimi K2 runs more than 10,000 concurrent Kubernetes sandboxes for its coding / SWE RL [5]. Mitigations: *microVM isolation* (Firecracker-class, sub-second boot) and *managed*

remote code-exec backends (E2B, Modal). *This is precisely AgentCore Runtime's per-session microVM — the seam this whole report builds on; the project scales it by round-robining a pool of Runtime ARNs (sticky per episode) and, on GPU, the Ray fractional farm (§"`demo_gpu_farm_fractional` and `demo_gpu_farm_whole` in depth").*

Cache-aware routing. Rollout throughput lives on prefix KV-cache reuse; a round-robin balancer scatters prefix-sharing requests and collapses the hit rate — acute in agentic RL, where a group of trajectories shares one task prefix and multi-turn rollouts grow a long shared prefix. Mitigations: RadixAttention plus SGLang's cache-aware router (~1.9× throughput; hit rate ~20 %→~75 %) [27], and session-affinity routing for multi-turn (slime `consistent_hashing` [9]). *This is the reasoning behind §"Scaling the loop": one cache-aware SGLang router for inference, round-robin only on the sandbox side.*

Fault tolerance. One step couples training + inference + reward + thousands of live sandboxes, so the blast radius is large. Mitigations: *async checkpointing*; *worker heartbeats* (slime's `/health_generate` with restart-and-re-sync, `--use-fault-tolerance` [9]); and *sandbox recycling* with timeouts and rate limits so one bad tool call does not abort a rollout. *The project's two resilience mechanisms (§"`HEARTBEAT=1`", §`demo_gpu_single` `SNAPSHOT_EVERY=1`) address the sandbox-side failure modes — idle reclamation and instance recycle.*

8.2 Algorithms

The train/inference logprob mismatch and off-policy correction. Even with identical parameters, the inference engine and the trainer assign different token probabilities (differing parallelism, precision, kernels), so "on-policy" RL is silently off-policy — biasing gradients and risking entropy collapse, and worse for long responses. Mitigations: *Truncated Importance Sampling* (clip π_learner/π_sampler, now in veRL) [22]; *fixing the precision root cause* by reverting to FP16 [23]; and *decoupled / staleness-aware PPO* [18]. *This project's TITO property (the proxy records the exact sampled `token_ids` + logprobs, no re-tokenization — §"Token-in-token-out") removes the tokenization half of the mismatch, but not the engine-vs-trainer numeric half; TIS is the missing piece, and the importance-sampling ratio is instrumented-but-unrecorded here.*

GRPO and its variants. GRPO drops PPO's critic by normalizing advantages within a group of K sampled responses [13], but has documented failure modes (entropy collapse, length bias, zero-gradient groups, MoE instability). Upgrades: *DAPO* (clip-higher ε_high≈0.28, dynamic sampling, token-level loss, overlong reward shaping) [14]; *Dr. GRPO* (removes the length and std normalization that inflate wrong-answer length) [16]; *GSPO* (a sequence-level, length-normalized importance ratio that stabilizes MoE RL and underpins Qwen3) [15]. *This project trains with GRPO; these are the drop-in improvements as runs scale up.*

Credit assignment over long horizons. An agentic episode is many steps but the verifiable reward is sparse and terminal, so trajectory-level GRPO mis-credits the actions inside a failed or successful trajectory. Mitigations: *hierarchical / turn-level grouping* (GiGPO, +9–12 % over GRPO on ALFWorld / WebShop) [26]; entropy-aware step attribution; and denser process rewards derived from outcome labels alone. *The demos use a binary end-of-episode reward with `EPISODE_MAX_TURNS=3`; long-horizon families (SWE, web) are where turn-level credit becomes necessary.*

Reward hacking and verifier trust. Any learned reward can be gamed, so frontier labs prefer rule-based verifiable rewards — DeepSeek-R1 explicitly avoids neural reward models to prevent reward hacking [2]. But verifiers are not automatically trustworthy: "Spurious Rewards" shows RLVR with *random* rewards still improved a Qwen math model, because a clipping bias amplifies pretraining behaviors rather than reward correctness — and the effect is model-dependent [24]. *The project's rewards are binary RLVR checkers run inside the sandbox (no LLM judge), which sidesteps neural-RM hacking; the caution is that a benchmark gain does not by itself prove the checker is doing the work.*

Entropy collapse. During long RL, entropy drops early and exploration dies; performance is effectively "traded from policy entropy" [25]. Mitigations: clip-higher [14]; covariance-targeted regularization (Clip-Cov / KL-Cov) that intervenes only on the few high-covariance tokens driving entropy decay [25]; and KL control or removal.

Curriculum, dynamic sampling, and the discard ratio. All-correct or all-wrong groups give zero advantage → wasted compute. Mitigations: *dynamic sampling* to keep only groups with 0 < correct < G (DAPO [14]) and *pass-rate difficulty curricula* (AdaRFT, up to 2× less training time [30]). *This is exactly where the project's `discard_ratio` metric would come alive — it is defined (goodput depends on it) but zero in the current runs.*

8.3 Takeaways

Most of these are already partly handled by the stack this report uses: slime provides async rollout, fault-tolerant SGLang workers, and cache-aware routing; AgentCore provides the isolated, scalable sandbox. The open work for a production run concentrates in three places — (1) turning on and *recording* the off-policy signals (staleness, importance-sampling ratio, discard ratio) and adding TIS; (2) moving from GRPO to a variant (DAPO / GSPO) as scale and response length grow; and (3) turn-level credit assignment for the long-horizon families. These are algorithm choices layered on top of the infrastructure the demos already exercise.

9 Summary

- **Can:** an isolated per-session sandbox for untrusted model output (CPU by default), fully in-account (IAM/SigV4, VPC), long sessions, bring-your-own container, and **GPU on Instances**; a clean one-hook integration into slime with the harness in-process.
- **Cannot:** a microVM has no GPU; there is no cross-session GPU fractioning (and no MIG on L4); and a session cannot exceed the microVM's 8 h cap without moving to Instances (14 d).
- **How to run:** build a sandbox image → deploy a Runtime (or a GPU capacity provider) → seed a task → `invoke_agent_runtime` per tool call → the in-sandbox checker gives the reward → slime trains via a single `generate` hook. CPU↔GPU is one env var.
- **Verified (what the thesis rests on):** AgentCore works as the RL sandbox for slime on every storyline path:
  - **the default runtime**: gsm8k and libs trained on the managed Code Interpreter with no image (2026-09-25; `eval/gsm8k` 0.90, `eval/libs` 1.0);
  - **custom images at scale**: 218 per-task Runtimes, with the cold start fully hidden by pre-open;
  - **the warm-start mechanisms**: measured live by the `warmstart/` probe;
  - **a warm CPU Instance pool**: 2 instances, warm opens 121–328 ms, Landlock isolation 22/22 live on the m5;

  - **decoupled rollout tiers**: the old 1:1:1 lanes held on AWS; the 4 : 256 : 256 pool passed a live pool check, RL smoke pending;
  - **the three GPU sharing modes**: the fractional farm re-run 2026-09-25 at 0.65, and the whole-GPU farm at 0.65 and 0.70.

  Two demos carry committed convergence-shaped curves (`demo_cpu_tasks DATASET=mbpp`, `demo_gpu_single`); the rest are loop-verified 10-step smokes.

  Honest caveats:

  - `demo_cpu_branching` has its three blockers fixed and verified without a trainer, but its RL smoke is pending ([PENDING: branching RL smoke fan_out and reward]);
  - the `demo_gpu_single` 2026-09-25 remote smoke failed at step 4 on runtime 500s ([PENDING: root cause of the demo_gpu_single remote-smoke op=open 500s]);
  - the instance pool's per-uid isolation failed live (AgentCore drops every capability) and was replaced by a Landlock jail, 22/22 live; its RL smoke passed (eval 0.7875, 2 cold boots, pre-jail image);
  - no demo has yet been run to convergence. This is a capability demonstration, not a convergence result.

- **Grounding:** the concepts and lifecycle are cross-checked between the official AgentCore documentation [D1–D13, D17] and AWS's own SDK, CLI, and samples [D14–D16]; the code is this project's `agentcore_slime_demo/` (harness, sandbox, tasks), `deploy/` (Dockerfiles + `deploy_two_runtime.py`), and `demos/`.

10 References — papers & models

Agentic RL & models. [1] Agent Lightning — Harnessed Agentic RL. https://arxiv.org/abs/2508.03680 ·[2] DeepSeek-R1. https://arxiv.org/abs/2501.12948 ·[3] Qwen3 Technical Report. https://arxiv.org/abs/2505.09388 ·[4] Llama-Nemotron. https://arxiv.org/abs/2505.00949 ·[5] Kimi K2. https://arxiv.org/abs/2507.20534 ·[6] SWE-bench. https://arxiv.org/abs/2310.06770 ·[6b] SWE-bench README (arm64 "experimental"; `--task-repo` local Buildx build). https://github.com/SWE-bench/SWE-bench ·[7] KernelBench. https://huggingface.co/datasets/ScalingIntelligence/KernelBench ·[8] MBPP. https://arxiv.org/abs/2108.07732 ·[9] slime. https://github.com/THUDM/slime ·[10] AceReason-Nemotron: Advancing Math and Code Reasoning through RL. https://arxiv.org/abs/2505.16400 ·[11] LiveCodeBench. https://arxiv.org/abs/2403.07974 ·[12] NVIDIA Nemotron Post-Training Dataset v2. https://huggingface.co/datasets/nvidia/Nemotron-Post-Training-Dataset-v2

11 References — AgentCore docs (1/2)

Official AgentCore docs. [D1] What is AgentCore. https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/what-is-bedrock-agentcore.html ·[D2] Host with Runtime (isolation, compute types). https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/agents-tools-runtime.html ·[D3] Runtime how it works (idle 15 m / max 8 h). https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/runtime-how-it-works.html ·[D4] Lifecycle settings. https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/runtime-lifecycle-settings.html ·[D5] Instances (GPU). https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/runtime-instances-how-it-works.html ·[D6] Code Interpreter. https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/code-interpreter-tool.html ·[D7] Code Interpreter session. https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/code-interpreter-session-characteristics.html

12 References — AgentCore docs (2/2)

[D8] InvokeAgentRuntime. https://docs.aws.amazon.com/bedrock-agentcore/latest/APIReference/API_InvokeAgentRuntime.html ·[D9] CreateAgentRuntime. https://docs.aws.amazon.com/bedrock-agentcore-control/latest/APIReference/API_CreateAgentRuntime.html ·[D10] NetworkConfiguration. https://docs.aws.amazon.com/bedrock-agentcore-control/latest/APIReference/API_NetworkConfiguration.html ·[D11] VPC interface endpoints. https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/vpc-interface-endpoints.html ·[D12] StopRuntimeSession. https://docs.aws.amazon.com/bedrock-agentcore/latest/APIReference/API_StopRuntimeSession.html ·[D13] Observability. https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/observability.html ·[D17] Runtime service contract (port 8080, /invocations, /ping). https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/runtime-service-contract.html

13 References — AgentCore SDK, CLI & samples

AWS code, cross-checked against the docs. [D14] AgentCore samples (`awslabs/agentcore-samples`). https://github.com/awslabs/agentcore-samples ·[D15] AgentCore Python SDK (`bedrock-agentcore`; `BedrockAgentCoreApp` / `@app.entrypoint`). https://github.com/aws/bedrock-agentcore-sdk-python ·[D16] AgentCore CLI (`@aws/agentcore`, recommended) https://www.npmjs.com/package/@aws/agentcore ·legacy starter toolkit https://github.com/aws/bedrock-agentcore-starter-toolkit

14 References — production agentic RL (open problems)

RL algorithms & systems. [13] DeepSeekMath (GRPO). https://arxiv.org/abs/2402.03300 ·[14] DAPO. https://arxiv.org/abs/2503.14476 ·[15] GSPO (Group Sequence Policy Optimization). https://arxiv.org/abs/2507.18071 ·[16] Understanding R1-Zero-Like Training (Dr. GRPO). https://arxiv.org/abs/2503.20783 ·[17] Kimi k1.5. https://arxiv.org/abs/2501.12599 ·[18] AReaL (asynchronous RL system). https://arxiv.org/abs/2505.24298 ·[19] HybridFlow (veRL). https://arxiv.org/abs/2409.19256 ·[20] StreamRL (disaggregated stream generation). https://arxiv.org/abs/2504.15930 ·[21] Asynchronous RLHF. https://arxiv.org/abs/2410.18252 ·[22] Truncated Importance Sampling / train–inference mismatch (Yao et al.). https://fengyao.notion.site/off-policy-rl ·[23] Defeating the Training–Inference Mismatch via FP16. https://arxiv.org/abs/2510.26788 ·[24] Spurious Rewards. https://arxiv.org/abs/2506.10947 ·[25] The Entropy Mechanism of RL. https://arxiv.org/abs/2505.22617 ·[26] GiGPO (hierarchical group advantages). https://arxiv.org/abs/2505.10978 ·[27] SGLang / RadixAttention.

https://arxiv.org/abs/2312.07104 — cache-aware router: https://lmsys.org/blog/2024-12-04-sglang-v0-4/ ·[28] APRIL (active partial rollouts). https://arxiv.org/abs/2509.18521 ·[29] Moonshot checkpoint-engine. https://github.com/MoonshotAI/checkpoint-engine ·[30] AdaRFT (difficulty curriculum). https://arxiv.org/abs/2504.05520